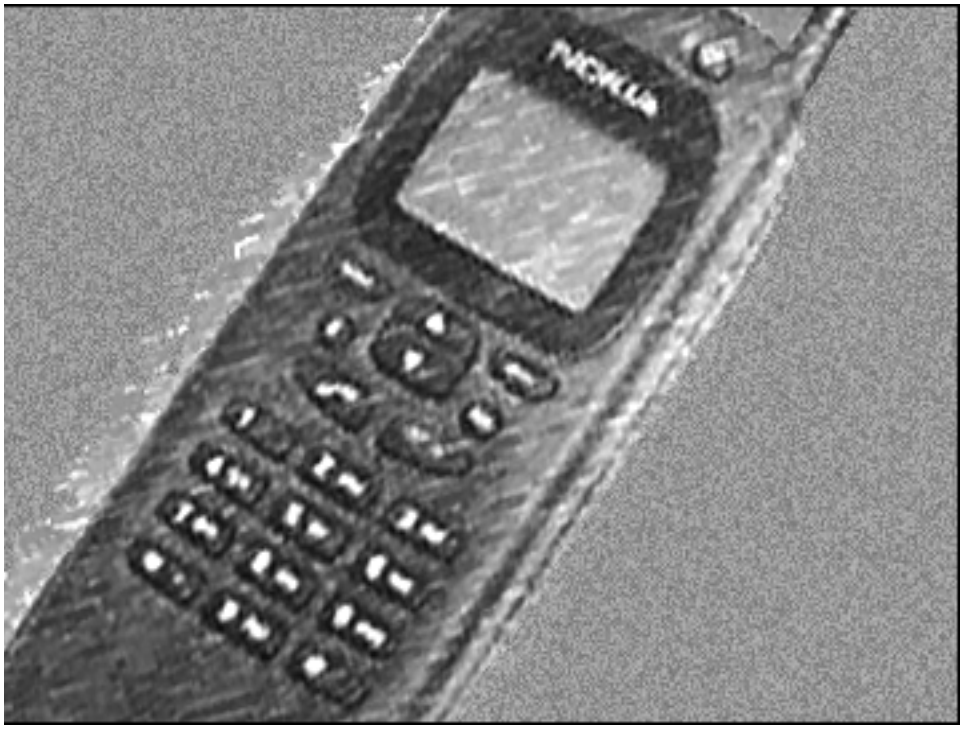
Nokia 9000 Communicator



Nokia 9000 Communicator SDK

Version N9000C 1.1



Nokia 9000 Communicator



Geoworks, Inc.
Alameda, CA



Geoworks provides this publication “as is” without warranty of any kind, either express or implied, including but not limited to the implied warranties of merchantability or fitness for a particular purpose. Geoworks may revise this publication from time to time without notice. Geoworks does not promise support of this documentation. Some states or jurisdictions do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you.

Copyright © 1996 by Geoworks, Incorporated.
All rights reserved. Published 1996
Printed in the United States of America

The material presented in the Virtual Serial Library chapter is Copyright © 1996 Nokia Mobile Phones, and is presented here with permission.

Geoworks®, Geoworks Ensemble®, Ensemble®, GEOS®, PC/GEOS®, GeoDraw®, GeoManager®, GeoPlanner®, GeoFile®, GeoDex® and GeoComm® are registered trademarks of Geoworks in the United States and Other countries.

Geoworks® Pro, PEN./GEOS, Quick Start, GeoWrite, GeoBanner, GeoCrypt, GeoCalc,GeoDOS, Geoworks® Writer, Geoworks® Desktop, Geoworks® Designer, Geoworks® Font Library , Geoworks® Art Library, Geoworks® Escape, Lights Out, and Simply Better Software are trademarks of Geoworks in the United States and other countries.

Trademarks and service marks not listed here are the property of companies other than Geoworks. Every effort has been made to treat trademarks and service marks in accordance with the United States Trademark Association's guidelines. Any omissions are unintentional and should not be regarded as affecting the validity of any trademark or service mark.



Nokia 9000 Communicator



1 Introduction 11

1.1 Nokia 9000 Features 13

1.2 Developing on the N9000 15

1.3 Setting Up the SDK 16

1.3.1 What You'll Need 16

1.3.2 Setting Up the Hardware 17

1.3.3 Installing the Software 18

1.3.4 Testing the Configuration 19

1.3.5 Troubleshooting Communications 27

1.4 Debugging on the Device 32

2 N9000C UI 35

2.1 FoamSubApplicationClass 37

2.1.1 FoamSubApplicationClass Instance Data 38

2.1.2 Using a FoamSubApplication Object 39

2.2 ComplexMonikerClass 40

2.2.1 ComplexMoniker Instance Data 41

2.2.2 ComplexMoniker Usage 49

2.2.3 ComplexMoniker Messages 51

2.3 Convenient Dialog Routines 53

2.4 FlashingNoteClass 54

2.4.1 FlashingNote Instance Data 55

2.4.2 Setting Up the Template 56

2.4.3 Initiating and Destroying the Flashing Note 58



2.5 FoamProgressDialog 58
2.5.1 FoamProgressDialog Instance Data 59
2.5.2 Setting Up the Template 60
2.5.3 Using a FoamProgressDialog 61
2.6 Routines for Lists 63

3 SMS Guide 65
3.1 SMS Overview 67
3.2 Using the Mailbox Library 68
3.2.1 Mailbox Overview 68
3.3 Sending an SMS Message 69
3.3.1 Sending and the Mailbox Library 70
3.3.2 Using the SMS Mailbox API 70
3.4 Receiving an SMS Message 87
3.4.1 SM Receive and the Mailbox Library 87
3.4.2 Receiving an SMS Message 88

4 Contacts 93
4.1 Setting up a Contact List 97
4.1.1 Multiple-Selection ContactLists 100
4.1.2 Remembering the Search 100
4.1.3 Detecting Unknown Phone Numbers 100
4.2 Choosing Contacts From a Log 101
4.3 Using a Dedicated Contact 103
4.4 Logging Calls 106
4.5 To Get More Info on a Contact 108
4.6 The User's Own Data 111



5 Distributing Your Application 113
5.1 Package Description File 115
5.2 Setting Up a PC Server 118
5.3 Setting Up a Web Server 119

6 Memory Levels 121
6.1 Memory Levels 123
6.2 Modifying Files on the FFS 123
6.3 Operations that Consume FFS Space 124
6.4 Utility Space Routines 125

7 The Socket Library 127
7.1 Introduction 129
7.2 Simple Example 134
7.3 Addresses 139
7.3.1 Letting the User Choose an Address 140
7.3.2 Managing Addresses 144
7.3.3 Hardware Ports 144
7.4 Making a Connection 145
7.5 Accepting a Connection 146
7.6 Loading on Connection 149
7.7 Sending and Receiving Data 150
7.7.1 Urgent Data 150
7.7.2 SocketSend() and SocketRecv() 151
7.7.3 Options 154
7.8 Closing the Connection 155



7.9 Checking the Connection 156
7.10 Domain-Specific Information 158
7.10.1 TCP/IP—GEOS Specific 158
7.10.2 TCP/IP—Standard 163
7.10.3 Loopback 164

8 Access Point Library 165
8.1 Access Points 167
8.2 Letting The User Choose 168
8.3 Searching For an Access Point 169
8.4 Locking and Unlocking 171
8.5 Adding an Access Point 171
8.6 Deleting an Access Point 171

9 Modem Connections 173
9.1 Introduction 175
9.2 Connecting 176
9.3 Sending Data 178
9.4 Receiving Data 180
9.5 Disconnecting 183

10 Virtual Serial Library 185
10.1 Overview 188
10.2 Function Reference 189
10.3 Differences from Normal Serial Driver 206
10.3.1 Differences in assembly language 206
10.3.2 Differences in C -language interface 207



11 E-mail 209
11.1 Example: Sending a Text Message 211
11.1.1 Essentials 211
11.1.2 Variations 218
11.2 The Preparation Sequence 221
11.3 Mailbox Library 224
11.4 Communicator-Specific Features 227
11.5 Useful Mailbox Messages 228

12 Fax Sending 235
12.1 Destination Phone Number 237
12.2 Drawing the Contents 240
12.2.1 Declaring the Objects 240
12.2.2 Supplying the Document's Name 241
12.2.3 Supplying the Fax's Contents 241

13 Update 245
13.1 Software Version Number 247
13.2 User's Name 248
13.3 IMEI Code Number 249
13.4 New SMS Header 249
13.5 New Standard Monikers, Bitmaps 252
13.6 Non-exclusive Lists 252
13.7 Incremental Searches 253
13.8 Contact's URL and Password 253
13.9 ContactList Searches 253
13.10 Unknown Phone Numbers 254





Introduction



1

1.1 Nokia 9000 Features 13
1.2 Developing on the N9000 15
1.3 Setting Up the SDK 16
1.3.1 What You'll Need 16
1.3.1.1 Hardware You'll Need 16
1.3.1.2 Software You'll Need 17
1.3.2 Setting Up the Hardware 17
1.3.3 Installing the Software 18
1.3.4 Testing the Configuration 19
1.3.5 Troubleshooting Communications 27
1.4 Debugging on the Device 32





Welcome to the world of GEOS! If you're planning on writing software for the Nokia 9000 Communicator smart phone device, then this is the book for you.

1.1 Nokia 9000 Features

The Nokia 9000 Communicator is a new kind of product: a smart phone. It's a cellular phone with data-integrated personal information manager applications. It can run add-on applications—applications which you write.

Form Factor

Dimensions are:

- 17 cm long (7 in)
- 6 cm wide (2 in)
- 4 cm thick (2 in)
- 400 grams (14 oz)—surprisingly light for such a power-packed device!

Hardware

- The phone functions are based on a Nokia 2110i.
- The communicator runs on a 24 MHz 80386 chip.
- The communicator memory consists of 4 Mb Flash ROM (containing the GEOS OS and applications), 2 Mb of DRAM (for program execution) and 2 Mb of Flash RAM (for data storage).

I/O and Communications

- The large display consists of an LCD 640x200 pixels with 4 levels of grayscale. The screen is neither touch-sensitive nor pen operable.
- The keyboard is a standard QWERTY keyboard. English, French German and Scandinavian versions are currently available.



- Communication is through the GSM digital network, which can transmit cellular voice, data (and fax) and SMS alpha-numeric pages.
- E-mail conforms to the SMTP and business card standards.
- Communication with the internet via TCP/IP and PPP.
- Infrared communication with outside devices via IrDA
- Serial line communication via a RS-232 compatible connection.

Power

- Power is supplied via Nokia Lithium-ion rechargeable battery packs. Charging takes 2.5 hours.
- Battery life can sustain continued conversation for two hours, 30 hours of standby mode, or one week, if left off. Beyond that, data should be backed up.

Built-In Programs

The Communicator comes with the following programs built in. Add-on applications may to work with or around these built-ins, depending on the nature of the add-on application.

- Telephone, Fax
- SMS
- Contacts
- Notes
- Internet Mail, WWW, Telnet, Terminal
- Calendar
- Calculator, Clock, Composer
- Document Outbox

1.2 Developing on the N9000

If you're already familiar with GEOS development, then developing for the Nokia 9000 Communicator is simple. (If you aren't, you'll probably want to turn to the Tutorial book after finishing this chapter.) Simply add new UI pieces and understand the following new libraries:

- SMS Library
- Mailbox Library
- Contact DB Library
- Foam Library

To get up to speed on developing for the Nokia 9000 Communicator, you'll want to learn about the following topics:

- **Basics**
  You'll want to take some time to set up the SDK (if you haven't already done so). You'll want to explore the Nokia 9000 Communicator's built-in programs: use the SDK's N9000 emulator software or a real Nokia 9000 Communicator to do so.
- **Using the UI elements**
  The Foam library contains object classes that have been custom-made for the Nokia 9000 Communicator. For example, your application's object should be a FoamSubApplication, instead of a GenApplication—this special class was set up to interface with the Communicator's program launchers. You will learn more about this in "Communicator UI," Chapter 2.
- **Using the SMS API to send messages**
  The simplest way for a program to send and receive messages via the phone is with SMS—the Short Messaging System. This system allows a program to send a 151-character message to any GSM phone number. You will learn more about this in "SMS Guide," Chapter 3.
- **The Contact Database**
  The Contact Database contains "address book" information—phone numbers, e-mail addresses, and other contact information for the user's contacts. Programs may look up this information and/or modify it. To learn about this library, see "Contacts," Chapter 4.

- **Loading the software onto the device**
  Once you've written your application, you'll want to get it into the hands of your users. You can make your application available on the worldwide web—any user with internet access will be able to download your program with their Communicator. You may take advantage of the Communicator's desktop connectivity software—if the user has a PC and your program files are on it, the user can download them to a Communicator very easily. To find out how to go about this, see "Distributing Your Application," Chapter 5.

1.3 Setting Up the SDK

Setting up your system involves setting up your hardware and installing software on it. This chapter explains how to do both and then provides a sequence of steps and troubleshooting tips to test your setup.

1.3.1 What You'll Need

This development kit includes much of the software you will need to write and debug GEOS applications. It does not, however, include a C compiler or the hardware you will need. Make sure you have all of the items on the following lists before proceeding.

1.3.1.1 Hardware You'll Need

This kit provides no hardware. To set up your system, you will need at least the following:

Development PC
The computer on which you will do your programming is referred to as the *Development PC*, the *Host PC*, or occasionally the *Workstation*. This PC must have an 80386 or more powerful processor and preferably eight or more megabytes of RAM. It should have a hard disk of at least 80 megabytes.

Target PC The computer on which you will run GEOS and on which you will test your applications is referred to as the *Target PC* or the

Test PC. This may be any Intel-compatible PC. Geoworks recommends that you have two test PCs: A powerful machine which you can debug on quickly, and an 24 MHz 386-class machine on which you can test your application's speed. When this isn't possible, you should try to have a more powerful machine in order to test various video drivers and memory configurations with your application.

Serial Cable(s) with Null Modem

The Host and Target PCs must be connected via serial link. Most developers try to keep their host and target close to one another; a six-foot cable should be adequate. A null modem is required in the serial connection between the host and target PCs. Without it, the send and receive pins on one of the machines will be reversed. Note that Lap-link cable includes this null-modem connection. Depending on the cables and connectors you have, you may need one or more serial line gender changers or other adapters.

1.3.1.2 Software You'll Need

As stated above, this kit does not provide all the software you'll need. Both the host and target machines must be loaded with some version of DOS, and you will also need to have a C compiler.

Currently, Borland C/C++ 3.0, 3.1, 4.0, and 4.5 compilers are supported for GEOS development. However, C++ code is not supported.

You may also want to use a task-switching environment on the host machine to make it easier and quicker to switch between coding, compiling, and debugging.

1.3.2 Setting Up the Hardware

First, set up your two PCs according to their included instructions (if any.

Next, connect the two machines via a null modem serial connection. Remember which serial ports you used on both machines; you will need to know when installing the software. You may need to add in gender changer(s), 9-to-25 pin adapter(s), or other connectors, depending on your



machines and your cables. A simplified diagram of the connection is shown in Figure 1-1.

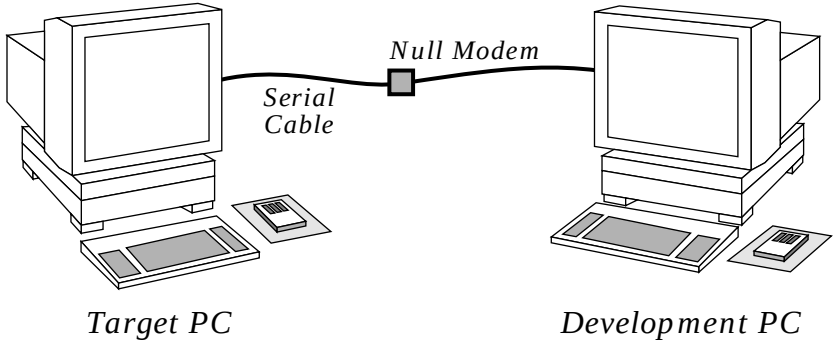


Figure 1-1 *Workstation Setup*
The test and development PCs communicate via a null modem serial connection.

1.3.3 Installing the Software

Once you have finished setting up your machines and have connected the serial cable, you must install the GEOS development kit. Place the SDK CD-ROM into your host machine's CD-ROM drive, change to that drive, and run the INSTALL program (DOS) or the SETUP program (MS Windows).

The installation program will ask questions so it can set up the DOS environment for use with the GEOS development tools.Answering these questions incorrectly can cause the SDK to not function properly.

Important: You must set both machines to use the same baud rate. They may use different COM ports, but they must have the same baud rate.

After you have installed the two disk sets, you must install your C compiler on the host (development) machine (if you have not already done so). Follow the instructions provided with your compiler.

Finally, try out both the error-checking and the non error-checking versions of GEOS on the target machine. To run the error-checking version, type

```
C:\>cd n9000ec
C:\N9000EC>geosec
```

This should start up an emulator for the Nokia9000 Communicator. If something goes wrong, you may need to re-install the target software.

To run the non error-checking version, type

```
C:\>cd n9000nc
C:\N9000NC>geos
```

Again, if the set-up does not go smoothly, try reinstalling the disks. If this does not work, check that you are using the proper hardware configurations. If one or both of the two N9000 directories does not exist, try reinstalling the target software.

1.3.4 Testing the Configuration

Once you have finished installing all the software, your system should be ready for you to begin programming. To check it, however, run through the following sequence of steps. You will learn later exactly what these steps do in more detail, but for now it is sufficient just to try them.

Host Machine: cd \pcgeos\appl\sdk_c\hello

On the host machine, change directories:

```
C:\>cd \pcgeos\appl\sdk_c\hello
```

If this directory does not exist, check that the \PCGEOS directory exists. If it doesn't, check which directory you installed into, and try that path instead. If this still doesn't work, make sure that *something* has been installed on your hard drive; if nothing has, you should try reinstalling the host disk set.

Host Machine: mkmf

Next, run the "make Makefile" program on the host machine with the **mkmf** command:

```
C:\PCGEOS\APPL\SDK_C\HELLO>mkmf
Don't forget to run "pmake depend"
```




Your directory should now contain a file called MAKEFILE.

- If you get a "Bad command or file name" error, make sure that your path includes C:\PCGEOS\BIN (or C:*DIR*\BIN, where *DIR* is the name of the root directory in which you installed the host portion of the SDK).

Host Machine: pmake depend

Next, create a dependencies file with the **pmake depend** command. The output you get should be similar to that shown below.

```
C:> PCGEOS\APPL\SDK_C\HELLO>pmake depend
makedpnd  ENDCMODULES  ENDASMMODULES GOC GOC goc -M -D__GEOS__ -IDEPENDS  -I.
-IC:\PCGEOS\INCLUDE  -IC:\PCGEOS\INCLUDE\ANSI  -I- -IDEPENDS  -I.
-IC:\PCGEOS\INCLUDE  -IC:\PCGEOS\INCLUDE\ANSI    -w -l -cb ENDFLAGS CPP BORLAND
cpp -D__GEOS__ -Ot -c -v -y -w -g255 -w-amp -w-pin -w-cln  -w-sig -w-sus -i200 -ml
-IDEPENDS  -I.   -IC:\PCGEOS\INCLUDE  -IC:\PCGEOS\INCLUDE\ANSI   ENDFLAGS
HELLO.GOC    ENDFILES
Processing HELLO.GOC...
Processing HELLO.CPP...
Borland C++ Preprocessor Version 3.1 Copyright (c) 1992 Borland International
hello.cpp:

        Available memory 3891140
findlbdr HELLO.GP DEPENDS.MK HELLOEC.GEO HELLO.GEO
```

There should now be a file in your directory called DEPENDS.MK.

- If you get an error like

  ```
  "makefile". line 13: Could not find GEOS.MK
  "LOCAL.MK", line 3: Could not find GEODE.MK
  Fatal errors encountered -- cannot continue
  ```

  you should make sure that your ROOT_DIR variable is set correctly. ROOT_DIR should be the name of whatever directory you installed the host files to. Normally this is C:\PCGEOS.

- If you get the error

```
C:\PCGEOS\BIN\PMAKE.EXE: Can't figure out how to make DEPEND. Stop
```

then your makefile was not created correctly. Try running **mkmf** again to make sure that a makefile is being created.

- If you get the error shown below, you should make sure that you have correctly installed your C compiler, and make sure that your C compiler's executable is in your path.

```
C:> PCGEOS\APPL\SDK_C\HELLO>pmake
makedpnd ENDCMODULES GOC GOC goc -M -D__GEOS__ -IDEPENDS -I.
-IC:\PCGEOS\INCLUDE -IC:\PCGEOS\INCLUDE\ANSI -I- -IDEPENDS -I.
-IC:\PCGEOS\INCLUDE -IC:\PCGEOS\INCLUDE\ANSI -w -cb ENDFLAGS HELLO.GOC
ENDFILES CPP BORLAND CPP -D__GEOS__ -u- -c -v -y -Ot -Oi -i200 -ml -I.
-IC:\PCGEOS\INCLUDE -IC:\PCGEOS\INCLUDE\ANSI ENDFLAGS HELLO.C ENDFILES
Processing HELLO.C...
Depends file removed.
*** Error code 1

Stop.
```

Host Machine: pmake

Next, make the executable file with the **pmake** command. The output you should see is shown below. (This, your first compilation ever with your new SDK, will take a couple of minutes. The compiler is caching information about the locations of included files. Though this takes some time for the first compile, it will make future compiles go faster.)

```
goc -DDO_ERROR_CHECKING -D__GEOS__ -IHELLO  -I.   -IC:\PCGEOS\INCLUDE
-IC:\PCGEOS\INCLUDE\ANSI  -I- -IHELLO  -I.   -IC:\PCGEOS\INCLUDE
-IC:\PCGEOS\INCLUDE\ANSI    -w -l -cb -o HELLO.EC HELLO.GOC
BCC -DDO_ERROR_CHECKING -D__GEOS__ -Ot -c -v -y -w -g255 -w-amp -w-pin -w-cln
-w-sig -w-sus -i200 -ml -IHELLO  -I.   -IC:\PCGEOS\INCLUDE
-IC:\PCGEOS\INCLUDE\ANSI
  -oHELLO.EBJ HELLO.EC
Borland C++  Version 3.1 Copyright (c) 1992 Borland International
hello.ec:
```




```
        Available memory 3000760
glue  -Og HELLO.GP -P 0.0 -R 0.0.0.0 -E -z         -m  -LC:\PCGEOS\INCLUDE\LDF  -T
1 -o HELLOEC.GEO HELLO.EBJ  -lC:\PCGEOS\INCLUDE\LDF\BORLAND.OBJ

Resource                            Size   # Relocs
-----------------------------------------------
CoreBlock                             0        0
dgroup                              144        3
HELLO_TEXT                          125        8
INTERFACE                           180        1
APPRESOURCE                         416        1

Total size: 865 bytes       Fixed size: 144 bytes
Uninitialized data/stack: 2000 bytes

Import    Number   Type      Protocol
-----------------------------------
geos        0      library    654.000
ui          1      library    743.001
```

You should now have a HELLOEC.GEO file in your current working directory on the host machine. If you do not, try the last few steps again, checking for errors in the output. If there are any, try reinstalling the host disk set and running through this procedure again.

Target Machine: cd \N9000EC

On your target machine, change to your N9000EC directory.

Target Machine: pccom

Now, on the target machine, run the **pccom** tool, which connects the target machine to the host machine.

```
C:\N9000EC>pccom
```

The screen should clear, its background should go dull gray, and the line

```
waiting for command from remote PC:
```

should appear at the top in yellow letters. If the **pccom** tool could not be found, make sure you typed it correctly, and then make sure it is in a directory in your DOS path.

Try typing the **Enter** key, which should return you to DOS.

- If it does not, then your communication set-up is not configured correctly. Double-check the values of your PTTY variable. If you're uncertain about the meaning of these values, or how they should be set, you should consult section 1.3.5 on page 27.

Assuming the **pccom** tool returned you to DOS, run the **pccom** tool again to get back to the "waiting for command from remote PC" line.

Host Machine: pcs

Return to the host PC. Download the newly-created HELLOEC.GEO file from the host to the target with the following command:

```
C:\PCGEOS\APPL\SDK_C\HELLO>pcs
sending HELLOEC.GEO to WORLD\EXTRAPPS
```

Your target PC should give output similar to the following:

```
waiting for command from remote PC:
Receiving: WORLD\EXTRAPPS\HELLOEC.GEO -
File Transfer Complete.

waiting for command from remote PC:
```

- If it does not, either HELLOEC.GEO does not exist (check the earlier steps) or the **pccom** tool is not working properly. If you get the error message "No matching files," that means that **pcs** cannot find your HELLOEC.GEO file. Make sure you did the **pmake** correctly.
- If HELLOEC.GEO exists but **pcs** still can't send it to the target machine, then you probably have a problem with your serial connection. Double check the values set in both machines' PTTY variables. If you aren't sure what these values should be, or if they look correct, then consult section 1.3.5 on page 27.
- If you have successfully gone through the communication troubleshooting procedure and **pcs** still isn't working, there's one more thing you should try: If both machines lock up soon after the "sending HELLOEC.GEO to WORLD\EXTRAPPS" message appears, you may be trying to transmit at too high a rate. Reduce the speed of transmission on both the host and target machines. If you've already looked through section 1.3.5 on page 27, then you know that you can do this by changing



the second field of the PTTY variable or by passing the /b flag to the **pccom** tool and **pcs**.

Host Machine: swat

Assuming all has gone well so far, run the debugger now by typing

```
C:\PCGEOS\APPL\SDK_C\HELLO>swat
```

The screen should clear after a few seconds, and the following should come up on the host's screen:

```
Swat version 2.0 (Nov  3 1995 10:34:10).
Using the trunk version of GEOS.
Looking for “loader”...(cached)...C:/PCGEOS/LOADER/TEXT/LOADEREC.SYM
Sourcing swat.tcl...done
PC Halted
Stopped in LoadGeos, GEOS system source file not available.
Continuing...
Looking for “geos    Eker”...(cached)...C:/PCGEOS/LIBRARY/KERNEL/RESPDEMO/GEOSEC
.SYM
Looking for “ms4     Eifs”...(cached)...C:/PCGEOS/DRIVER/IFS/DOS/MS4/MS4EC.SYM
Looking for “netware Eifs”...(cached)...C:/PCGEOS/DRIVER/IFS/DOS/NETWARE/NETWARE
E.SYM
netware exited.
```

...Many, many Looking for... and Thread... lines...

```
Thread 0 created for patient geoplann
Thread 1 created for patient geoplann
Thread 1 created for patient indicato
Thread 0 of rtcm exited 0
Thread 1 of indicato exited 0
faxfile exited.
clrfax exited.
Thread 0 of clrfax exited 0
Looking for “vmtree  Edrv”...(cached)...C:/PCGEOS/DRIVER/MAILBOX/DATA/VMTREE/VMT
REEEC.SYM
vmtree exited.
Looking for “serial  Edrv”...(cached)...C:/PCGEOS/DRIVER/STREAM/SERIAL/SERIALEC.
SYM
serial exited.
```



- If you get an error like the following:

```
Abnormal program termination:Memory protection fault
CS:EIP = 000Fh:0000000Ahof PC/GEOS
```

  then chances are there is a problem in your CONFIG.SYS file, either on the FILES or BUFFERS line. Your FILES value should *probably* be at least 80 (at least 100 if you will be task-switching while using Swat). You may note the use of the word "probably" in that last sentence. The third-party memory manager used by Swat has trouble on some hardware configurations; for these machines, you should try lowering the FILES number in your CONFIG.SYS to 60 and/or raise the BUFFERS value to 40.

  Another possible cause for this sort of error is that your environment variables may be over-running their space. To increase the environment space, make sure there is a line like the following in your CONFIG.SYS file following the pattern:

```
shell = command.com /e:XXXX /p
```

  where XXXX is a number, 1024 or higher.

- If the only thing that appears on your host machine is

```
Using the trunk version of GEOS.
```

  and your target machine is frozen (i.e. nothing else appears for at least a minute), you should probably try communicating at a slower rate. Try changing the PTTY variables to use lower values (such as 9600, if you are trying with 19200). Remember that the value should be the same on both machines.

- If you get the following sorts of errors (this will occur after a fair amount of normal Swat output)

```
Looking for "sound Elib"...C:/PCGEOS/Library/Sound/soundec.geo
Looking for "standardEdrv"...hmmmm.
Can't find executable file for "standardEdrv" (version mismatch?)
Answer "quit" to exit to the shell
Answer "detach" to detach and return to top level
Answer "ignore" to ignore this patient
Where is it? C:\PCGEOS\APPL\SDK_C\HELLO/
```




…then your FILES value in your CONFIG.SYS file may be set too low. Try changing this value to something higher (see above).

- If you get the following error:

```
Looking for “geos Eker”...c:/pcgeos/Library/Kernel/geosec.geo
Interrupt 13: Protection violation
Stopped in 3328h:2a6dh, address 3328h:2a6dh
Swat+10861: CALL Swat+4114
(geos:0)
```

Then interrupts on your target machine are interfering with interrupts which Swat uses to communicate between machines. Exit the **pccom** tool on the target machine (if you encounter this error, chances are you will need to restart your target machine), and make sure that from now on you call the **pccom** tool with the /i:d flag. This will ask the **pccom** tool to ignore spurious interrupt 13 signals (d is 13 in hex). To do this, create a file called COMM.BAT on the target machine in your target software directory with the following contents:

```
pccom /i:d
```

From now on, whenever you would normally invoke the **pccom** tool on the target machine, just use **comm** instead.

Host Machine: Ctrl-C

Type Ctrl-C on the host machine. This will tell Swat to freeze the target machine. Swat will then present its prompt:

```
PC Halted
Stopped in Idle, address 2d44h:c6dbh
Idle+64:              POP     DX                ;[SP]=3729h
(geos:0) 1 =>
```

Host Machine: quit

When you can get through all of those steps without hitting any errors, your setup is probably in good working order.

To exit from Swat (so that you can go on to other things), type **quit** at the Swat prompt. You may need to enter the DOS command **cls** to reset your screen after exiting Swat.

1.3.5 Troubleshooting Communications

The SDK install scripts work well for those programmers who know which COM ports and IRQ levels their machines will use when communicating. However, when something goes wrong with this process, it can be very difficult to determine the correct combination of port and interrupt numbers. This section contains a systematic way to determine the correct configuration to use.

If you've already got communication working between your machines (you used Swat successfully in the previous section), then you may thankfully skip this section and go on to section 1.4 on page 32.

The Cables

Your machines should be connected by a null-modem serial line. Lap-link serial cable works admirably for this purpose.

The Settings

Communications between machines are controlled by some settings on both machines. To get communication working, you will be working with these settings.

- The COM port setting tells the **pccom** tool which serial port the serial communication cable is plugged into. This value can range from one to four.
- The communication speed determines how quickly your machines will try to communicate. We suggest a speed of 19200 baud; if one of your machines (presumably your target machine) is slow, you may have to use 9600 baud instead.

If your host is a 486 or better, you might try using 38400 baud, perhaps after making sure that things work at 19200.

- Each COM port has an IRQ level associated with it. If you try to send a signal to a COM port using the wrong IRQ level, your machine tends to generate some bad interrupts and freeze.

Setting and Overriding the PTTY variable

For the communication software to work correctly, both your target and host machines must have a correctly set PTTY variable. Type **set** on both machines to find out what the present value is for PTTY. The **set** command should return output that includes a line that looks something like the following (the numbers may be different):

```
PTTY=1,19200,4
```

or

```
PTTY=1,19200
```

If you are reading this chapter, then at least one of these values on at least one of your machines has probably been set incorrectly.

- The first number of this variable signals which COM port this machine will use to communicate. This number must be an integer between one and four. The two PCs may have different values.
- The second number specifies the communication speed, measured in baud. This value must be the same on both machines.
- The third number, if any, specifies the IRQ level to use when communicating. If there is no third number, then it is assumed that you are using the IRQ level normally associated with the given COM port. (I.e. if the first number of your PTTY variable is 1 or 3, the communication software will assume you want to use IRQ level 4. If the first number of your PTTY variable is 2 or 4, IRQ level 3 is assumed.)

To change your PTTY variable, issue the command

```
set ptty=1,19200,4
```

or

```
set ptty=1,19200
```

Note that there are no spaces in this command other than that between "set" and "ptty". If you include a space before the equals sign, you are actually setting the value of a new variable, "PTTY " (PTTY with a space after it). If you place a space after the equals sign, the communication software won't be able to interpret the value. There should also be no spaces after the commas.

When you have figured out the correct values to use when communicating, you will want to make sure there is a line in your AUTOEXEC.BAT file that sets the variable correctly.

When invoking the **pccom** tool, you can override the values set by the PTTY variable by passing the /c, /b, and /I flags. For instance, typing

```
pccom /c:2 /b:9600 /I:3
```

will invoke the **pccom** tool using COM port 2, IRQ level 3, and a 9600 baud communication rate, no matter what values have been set in the PTTY variable.

Step 1: Make Your Best Guess

When you troubleshoot your communication set-up, the most difficult problem to solve is the case where you don't know which COM port either machine is using. If you are in this situation now, some research might be in order. The documentation of one or more of your machines should contain a clue as to which COM port number is associated with a particular serial socket. If you have another device which you know uses a serial port, then you can be sure that the cable is not hooked up to that port. Thus, if you know that your target machine has a mouse plugged into serial port two, you know that you will not be communicating with a COM port number of two.

If you narrow down the COM port choices now, you will save yourself a lot of time in the steps to follow.

Step 2: Invoke pccom on Both Machines

At the DOS prompt, type "pccom" on both machines. If you are passing flags to the **pccom** tool at either machine, now is the time to pass them. As you continue with troubleshooting, you will probably have to exit the **pccom** tool on one or both machines several times to change COM port and IRQ level settings. Always remember to start up the **pccom** tool again, as no communication will take place if it is only running on one machine.



Step 3+: Type “abc” On The Host Machine

Depending on what happens when you do this, you can narrow in on your communication problem.

“abc” appears on the Target (i.e. the other) machine—Success!

You have found the correct communication settings. Edit your AUTOEXEC.BAT files so that your PTTY variables will be set correctly when you next restart your machines. If you have been experimenting with communications settings, you may wish to change your PTTY variables immediately, or just restart your machines after changing the AUTOEXEC.BAT files.

No text appears, Neither machine frozen—Wrong COM port?

Unfortunately, this set of symptoms could mean almost anything. The most common cause is an incorrectly chosen specified COM port—the host machine successfully sent the signal to the COM port you selected. However, the signal never got to the target machine. Either the host’s COM port setting was wrong, so that the signal was sent to the wrong port and never reached the cable; or the target’s COM port was wrong, so that it was listening at the wrong port. It is possible that in fact both machines have the wrong COM port settings.

Try using different COM port settings on one or both machines. (Do this by changing the first part of your PTTY variable or by passing a /c: flag to the **pccom** tool.) Of course, if you are rather sure about which COM port one of the machines is using, you should try changing the other one’s settings first.
Remember that the COM port value may be 1, 2, 3, or 4. If you know that some device is already using a port, than you need not test that port (assuming you haven’t somehow managed to plug a serial cable and some other device into the same serial port socket).

No text appears, Host not frozen, Target Frozen—Target IRQ wrong.

You’ve almost achieved the right settings; your host machine is sending the signal correctly, and your target machine is listening at the right port. However, your target machine is getting confused when the signal comes in. Change the IRQ level setting on your target machine, either by changing the PTTY variable, or by passing an /I: flag to the **pccom** tool.

Before continuing with tests, "unfreeze" your target by restarting it.

No text appears, Host is frozen—Host IRQ level is wrong.

The IRQ level you have chosen to use with your host's COM port is not correct for that COM port. Note that there is no guarantee that you have chosen the correct COM port number either. Try using a different IRQ level, and when you find the correct one for that COM port, you might want to note it down.
Before continuing with tests, "unfreeze" your host by restarting it.

"a" on Target, Host frozen—Everything's correct except IRQ level.

The host successfully transmitted the "a" to the target. The target machine listened at the right port and sent back a signal that it was ready for the next character. The host received this signal, but could not handle it and froze. Try a different IRQ level for your host (by using a different number for the third part of your host's PTTY variable or by using the /I: flag when next invoking the **pccom** tool on the host machine).
Before continuing with tests, "unfreeze" your host by restarting it.

"ΔΔΔ" appears on the Host (i.e. the same) machine—Incorrect cables.

You do not have a null-modem connection in the serial line between your machines. The null-modem acts like a telephone line, making the host's "mouth" talk to the target's "ear," and that the target's "mouth" talks to the host's "ear." Without the null-modem connection, your machines will act as if they are talking on the phone, but one of them is holding their receiver upside down—listening at the mouthpiece and talking into the earpiece. Your host's voice is being echoed back into its own "ears," getting corrupted in the process. Use a null-modem connection.

The good news is that your signal is in fact reaching the cable, and thus your host machine is using the correct COM port and IRQ level settings.

If you're unsure about your target machine settings, you might try reversing these tests, typing "abc" on the target machine to see what happens. The **pccom** communication is bi-directional, so you may use the above tests just as well, substituting the word "host" for "target," and vice versa.

1.4 Debugging on the Device

For the sake of speed, you'll want to do most of your debugging using a fast PC as your target machine. However, you'll need to use an actual Nokia 9000 Communicator as the target to test some things; even if your program doesn't use any Communicator-specific features, you should do *some* testing on the device.

To connect the Swat debugger to the Nokia 9000 Communicator, carry out the following steps

1. Make sure that the host machine's swat communication rate is set at 38400. For information about how to check the communication rate (and to change it, if necessary), see section 1.3.5 on page 27.
2. Remove the Communicator's batteries and put them back in.
3. On the Communicator, quickly press Shift d o (all three keys at once) and hold them down. You'll need to hold them for a few seconds. This starts the software on the Communicator with a special program that listens for signals from Swat.
4. A screen with several lines of small text should appear—these should show commands being run at a DOS prompt. At this point, you may take your fingers off the keys. (If you see the Nokia 9000 Communicator splash screen, that means that something went wrong.)
5. Attach the 6-pin connector to the unit (if it isn't already), with the serial line going to your host machine. (If the connection doesn't seem to be working, you can test it by using the **Install/Remove software** built-in application and the Nokia Communicator Server program on your PC—the server comes with the Nokia 9000 Communicator.)
6. Run Swat on the host machine as you did in section 1.3.4 on page 19, except that you'll want to Swat using the non-error-checking software. (Use **swat -n** from the DOS prompt, or use the Swat NC icon from the MS Windows Program Manager) You're ready to start debugging!

If you press Shift-d-b instead of Shift-d-o, then the Swat stub (the program on the N9000 that communicates with Swat) will start up, but the Nokia 9000 software will continue to run on top of it. You may still attach Swat (though if something goes wrong while you're attaching, it will be harder to tell when it goes wrong).



As Swat loads the .sym symbol files for the apps and libraries on-board the phone, it will not be able to find a matching .sym file for the smsrcv.geo application geode. It will ask you where it is. You have several options:

1 Type ctrl-x (to erase the input line) and type "I" for "Ignore." This will ignore this patient permanently: Swat will never try to load smsrcv.sym again, and no symbols from that patient will ever be displayed by Swat. This, however, should not present a problem, since you should never need to debug the on-board SMS Receive application.

(To unignore a permanently-ignored patient, use the Swat command "unignore".)

2 Type ctrl-x (to erase the input line) and type "i" for "ignore." This will ignore the patient only in your current Swat session, but the next time you launch Swat, it will try to load smsrcv.sym (unsuccessfully) again, so you'll have to ignore the patient again.

3 You can use the .sym file provided on the SDK, though it doesn't have the exactly correct serial number (i.e., this smsrcv.sym file wasn't generated at the same time as the smsrcv.geo geode). Type in:

```
[drive]:\pcgeos\appl\fapps\oem\smsrcv\smsrcv.geo
```

(Here, Swat is case-insensitive.)

Swat will inform you that smsrcv.sym hasn't got the correct serial number, and ask if you want to use it anyway: respond with "y", and Swat will then finish loading.

Again, this is a temporary setting, so each time you start Swat, you would have to type this in to use the .sym file on the SDK.

IR connections and PC connectivity will no longer operate when the Nokia 9000 Communicator is connected via Swat to a host PC. (The Swat stub on the Nokia 9000 Communicator is using the port.) If you try to use IR or PC connectivity, the Nokia 9000 Communicator will lock up.

Since the Nokia 9000 Communicator only uses the non-error- checking versions of the software, you will notice that Swat uses the non-EC versions of the .sym files as it loads.

After using "quit" to exit Swat, you'll notice that the phone has a DOS prompt, "A: mini>". There is no keyboard driver at the DOS level, however, so you cannot run any command or batch file to start the Nokia 9000



Communicator waiting for Swat to connect. If you want to start a new Swat session after using “quit”, you'll have to remove the batteries from the Nokia 9000 Communicator and redo the steps above.



2

2.1 FoamSubApplicationClass 37
2.1.1 FoamSubApplicationClass Instance Data 38
2.1.2 Using a FoamSubApplication Object 39
2.2 ComplexMonikerClass 40
2.2.1 ComplexMoniker Instance Data 41
2.2.1.1 The ComplexMoniker Keyword 42
2.2.1.2 The ComplexMoniker Text 42
2.2.1.3 The ComplexMoniker Icon Bitmap 45
2.2.1.4 Text Styles 47
2.2.1.5 Font Sizes 48
2.2.1.6 ComplexMoniker Vardata 48
2.2.2 ComplexMoniker Usage 49
2.2.3 ComplexMoniker Messages 51
2.3 Convenient Dialog Routines 53
2.4 FlashingNoteClass 54
2.4.1 FlashingNote Instance Data 55
2.4.2 Setting Up the Template 56
2.4.3 Initiating and Destroying the Flashing Note 58
2.5 FoamProgressDialog 58
2.5.1 FoamProgressDialog Instance Data 59
2.5.2 Setting Up the Template 60
2.5.3 Using a FoamProgressDialog 61
2.6 Routines for Lists 63





The Nokia 9000 contains a very specific look and feel. This look and feel is created through use of a specific user interface developed for that device. This specific UI contains not only visual representations for GEOS objects, but also contains additional objects that are particularly tailored for the Nokia 9000 device.

Most of these objects have been grouped into the "Foam" library. This library is a special collection of classes, objects and routines specifically applicable to the Nokia 9000 device. Because of its multi-faceted nature, the Foam library is not a coherent library so much as a "toolbox" of useful widgets.

We will not attempt to cover every piece of the Foam library within this chapter. Instead, we will discuss the main objects that are particularly useful for creating your application.

These objects are:

- FoamSubApplicationClass
- ComplexMonikerClass
- FlashingNoteClass
- RepeatingGenGlyphClass
- Special Foam dialog box routines
- Special routines for presenting lists

2.1 FoamSubApplicationClass

The Nokia 9000 device is unique in that it is also a development platform. Applications may be written to be incorporated into the device as part of a special release. The Nokia 9000 does not have a file manager, per se, to enable launching these applications. The applications must be launched through a currently existing application. **FoamSubApplicationClass** allows you to indicate under which pre-existing application the new application should reside.

"Add-on" applications for the Nokia 9000 device, whether included on the device as part of a value-added release, or developed as an aftermarket product, will fall into one of three categories:

- Service applications that communicate to the outside world.
- System applications that perform a system function. (PC connectivity tools fall under this category, for example.)
- "Extra" applications that don't fall into either of the two prior cases.

For each of these cases, there is an application already on the device that launches the sub-application. Service applications are launched using the SVCAPPS launcher; system applications use the SYSAPPS launcher; extra applications use the EXTRAPPS launcher.

Add-on applications are declared using the FoamSubApplication object, and an instance data field is set within the object indicating the "main" application it should be tied with.

Because a sub-application is not a separate application, special care needs to be taken when the sub-application is closed. This is discussed in "Using a FoamSubApplication Object" on page 39.

2.1.1 FoamSubApplicationClass Instance Data

There is only one instance data field in a **FoamSubApplicationClass** object. This field is noted below.

Code Display 2-1 FoamSubApplicationClass Instance Data

```
@instance FApplication FSAI_launcherApp;
```

FSAI_launcherApp stores an **FApplication** type referring to the application it should reside under.

For FoamSubApplication objects, the following **FApplication** values are the only values that are relevant:

FA_SERVICES_APP
FA_SYSTEM_APP
FA_EXTRAS_APP

2.1.2 Using a FoamSubApplication Object

MSG_FSA_RETURN_TO_LAUNCHER

FoamSubApplication objects do not close like other applications. Because of this, they need to provide their own special "Close" trigger. However, FoamSubApplication objects should not simply "close" when the user presses this trigger. Instead, they should send MSG_FSA_RETURN_TO_LAUNCHER to this FoamSubApplication object.

This message changes the **[hardIconBar]** entry of the .INI file for the sub-application to be the launcher application (stored in *FSAI_launcherApp*). This allows the sub-application to seemingly close without closing the parent application. It is the responsibility of the application to provide a "Close" trigger that sends this message to the **FoamSubApplicationClass** object.

The "Close" trigger is declared as a **ComplexMonikerClass** object of type CMT_CLOSE. The ComplexMoniker is discussed more fully in "ComplexMonikerClass".

Code Display 2-2 Using the FoamSubApplicationClass Object

```
/*
 * Declare the FoamSubApplication object as you would a normal GenApplication
 * object. This sub-app will be launched from the Extras application.
 */

@start   AppResource

@object FoamSubApplicationClass MyApp = {
    GI_comp = @MyPrimary;
    FSAI_launcherApp = FA_EXTRAS_APP;
    gcnList(MANUFACTURER_ID_GEOWORKS, GAGCNLT_WINDOWS) = @MyPrimary;
    GI_visMoniker = list { @MyTextMoniker };
}

@visMoniker MyTextMoniker = "Nokia UI\rSample App";

@end     AppResource;
```

```
/*
 * Somewhere in your sub-app, create a CMT_CLOSE trigger.
 */

@object ComplexMonikerClass CloseTrigger = {
    ComplexMoniker = GenTriggerClass;
    CMI_topText = CMT_CLOSE;

    GTI_actionMsg = MSG_FSA_RETURN_TO_LAUNCHER;
    GTI_destination = @TemplateApp;

    HINT_SEEK_MENU_BAR;
    HINT_SEEK_REPLY_BAR;

    /*
     * Close Command triggers should always be in the fourth position (3 because
     * HINT_SEEK_SLOT refers to zero-based slot positions).
     */
    HINT_SEEK_SLOT = 3;

}
```

2.2 ComplexMonikerClass

ComplexMonikerClass is a versatile class, providing powerful behavior with little overhead. A complex moniker serves two purposes. First, it allows a developer to easily create a moniker containing both graphics and text. Second, it allows both text and graphics to be easily localized by referring to both items "generically."

A complex moniker may take a variety of forms. **ComplexMonikerClass** is what's known in GEOS as a "variant" class — the object's parent class is dynamically assigned when the object is visually built.

A complex moniker may contain both a text string and a bitmap. The text string may be either a single line or two lines (separated by a '\r' character

in the text buffer). If the text is two lines, it will be right-justified; this allows text strings in the command bar to align properly.



Figure 2-1 *ComplexMoniker objects*

Several ComplexMonikers can be found in the display shown above. The "Enter Number" trigger relies upon ComplexMoniker's ability to handle a two-line moniker. The "Call" trigger takes advantage of an automatically localized name provided by the built in CMT_CALL ComplexMonikerType. The "Telephone Directory" interaction is also a ComplexMoniker; it has both a graphic and a text moniker. The graphic in this case is also automatically localized by using the built-in CMB_CONTACT_LIST ComplexMonikerBitmap.

2.2.1 ComplexMoniker Instance Data

ComplexMonikerClass contains the following instance data:

Code Display 2-3 ComplexMonikerClass Instance Data

```
ComplexMoniker          ClassName

@instance TextStyle     CMI_textStyle = (TS_BOLD);

@instance word          CMI_topText;

@instance word          CMI_iconBitmap;

@instance byte          CMI_fontSize;
```

2.2.1.1 The ComplexMoniker Keyword

Because **ComplexMonikerClass** is a variant class, it needs to have its superclass assigned when it is built. The *ComplexMoniker* keyword assigns the parent class of this complex moniker from **GenClass** (the default) to the class name specified. The following classes are supported:

GenTriggerClass
GenInteractionClass
GenItemGroupClass
GenItemClass
GenBooleanListClass
GenDynamicListClass
ContactListClass

Each of these classes will behave quite differently, but all will exhibit the same characteristics in displaying their text and icon monikers that are gained through using **ComplexMonikerClass.** Note that using a ComplexMoniker object will cause the specific UI to ignore any moniker stored in the *GI_visMoniker* instance data field in the above classes.

2.2.1.2 The ComplexMoniker Text

CMI_topText stores the "text" to display. This may be either a Chunkhandle to a text string, or a **ComplexMonikerText** identifier. If the value is a **ComplexMonikerText** identifier, the specific UI will determine what exact text is displayed.

As noted, the text may contain be two lines. The top text is limited to COMPLEX_MONIKER_MAX_TOP_TEXT_LENGTH while the bottom is limited to COMPLEX_MONIKER_MAX_BOTTOM_TEXT_LENGTH.

Below is a complete list of types; in most cases the (English) text on the Nokia 9000 device is similar, if not identical, to the text alluded to by the identifier's name. Those cases where the actual text is noticeably different are noted. Using such an identifier is beneficial for localization purposes.

CMT_NULL Never set a moniker to this value. It exists purely as a placeholder.

Basic strings:

CMT_OK	
CMT_CLOSE	
CMT_CANCEL	
CMT_YES	
CMT_NO	

Command strings:

CMT_CLEAR	
CMT_CLEAR_ALL	
CMT_CHANGE	
CMT_RENAME	
CMT_COPY	
CMT_SETTINGS	
CMT_OPEN	
CMT_CREATE_NEW	“New”
CMT_DELETE	
CMT_ROTATE	
CMT_SELECT	
CMT_START	
CMT_GO	
CMT_INSTALL	
CMT_DEINSTALL	“Remove”
CMT_NEXT	
CMT_PREVIOUS	
CMT_USER_DEFINED	“User defined:”
CMT_ZOOM	
CMT_ZOOM_IN	
CMT_ZOOM_OUT	
CMT_VIEWING_TOOLS	
CMT_OPTIONS	

Communication strings:

CMT_PHONE	“On”
CMT_CALL	
CMT_CALL_LOWERCASE	“Call”
CMT_SEND	
CMT_SEND_OPTIONS	
CMT_CONNECT	
CMT_CALLS	
CMT_FAXES	
CMT_DATA_CALLS	
CMT_MESSAGES	
CMT_RECENT_CALLS	
CMT_RECENT_FAXES	“Recent numbers”
CMT_RECENT_MESSAGES	“Recent numbers”



CMT_RECEIVED_CALLS	
CMT_RECEIVED_FAXES	“Received fax numbers”
CMT_RECEIVED_MESSAGES	“Received message numbers”
CMT_MISSED_CALLS	
CMT_LAST_DIALED_CALLS	“Dialed calls”
CMT_TO_COLON	“To:”
CMT_FROM_COLON	“From:”
CMT_TEXT_SEND	“Send text”
CMT_TEXT_CAPTURE	“Capture text”
CMT_CAPTURE	
CMT_STOP_CAPTURE	
CMT_FORWARD_FAX	“Forward”
CMT_SELECT_RECEIVER	“Recipient”
CMT_HANGUP	
CMT_FORWARD	
CMT_REPLY	

Miscellaneous commands:

CMT_CLEAR_LIST	
CMT_NO_CONTACT_INFORMATION	“No log information”
CMT_CONTACT_CARD	
CMT_NONE	
CMT_CLEAR_ALL_COMMAND	
CMT_ACCEPT	
CMT_RESET	
CMT_STOP	
CMT_ACTIVATE	
CMT_DISABLE	
CMT_HELP	
CMT_BACK	
CMT_SPECIAL_KEYS_COMMAND	“Keys”
CMT_SPECIAL_KEYS	
CMT_TEXT	
CMT_TEXTS	
CMT_ZOOM_PLUS	“Zoom in”
CMT_ZOOM_MINUS	“Zoom out”
CMT_INSERT	
CMT_OFF	
CMT_ON	

Code Display 2-4 Setting a Complex Moniker's Text

```
/*
 * The text in a complex moniker is stored within the CMI_topText instance field.
 * This text may either be referenced by a ChunkHandle ...
 */

@object ComplexMonikerClass MyMoniker = {
    ComplexMoniker = GenTriggerClass;
    CMI_topText = @MyText;
}

/* Use the TCHAR type to ensure the text is DBCS-compliant. */

@chunk TCHAR MyText[] = “Sample Text”;

/*
 * ... or CMI_topText can store a ComplexMonikerText type which indicates the text
 * generically.
 */

@object ComplexMonikerClass MyClearTrigger = {
    ComplexMoniker = GenTriggerClass;
    CMI_topText = CMT_CLEAR;
}
```

2.2.1.3 The ComplexMoniker Icon Bitmap

CMI_iconBitmap stores the bitmap to display within this moniker. This may either be a ChunkHandle to a Bitmap structure or a member of the **ComplexMonikerBitmap** types. These types are enumerated below:

CMB_LEFT_FINGER
CMB_RIGHT_FINGER
CMB_CONTACT_CARD
CMB_CONTACT_LIST
CMB_SPEED_DIAL
CMB_SETTINGS_TITLE_ICON
CMB_SERVICES_VIEW
CMB_SERVICES_DEFINE
CMB_QUESTION_ICON
CMB_OPEN_FOLDER
CMB_DRAWER



CMB_RENAME_COPY
CMB_FOLDER
CMB_SECURITY_SUCCESS
CMB_SECURITY_ERROR
CMB_PC_ICON
CMB_INPUT_TITLE_ICON
CMB_WARNING_TITLE_ICON

Again, using a generic identifier eases localization.

You can create your own bitmaps by using the GEOS Icon Editor. To do so, perform the following steps:

- Start GEOS (C:\PCGEOS\GEOS2XNC\LOADER.EXE
- Launch the Icon Editor application.
- Select either the 5th format (32x20 monochrome) or the 1st format (48x30 color). The first is used to create a monochrome bitmap; the second is used to create a 4-bit grayscale bitmap.
- Edit the bitmap as desired.

  The Nokia 9000 is a grayscale device. It is capable of handling 4-bit grayscale bitmaps (**BMFormat** of BMF_4BIT). You may also create simple monochrome bitmaps (**BMFormat** of BMF_MONO).

  The GEOS Icon Editor only creates color bitmaps. The following colors in the Icon Editor will be mapped to the matching grey-scale shades on the device to create a grayscale bitmap:

Icon Editor Color	Nokia 9000
WHITE	WHITE
BLACK	BLACK
LIGHT_VIOLET	LIGHT GRAY
DARK_GREEN	DARK GRAY

- Write the file out as *filename*.goh using the following options:
  Current Format
  Vismoniker source code
  Compacted
  Write mask (default)
- Edit the first line of the *filename*.goh file and replace this line:

```
@visMoniker Moniker ={
```

with:

```
@chunk byte YourIconName[] = {
```

- Delete the following lines from the beginning of the file:

```
size = <...>;
style = <...>;
color = <...>;
aspectRatio = <...>;
cachedSize = <...,...>;
gstring { GSFillBitmapAtCP(<...>),
```

- Delete the following lines from the end of the file:

```
GSEndString()
}
```

- Finally, add a semi-colon to the end of the file.

Your resulting bitmap, ready for inclusion in your app, should look like this:

```
@chunk byte MyIconName[] = {
    Bitmap (32,20,BMC_PACKBITS,
            (BMT_MASK|BMF_4BIT)),
    0xfd, 0xff, 0x03, 0xbf, ...
};
```

2.2.1.4 Text Styles

CMI_textStyle stores a GEOS **TextStyle**, defined in **graphics.h**. By default, text is shown as TS_BOLD. The full list of styles appears below:

TS_BOLD (default)
TS_OUTLINE
TS_ITALIC
TS_SUPERSCRIPT
TS_SUBSCRIPT
TS_STRIKE_THRU
TS_UNDERLINE

2.2.1.5 Font Sizes

CMI_fontSize stores a font size (in points). As indicated in the “Nokia 9000 Basics” chapter, there are four pre-established font sizes that you should choose from:

```
#define FOAM_LARGE_FONT_SIZE  20
#define FOAM_NORMAL_FONT_SIZE 18
#define FOAM_LIGHT_FONT_SIZE  17
#define FOAM_SMALL_FONT_SIZE  16
```

The large font size is best reserved for commands and title bar monikers. In most other cases, use the normal font size.

The default font can be obtained by those routines or objects that demand a font name by using the **VisTextDefaultFont** value VTDF_RESPONDER.

2.2.1.6 ComplexMoniker Vardata

There are several vardata attributes that you may attach to the Complex Moniker. These are noted below:

Code Display 2-5 ComplexMoniker Vardata

```
@vardata        void    ATTR_COMPLEX_MONIKER_PLACE_BITMAP_AT_LEFT;

@vardata        void    HINT_COMPLEX_MONIKER_DRAW_SEPARATOR;
@vardata        void    HINT_COMPELX_MONIKER_DONT_DRAW_SEPARATOR;
```

ATTR_COMPLEX_MONIKER_PLACE_BITMAP_AT_LEFT will position the icon bitmap (if any) to the left of the text.

HINT_COMPLEX_MONIKER_DRAW_SEPARATOR draws a single-line separator below the complex moniker.

HINT_COMPLEX_MONIKER_DONT_DRAW_SEPARATOR prevents the drawing of a single-line separator below the complex moniker.

2.2.2 ComplexMoniker Usage

MSG_COMPLEX_MONIKER_GET_MONIKER,
MSG_COMPLEX_MONIKER_REPLACE_MONIKER

In most cases, you will want to use a ComplexMoniker when you create Command triggers. Command triggers on the Nokia 9000 appear along the right edge of the screen, and correspond to the four off-screen buttons that they abut.

To create a ComplexMoniker command trigger, follow these steps:

- Set the *ComplexMoniker* keyword to **GenTriggerClass**.
- Include the hint HINT_SEEK_MENU_BAR and HINT_SEEK_REPLY_BAR in the object declaration.
- Set the HINT_SEEK_SLOT hint to 0,1,2, or 3, corresponding to the (zero-based) slot you wish to place the command trigger within.
- (If you are not creating a command trigger, set *CMI_fontSize* to something other than FOAM_LARGE_FONT_SIZE.)

Code Display 2-6 Complex Moniker Example (GenInteraction)

```
@object ComplexMonikerClass AboutBox = {

/* This object will behave as a GenInteration (grouping) object */
    ComplexMoniker = GenInteractionClass;
    CMI_topText = @AboutBoxTitle;
    CMI_iconBitmap = @AboutIcon;
    GI_comp = @AboutText;

/* Geometry Hints */

    HINT_DRAW_IN_BOX;
    HINT_DRAW_SHADOW;
    HINT_PLACE_MONIKER_TO_LEFT;
    HINT_EXPAND_WIDTH_TO_FIT_PARENT;
    HINT_EXPAND_HEIGHT_TO_FIT_PARENT;

/* ComplexMoniker Hints */

    HINT_COMPLEX_MONIKER_DRAW_SEPARATOR;
    ATTR_COMPLEX_MONIKER_PLACE_BITMAP_AT_LEFT;

}
```




```
@chunk TCHAR AboutBoxTitle[] = “CmplxMon Sample Application”;
```

Code Display 2-7 ComplexMoniker Example (GenDynamicList)

```
@object ComplexMonikerClass CMTextsList = {
    ComplexMoniker = GenDynamicListClass;
    CMI_topText = @CMTextsListTitle;
    CMI_fontSize = FOAM_NORMAL_FONT_SIZE;
    CMI_iconBitmap = CMB_OPEN_FOLDER;
    GIGI_destination = process;
    GIGI_applyMsg = MSG_CMPLXMON_PROCESS_SHOW_CURRENT_CMTEXT;
    GDLI_queryMsg = MSG_CMPLXMON_PROCESS_GET_CMTEXT;

/* GDLI_numItems will be set at app-launch time. */

    HINT_DRAW_IN_BOX;
    HINT_DRAW_SHADOW;
    HINT_PLACE_MONIKER_ABOVE;
    HINT_PLACE_MONIKER_TO_LEFT;
    HINT_COMPLEX_MONIKER_DRAW_SEPARATOR;
    HINT_MINIMIZE_CHILD_SPACING;
    HINT_EXPAND_WIDTH_TO_FIT_PARENT;
    HINT_EXPAND_HEIGHT_TO_FIT_PARENT;
    HINT_ITEM_GROUP_SCROLLABLE;
    ATTR_COMPLEX_MONIKER_PLACE_BITMAP_AT_LEFT;
    ATTR_GEN_SEND_APPLY_MSG_ON_APPLY_EVEN_IF_NOT_MODIFIED;
    ATTR_GEN_ITEM_GROUP_STATUS_MSG =
        MSG_CMPLXMON_PROCESS_SHOW_CURRENT_CMTEXT;
    }
}
```

Code Display 2-8 ComplexMoniker Example (Command Trigger)

```
@object ComplexMonikerClass MyCommandTrigger = {
    ComplexMoniker = GenTriggerClass;
    CMI_topText = @CommandTextTitle;
    GTI_actionMsg = MSG_GEN_INTERACTION_INITIATE;
    GTI_destination = @DialogThatThisTriggerBringsUp;
```

```
    HINT_SEEK_MENU_BAR;
    HINT_SEEK_REPLY_BAR;
    HINT_SEEK_SLOT = 0;
}

@chunk byte CommandTextTitle[] = “I Commeand Thee!”;
```

2.2.3 ComplexMoniker Messages

MSG_COMPLEX_MONIKER_GET_MONIKER,
MSG_COMPLEX_MONIKER_REPLACE_MONIKER

To retrieve a complex moniker, send the ComplexMoniker MSG_COMPLEX_MONIKER_GET_MONIKER. This message takes a single argument, a pointer to a structure of type **GetComplexMoniker**. The structure will be filled in with the relevant information by the message.

```
typedef struct {
    TextStyle           GCM_textStyle;
    byte                GCM_fontSize;
    ComplexMonikerText  GCM_topText;
    ComplexMonikerBitmapGCM_iconBitmap;
} GetComplexMoniker;
```

Send MSG_COMPLEX_MONIKER_REPLACE_MONIKER to replace a complex moniker with a new moniker.

This message takes two arguments: a **ReplaceComplexMoniker** structure and a **ReplaceComplexMonikerChunkHandles** structure.

The **ReplaceComplexMoniker** structure should hold the parameters for the new moniker.

```
typedef struct {
    TextStyle                   RCM_textStyleSet;
    TextStyle                   RCM_textStyleClear;
    word                        RCM_fontSize;
    dword                       RCM_topTextSource;
    dword                       RCM_iconBitmapSource;
    ComplexMonikerSourceType    RCM_topTextSourceType;
```




```
    ComplexMonikerSourceType    RCM_iconBitmapSourceType;
    word                        RCM_iconBitmapSize;
    RCMOverwrite                RCM_overwrite;
} ReplaceComplexMoniker;
```

RCM_textStyleSet stores a **TextStyle** record of styles that should be set (masked) on top of any currently existing styles.

RCM_textStyleClear stores a **TextStyle** record of styles that should be cleared if they are set among the currently existing styles.

RCM_fontSize stores the new font size. Pass 0 to indicate no change.

RCM_topTextSource stores a dword value containing, or pointing to, the source for the new text. The type of value stored here is indicated in *RCM_topTextSourceType*.

RCM_iconBitmapSource stores a dword value containing, or pointing to, the source for the new bitmap. The type of value stored here is indicated in *RCM_iconBitmapSourceType*.

RCM_topTextSourceType stores the type of text reference contained in *RCM_topTextSource*.

RCM_iconBitmapSourceType stores the type of bitmap reference contained in *RCM_iconBitmapSource*.

RCM_iconBitmapSize stores the size of the icon bitmap, in bytes.

RCM_overwrite indicates whether the existing complex moniker chunks should be overwritten. Pass TRUE if you wish this occur.

MSG_COMPLEX_MONIKER_REPLACE_MONIKER also passes a **ReplaceComplexMonikerChunkHandles** structure. This structure is used to place returned chunk handles of the text and bitmap.

```
typedef struct {
    ChunkHandle   RCMCH_topText;
    word          RCMCH_unusedCX;
    ChunkHandle   RCMCH_iconBitmap;
    word          RCMCH_unusedBP;
} ReplaceComplexMonikerChunkHandles;
```

RCMCH_topText contains the ChunkHandle of the text portion of the new complex moniker.

RCMCH_iconBitmap contains the ChunkHandle of the bitmap (graphics) portion of the new complex moniker.

2.3 Convenient Dialog Routines

The Foam library provides a number of routines useful for displaying simple dialog boxes to the user. Each of these displays a dialog box. The dialog box will have one or more buttons. Most of the functions have a return value; the function's return value will be an **InteractionCommand** value corresponding to which button the user pressed to dismiss the dialog. The **FoamDisplay...NoBlock()** routines don't have an explicit return value; they send a message if the user presses the OK button.

The following routines are available:

FoamDisplayWarning(), **FoamDisplayWarningNoBlock()**
: These routines present a dialog box notifying the user about an action. The user may dismiss the dialog, but may not cancel the action. The function takes one argument, the optr of a string buffer to display. This function has no return value.

FoamDisplayError(), **FoamDisplayErrorNoBlock()**
: These routines present a dialog box notifying the user that an error has occurred. The user may dismiss the dialog via the OK button. The function takes one argument: the optr of the string buffer containing the text to display. The function has no return value.

FoamDisplayOkCancelWarning()
: This function presents the user with a warning in an OK/Cancel dialog. The function takes one argument: the optr of the string buffer to display. This function returns an **InteractionCommand** value; if the user presses the OK button, the function returns IC_APPLY.

FoamDisplayQuestion()
: This routine presents a Yes/No dialog. It takes one argument, the optr of the text buffer containing the string to display. If the user presses the Yes button, the function will return IC_YES.

FoamDisplayNote(), **FoamDisplayNoteNoBlock()**
These functions display an OK dialog box displaying a text message to the user. The function takes one argument, the optr to a string buffer with the text to display.

FoamDisplayDeleteWarning()
This routine presents an OK/Cancel dialog box asking for a file deletion confirmation. This function takes three arguments: a pointer to a string buffer containing the file name, and optrs to two string chunks: one to display before the file name, and one to display after. If the function returns IC_YES, the user has confirmed that the file should be deleted.

FoamDisplayDeleteWarningNoBlock()
This routine presents an OK/Cancel dialog box asking for a file deletion confirmation. This function takes three arguments: the optr of the text string asking the user for confirmation, the message to send if the Delete button is pressed, and the object to send that message to. This function has no explicit return value. If the user Cancels the dialog, it won't send any message.

For dialog box types not provided by these functions, you will have to set up your UI, using GenInteraction objects in the usual way—or you may use the **FlashingNoteClass** or **FoamProcessDialogClass**, described later in this chapter.

2.4 FlashingNoteClass

Sometimes, you may want to send a notification to a user without requiring the user to respond. Ideally, you would like a dialog box to appear on screen

for a few seconds, and then go away, or automatically disappear once any key is pressed. You can get this behavior with a **FlashingNoteClass** object.

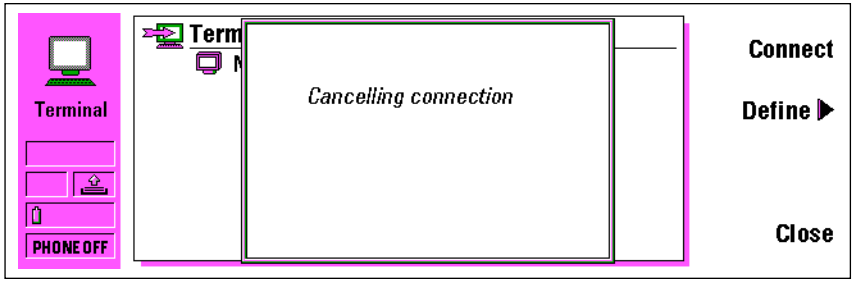


Figure 2-2 *FlashingNote*
The "Cancelling Connection" dialog appears for a period of time (the default is 3 seconds) and then disappears. The user need not respond to this information.

You can provide the following parameters to a Flashing Note object:

- A minimum duration to stay on-screen. After the minimum duration, the dialog can be dismissed by pressing any key.
- A maximum duration to stay on-screen. (The dialog will dismiss itself if this time is reached.)

A Flashing Note should be placed as the root of an unattached tree within its own UI block (resource). In this manner, it will act as a template. To initiate the flashing note, call the routine **PutUpDialogViaUIThread(),** passing the optr of the dialog box template. This routine will copy the template resource, attach it to the application tree, and manage its thread of execution. The dialog will be run within the specific UI thread, avoiding blocking the application thread.

2.4.1 FlashingNote Instance Data

FlashingNoteClass only contains one piece of instance data, but it also changes the defaults for its superclass, **GenInteractionClass**.

Code Display 2-9 FlashingNoteClass Instance Data

```
#define FLASHING_NOTE_DEFAULT_DURATION          (3*60) /* 3 seconds */
#define FLASHING_NOTE_DEFAULT_MINIMUM_DURATION  (1*60) /* 1 second */

@instance word          FNI_duration;

@default GII_attrs =            GIA_NOT_USER_INITIATABLE | GIA_SYS_MODAL);
@default GII_visibility =       GIV_DIALOG;
@default GII_type =             GIT_MULTIPLE_RESPONSE;
@default FNI_duration =         FLASHING_NOTE_DEFAULT_DURATION;

@vardata word   HINT_FLASHING_NOTE_DESTROY_SELF_AFTER_DISMISS;
@vardata word   ATTR_FLASHING_NOTE_MINIMUM_DURATION;
```

FNI_duration stores the maximum duration (in ticks — 1/60ths of a second) for the dialog to remain on-screen. Do not set this to zero; doing so will make the dialog box “permanent” and incapable of dismissal.

2.4.2 Setting Up the Template

Usually, a **FlashingNoteClass** object (and its children) should be contained within their own resource. The resource should remain unattached to the application tree. The routine **PutUpDialogViaUIThread()** will take care of copying and attaching the group of objects.

To set up your template, perform the following steps:

- Declare the template’s resource and make it *notDetachable*.
- Define the root object as a **FlashingNoteClass** object.
- Set the root object not usable (~GS_USABLE).
- Set the *FNI_duration* instance field to the number of ticks (1/60ths of a second) you wish to have the dialog box on-screen. By default, this is 180 ticks (3 seconds).
- If you want a minimum time for the dialog box to remain on-screen, add ATTR_FLASHING_NOTE_MINIMUM_DURATION and set it to the number of ticks (1/60ths of a second) to remain on-screen. By default, this is 60 ticks (one second).

- Add HINT_FLASHING_NOTE_DESTROY_SELF_AFTER_DISMISS if you don't want to manually destroy it.
- Add the hints HINT_WINDOW_NOT_TITLE_BAR and HINT_DRAW_IN_BOX.

Code Display 2-10 FlashingNote Template

```
@start FlashingNoteTemplateResource, notDetachable;

@object FlashingNoteClass FlashingNoteTemplate = {
    GI_states = @default & ~GS_USABLE;
    GI_comp = @DialogBoxTemplate;
    FNI_duration = 600;                          /* 10 seconds */

    HINT_DRAW_IN_BOX;
    HINT_FLASHING_NOTE_DESTROY_SELF_AFTER_DISMISS;
    HINT_WINDOW_NO_TITLE_BAR;
    ATTR_FLASHING_NOTE_MINIMUM_DURATION = 300;   /* 5 seconds */
}

/*
 * We want a separator, so we will create a sub-group interaction as a complex
 * moniker.
 */

@chunk TCHAR DialogTemplateText[] = “Flashing Note”;

@object ComplexMonikerClass DialogBoxTemplate = {
    ComplexMoniker = GenInteractionClass;
    CMI_topText = @DialogTemplateText;
    CMI_fontSize = FOAM_NORMAL_FONT_SIZE;
    GI_comp = @DialogText;

    HINT_PLACE_MONIKER_ABOVE;
    HINT_COMPLEX_MONIKER_DRAW_SEPARATOR;
    HINT_FIXED_SIZE = { SST_PIXELS | FOAM_DEFAULT_BUBBLE_WIDTH, 0 };
}

@chunk TCHAR DialogTextText[] = “This particular flashing note will stay up a
                                minimum \
                                of 5 seconds, and will dismiss itself if you
                                press any key after that.\r\r\
                                It will dismiss itself automatically after 10
                                seconds.\r”;
```




```
@object GenTextClass DialogText = {
    GTXI_text = @DialogTextText;
    GI_attrs = @default | GA_READ_ONLY;
}

@end FlashingNoteTemplateResource;
```

2.4.3 Initiating and Destroying the Flashing Note

Initiating the flashing note is simple. Simply call **PutUpDialogViaUIThread()**, passing the optr of the root Flashing Note object.

If that object contains HINT_FLASHING_NOTE_DESTROY_AFTER_DISMISS, the dialog will be automatically detached and destroyed after it disappears from view (though the template block will remain untouched, of course).

2.5 FoamProgressDialog

Your application may occasionally need to perform a task of a long duration. To prevent user frustration, it is wise in these occasions to use a **FoamProgressDialogClass** object to inform the user of the progress of the task. This gives the user much-appreciated feedback in such times.

The FoamProgress dialog provides the following functionality:

- A default animation of gears in motion, informing the user that an action is taking place.
- A “status indicator” consisting of a series of five dots. These five dots get displayed as the task gets closer to completion. Next to this status indicator, you may include status text, if desired.
- Descriptive text of the task.

- A default Cancel trigger.

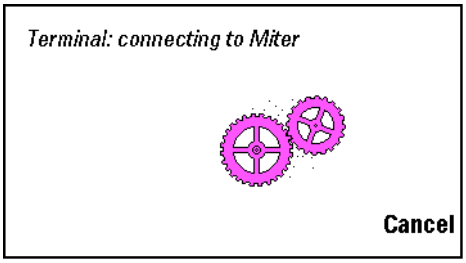


Figure 2-3 *FoamProgressDialog*
The rotating gears animation is accomplished by means of a FoamProgressDialogClass object which cycles through four monikers. The cancel trigger is provided by default.

As with a Flashing Note, a FoamProgress dialog should be placed as the root of an unattached tree within its own UI block (resource). To initiate the Foam Progress dialog, call the routine **PutUpDialogViaUIThread(),** passing the optr of the dialog box template. This routine will copy the template resource, attach it to the application tree, and manage its thread of execution. The dialog will be run within the specific UI thread, avoiding blocking the application thread.

The dialog can be taken down with **TakeDownDialogViaUIThread()** or by placing HINT_FOAM_PROGRESS_DIALOG_DESTROY_SELF_AFTER_DISMISS on the object.

2.5.1 FoamProgressDialog Instance Data

FoamProgressDialogClass contains several pieces of instance data, and also alters the defaults of its superclasses, **GenControlClass** and **GenInteractionClass**.

Code Display 2-11 FoamProgressDialogClass Instance Data

```
@instance       ChunkHandle     FPDI_descriptionText    = NullChunk;
@instance       ChunkHandle     FPDI_statusText         = NullChunk;
@instance       optr            FPDI_cancelDestination  = NullOptr;
@instance       word            FPDI_cancelActionMsg    = 0;
```

```
@vardata void    HINT_FOAM_PROGRESS_DIALOG_DESTROY_SELF_AFTER_DISMISS;
@vardata void    ATTR_FOAM_PROGRESS_DIALOG_DONT_DISMISS_ON_CANCEL;

@default         GII_visibility  = GIV_POPUP;
@default         GII_type        = GIT_MULTIPLE_RESPONSE;
@default         GII_attrs       = (GIA_NOT_USER_INITIATABLE | GIA_MODAL);
@default         GI_attrs        = GA_SIGNAL_INTERACTION_COMPLETE;
@default         GI_states       = (@default | GS_ENABLED);
```

FPDI_descriptionText contains text to display to the user, informing them of the task being performed.

FPDI_statusText contains text to display next to the status indicator.

FPDI_cancelDestination contains the object that should receive notification (via *FPDI_cancelActionMsg*) if the action is cancelled by the user. The object must be capable of handling cancellation in a clean manner.

FPDI_cancelActionMsg stores the message to send if the user cancels the operation. This message will be sent to the *FPDI_cancelDestination*, where it should be handled appropriately.

HINT_FOAM_PROGRESS_DIALOG_DESTROY_SELF_AFTER_DISMISS indicates that the resource block containing this dialog should be destroyed after the dialog is dismissed. This hint indirectly calls **UserDestroyDialog();** do not use this hint if you will use **TakeDownDialogViaUIThread()** instead.

ATTR_FOAM_PROGRESS_DIALOG_DONT_DISMISS_ON_CANCEL indicates that the dialog will not be dismissed if the user presses "Cancel." The application is responsible for handling this situation, and eventually dismissing the dialog.

2.5.2 Setting Up the Template

A **FoamProgressDialogClass** object (and its children) should be contained within their own resource. The resource should remain unattached to the

application tree. The routine **PutUpDialogViaUIThread()** will take care of copying and attaching the group of objects.

To set up your template, perform the following steps:

- Declare the template's resource and make it *notDetachable*.
- Define the root object as a **FoamProgressDialogClass** object.
- Set the root object not usable (~GS_USABLE).
- Set the object GIA_SYS_MODAL (and make sure to turn off its default GIA_MODAL flag).
- Set the *FPDI_descriptionText* instance field with text describing the action.
- Set *FPDI_statusText* with text to reside along the status indicator.
- Set HINT_DRAW_IN_BOX.

Code Display 2-12 FoamProgressDialog Template

```
@start FoamProgressDialogTemplate, notDetachable;

@chunk TCHAR DescriptionText[] = “While, we’re waiting a series of dots will
appear below.”

@chunk TCHAR StatusText[] = “Here are the dots: “;

@object FoamProgressDialogClass ProgressSample = {
    FPDI_descriptionText = @DescriptionText;
    FPDI_statusText = @StatusText;
    GII_attrs = (@default | GIA_SYS_MODAL) & ~GIA_MODAL;
    GI_states = (@default) & ~GS_USABLE;
    HINT_DRAW_IN_BOX;
}

@end FoamProgressDialogTemplate;
```

2.5.3 Using a FoamProgressDialog

MSG_FOAM_PROGRESS_DIALOG_APPEND_STATUS_TEXT



Code Display 2-13 Using a FoamProgressDialog

```
static ShowProgressDialog ()
{
    optr                progressDialog;
    word                i;
    SemaphoreError      semResult;
    TCHAR               statusAppendStr[] = " . ";

    progressDialog = PutUpDialogViaUIThread(@FoamProgressDialogTemplate);

    for ( i=0; i<5; i++) {
        semResult = ThreadPTimedSem(cancelSemaphore, 0);

        if (semResult == SE_TIMEOUT) {
            @send ProgressDialog::MSG_MY_PROGRESS_DIALOG_CANCEL_RECEIVED();
        break;
        }

        else (
            ThreadVSem(cancelSempahore);
            TimerSkeep(30);
            @call progressDialog::MSG_FOAM_PROGRESS_DIALOG_APPEND_STATUS_TEXT(
                                statusAAppendStr);
        }
    }

    TimerSleep(30);
    TakeDownDialogViaUIThread(progressDialog);

}

/* Handler for MSG_MY_PROGRESS_DIALOG_CANCEL_TRIGGERED */

@method MyProgressDialogClass, MSG_MY_PROGRESS_DIALOG_CANCEL_TRIGGERED
{
    SemaphoreError      semResult;

    semResult = ThreadPSem(cancelSemaphore);

    EC_ERROR_IF(
        (semResult == SE_TIMEOUT), CANNOT_GRAB_SEMAPHORE_CANCELPROGDIALOG);
}

/* Handler for MSG_MY_PROGRESS_DIALOG_CANCEL_RECEIVED */
```

```
@method MyProgressDialogClass, MSG_MY_PROGRESS_DIALOG_CANCEL_RECEIVED
{
    ThreadVSem(cancelSemaphore);
}
```

2.6 Routines for Lists

There are two routines useful for presenting the user with a list that is alphabetically sorted and/or presents has multi-column items.

The **FoamGetLexicalOrder()** routine is used for alphabetically ordering strings. It takes a char value and returns a byte-size number. This number represents that character's position in a sorted multi-language "alphabet". If you are presenting the user with a sorted list of strings, this is the routine to use to sort the list.

The **CreateVisMonikerLine()** routine puts together a multi-column visual moniker. It puts the moniker together from an array of strings and/or bitmaps you supply. It returns the optr of the new moniker. It takes the following arguments:

- a font point size: All text will use this point size.
- a font: All text will use this font.
- the number of columns
- an array of **VisMonikerColumn** structures: This data contains the data which will be combined into the multi-column moniker.

The **VisMonikerColumn** structure is defined:

```
typedef struct {
      byte            VMC_width; /* 0 for bitmaps */
      const void *    VMC_ptr;
      Justification   VMC_just;
      TextStyle       VMC_style;
      ColumnBorder    VMC_border;
} VisMonikerColumn;
```




For each field, pass a *VMC_width* to specify the number of pixels to allow to draw the text, or 0 to signal that this column is a bitmap. *VMC_ptr* should point to a string buffer if the column contains text; it should point to a simple bitmap if the column contains a bitmap.

When you are done using the multi-column moniker, it is up to you to free its memory. Get the memory block handle from the optr and pass that handle to **MemFree()**.

SMS Guide



3

3.1 SMS Overview 67
3.2 Using the Mailbox Library 68
3.2.1 Mailbox Overview 68
3.3 Sending an SMS Message 69
3.3.1 Sending and the Mailbox Library 70
3.3.2 Using the SMS Mailbox API 70
3.3.2.1 Setting Up the Text Message 71
3.3.2.2 Storing the Text Within a VM File 72
3.3.2.3 Setting Up the SMSending Options 75
3.3.2.4 Setting Up the MailboxRegisterMessageArgs 78
3.3.2.5 Registering the Message 85
3.4 Receiving an SMS Message 87
3.4.1 SM Receive and the Mailbox Library 87
3.4.2 Receiving an SMS Message 88
3.4.2.1 Acknowledging Receipt of the Message 88
3.4.2.2 Retrieving the Text from the VM Block 89
3.4.2.3 Cleaning Up 90





The Short Message Service (SMS) is a protocol designed to transmit short alpha-numeric messages via a dedicated cellular provider's "service center." SMS is built into the GSM (Global System for Mobile communications) standard that has gained general acceptance within Europe; SMS will also most likely be integrated into whatever digital cellular network becomes the de-facto standard within the United States.

The Nokia 9000 Communicator is compliant with the transmission and reception of SMS protocol messages according to the GSM standard. With SMS, it is possible to create applications and services that request and retrieve short text messages (up to 160 characters) to any server or user on the GSM network. The ability to send and receive short messages can enable a wide variety of services. For example, short messages can be used to retrieve stock quotes, place horse-racing bets at a paramutual facility, or play chess with another member of the GSM network. The possible uses for the SMS protocol are almost limitless. A service need only be connected to the GSM network.

Geoworks' Mailbox library includes built-in transmission and receipt of SMS messages. The elements of the Mailbox library that are of concern to SMS application developers have been excerpted and included in this chapter.You will not need to understand the inner workings of the Mailbox library to write an SMS application or service.

Before reading this chapter, you should be familiar with cellular technologies, the SMS standard and elementary GEOS programming—in particular, the use of GEOS Virtual Memory (VM) files.

3.1 SMS Overview

The Short Message Service protocol handles a "short message" of up to 160 characters. The SMS standard of 160 characters allows you to send a text message to any single node (address) on the GSM network by way of a dedicated cellular service provider.



The cellular service provider is responsible for managing incoming SMS messages and holding them until the recipient's phone becomes active. The length of time that an SMS message should be held can be specified by the sending party. This is known as a *validity period*.

The cellular provider may also elect to deliver the message in a format other than SMS. This is done at the server through a *message conversion* into the alternative format (e.g., as a fax message, as an email, as a page, etc.) This alternative method of delivery may also be specified by the sending party (though, of course, the recipient must have the means to handle such a format).

Because the SMS message is so short, it is guaranteed to be delivered as a single packet of information.

3.2 Using the Mailbox Library

The Nokia 9000 Communicator comes with built-in capabilities to send and receive SMS messages. This is enabled through use of Geoworks' Mailbox library. Because the Mailbox library includes *all* Inbox/Outbox functionality (email, faxing, etc.), it contains a large amount of information that is not relevant to an SMS service developer. This section will only detail general information about the Mailbox library and API specific to SMS application developers.

3.2.1 Mailbox Overview

The purpose of the Mailbox library is to let applications send and receive information to and from other computers, through a variety of communications media. The mailbox insulates the application as much as possible from the mechanics of transmitting and receiving the information.

In order to use the Mailbox library, an application must package information in a format acceptable to both a data driver (which stores the message) and a transport driver (which sends the message) — in this case the SMS transport driver. The application then *registers* this message with the

Mailbox library. During the registration process, you specify in which data format the message is stored.

The Mailbox library stores messages that have not yet been transmitted in its *outbox*. Messages are held in the outbox until the hardware connection necessary for transmission is available. An application can also specify that a message not be delivered until a specific time. In this case, the message will automatically remain in the outbox until that time. While a message is in the outbox, the device's user can examine information about the message and delay or even cancel its delivery.

When the time comes to send the message, the Mailbox library activates the appropriate transport driver. This activation is transparent to an application developer.

3.3 Sending an SMS Message

There are three pieces of information that you will need to know when you send an SMS message. These are rather self-evident:

- The text message.
- The recipient's GSM phone number.
- The cellular provider's phone number. All SMS messages are routed through the cellular provider.

If you wish the SMS message to be sent to a particular application on the receiving Nokia 9000 Communicator, you will also need the following information:

- The application token (**TokenChars**) of the receiving app.
- The **ManufacturerID** of the receiving app.

In GEOS, these two pieces of information are referred to as a **GeodeToken**. If the application is your own, the **GeodeToken** is easy enough to determine (since you will have defined it yourself). If the application is not your own, you will need to contact that manufacturer or cellular provider for this information.



3.3.1 Sending and the Mailbox Library

Any SMS message that is sent out to the cellular provider is routed through the mailbox's *outbox*. There are three basic paradigms involved in packaging up information for use by the outbox. Although knowledge of how they work is not essential to sending an SMS message, it may be important when troubleshooting problems.

Apart from any parameters sent to the mailbox, the three basic pieces of information the outbox needs to know in order to send a message are:

- In what data format the message is "wrapped." The mailbox can handle any data format as long as it is wrapped (contained) in a format it can understand. The mailbox calls an appropriate "data driver" when storing the message before it is sent.
- The type of data being sent. (The mailbox, itself, does not need to know the actual data; it only needs to know how the data is wrapped.) Telling the mailbox what the data is provides a black box abstraction.
- The transport driver to use when sending the message.

When a message is placed in the outbox, the mailbox calls the appropriate data driver to store the data. When a connection is made, the mailbox calls the appropriate transport driver and sends the message. Note that the mailbox itself does not need to know the message's contents, or even its format.

3.3.2 Using the SMS Mailbox API

`mailbox.goh`

To send an SMS message to the outside world, an application will need to perform the following steps:

1 Set up the text message. In many cases, the text will reside in a simple text buffer. If you wish to address the SMS message to a particular application, you will need to prepend a nine-character prefix to the text message. The message itself will then be, at most, 151 characters in length. Without a prefix, you may send a 160-character SMS message. (See "Setting Up the Text Message" on page 71.)

2. Copy the text into a VM file. The Mailbox library may store an SMS text message in a variety of formats (indicated by type **GeoworksMailboxDataFormatID**), but a VM file is the easiest to manipulate. This VM file will be accessed by a Mailbox data driver. (See "Storing the Text Within a VM File" on page 72.)
3. Set up the **SMSendingOptions** parameters in a DB item. These will be used by the SMS transport driver when the message is delivered. (See "Setting Up the SMSending Options" on page 75.)
4. Set up the **MailboxRegisterMessageArgs** structure. (See "Setting Up the MailboxRegisterMessageArgs" on page 78.)
5. Call **MailboxRegisterMessage()**. (See "Registering the Message" on page 85.)
6. Upon success, notify the Mailbox library that we are done with the VM file.

These steps are detailed in the sections following.

3.3.2.1 Setting Up the Text Message

foam.goh

The Short Message Service protocol is able to handle a "short message" of up to 160 characters. The Foam library defines this character limit:

```
#define FOAM_MAX_SMS_TEXT_SIZE 160
```

The Nokia 9000 Communicator also allows you to further specify a *particular* application within the Nokia 9000 Communicator by adding a prefix to the SMS message (thus reducing the amount of useful text).

This prefix consists of two backslashes ("//"), a 4-byte GEOS application token (**TokenChars**), a 4-byte GEOS **ManufacturerID**, and a carriage return (ASCII 13, or 0x0D hex). The Mailbox library will correctly read all characters up to the carriage return.

For example:

```
//SKAA11\r
```

is a valid prefix.

The application **TokenChars** and **ManufacturerID** should match those of the intended *receiving* application. If you define your own application, and only wish to communicate with its counterpart running on another device, you will define this string in the application's **.gp** file. The string is constructed from the *tokenchars* and *tokenid* lines of the **.gp** file.

You may also send a message to a different application, as long as you know its **TokenChars** and **ManufacturerID**.

This prefix should leave at least 149 characters to store the alpha-numeric text message. When concatenating two strings, be sure that you do not overflow the bounds of the array.

Code Display 3-1 Setting Up an SMS Text Message

```
/*
 * Declare the buffer to store the text message and write the prefix to the buffer.
 */
TCHAR smsMessage[FOAM_MAX_SMS_TEXT_SIZE + 1] = "//SKAA11\r";
TCHAR smsText[] = "Here's my test message. I better make sure it fits.";

/*
 * Add the body of the text to the buffer but first check the bounds.
 */
if ( strlen( smsMessage ) + strlen( smsText ) <= FOAM_MAX_SMS_TEXT_SIZE )
{
    strcat( smsMessage, smsText );
}
```

3.3.2.2 Storing the Text Within a VM File

`MailboxGetVMFile()`, **`Mailbox\vmtree.goh`**

Text within an SMS message needs to be stored by the Mailbox until it connects with a cellular service provider. When you register a message with the Mailbox library, you indicate what storage format you have selected. The Mailbox uses a *data driver* to store and retrieve messages. There are a number of data drivers available, but we only need to know a few.

SMS messages may either be stored as VM (Virtual Memory) trees or as GEOS files, within the Mailbox itself. VM trees offer a lower overhead, and are simpler to use, so that is the format we have documented. (It is possible that future data drivers will become available as well.) If you are unfamiliar with GEOS VM trees, consult the Virtual Memory chapter of the Concepts book.

The mailbox data driver stores its data in what is called a VM tree. This tree is referenced by a **VMTreeAppRef** structure.

```
typedef struct {
    VMChain       VMTAR_vmChain;
    VMFileHandle  VMTAR_vmFile;
} VMTreeAppRef;
```

VMTAR_vmChain stores the **VMChain** to pass to the data driver. For SMS messages, simply use the VMCHAIN_MAKE_FROM_VM_BLOCK macro to construct a **VMChain** from a single **VMBlockHandle**.

VMTAR_vmFile stores the **VMFileHandle** which contains the VM block.

The Mailbox stores these VM trees locally within an administation file that it maintains. To store the text within a Mailbox-compatible VM file, perform the following steps:

1. Call **MailboxGetVMFile()** to retrieve a VM file handle within the Mailbox. This routine expects an argument indicating the expected number of blocks to be added to the file; because the SMS message can be contained in a single block, pass 1. Also pass a buffer to store the returned **VMStatus**.

   This routine returns a **VMFileHandle**.

2. Allocate a VM block within this file, using **VMAlloc()**.

3. Lock the block down using **VMLock()**. This routine returns the segment address of the locked block. You will also receive a temporary memory handle that you can use to manipulate the block.

4. Copy the text from your text buffer into the locked block starting at its segment address. (You will need to skip two bytes for the **VMChainLink** that begins the block.)

5. Dirty the block using **VMDirty()**.



6 Unlock the block using **VMUnlock()**.

7 Store the VM file and VM block within a **VMTreeAppRef** structure. This format is understood by the mailbox data driver.

Code Display 3-2 Storing the Text In a VM File

```
VMFileHandle     vmFile;
VMBlockHandle    vmBlock;
TCHAR            *textPtr;
MemHandle        mh;
word             vmStatus;
VMTreeAppRef     vmRef;

/*
 * Obtain the handle of a writable VM file from the Mailbox library. We pass one
 * since we do not plan to add additional blocks to this file (because SMS messages
 * are so short).
 */
vmFile = MailboxGetVMFile(1, &vmStatus);

/*
 * Allocate a VM block and store the text within this block. The block will need
 * to fit a header (VMChainLink), the SMS message itself, and a null terminator.
 */
vmBlock = VMAlloc(vmFile, (sizeof(VMChainLink) + strlen(smsMessage) +1), 0);
charPtr = VMLock(vmFile, vmBlock, &mh);

/*
 * We advance past the VMChainLink contained at the beginning of the VM
 * block and copy the text to that location.
 */
strcpy(textPtr + sizeof(VMChainLink), smsMessage);

/*
 * Dirty the block and unlock it.
 */
VMDirty(mh);
VMUnlock(mh);
```

```
/*
 * Store the VM file and VM block within a VMTreeAppRef structure. This is the
 * format that the mailbox library understands. Essentially, it ia a VM “chain”
 * of a single VM file.
 */
vmRef.VMTAR_vmChain = VMCHAIN_MAKE_FROM_VM_BLOCK(vmBlock);
vmRef.VMTAR_vmFile = vmFile;
```

3.3.2.3 Setting Up the SMSending Options

MailboxGetAdminFile(), **smdefine.goh**, SMSendingOptions, ReadCSNumberFromIniFile()

When registering a mailbox message, the application must specify a data driver to use when storing the message. It will also need to specify which transport driver to use when the message is to be delivered. The GEOS SMS transport driver expects a number of parameters indicating how the SMS message should be sent. These parameters are passed as a **SMSendingOptions** structure within a GEOS **DBItem**.

To store these parameters within a compatible DB item, perform the following steps:

1. Call **MailboxGetAdminFile()** to retrieve the **VMFileHandle** of the Mailbox administration file. This is where you will copy your SMS parameters.
2. Allocate an ungrouped DB item within this file, using **DBAllocUngrouped()**.
3. Lock the DB item using **DBLockUngrouped()**.
4. Copy your basic **SMSendingOptions** parameters into this DB item.
5. Read the service center number from the .INI file using **ReadSCNumberFromIniFile()**. Copy that number into the DB item.
6. Dirty the block using **DBDirty()**.
7. Unlock the block using **DBUnlock()**.

The SMSendingOptions structure

The **SMSendingOptions** structure is defined within **smdefine.goh** and is shown below:

```
typedef struct {
      SMReplyPath       SMSO_replyPath;
      SMValidityPeriod  SMSO_validityPeriod;
      SMMessageConversionSMSO_messageConversion;
      TCHAR       SMSO_scAddress[MAX_ADDRESS_SIZE];
} SMSendingOptions;

#define MAX_ADDRESS_SIZE 22
```

SMSO_replyPath contains an **SMReplyPath** value which indicates whether the recipient may reply to the message through the sender's service provider. Possible values are:

SMRP_NO
This flag is set by default and indicates that the recipient must reply through the recipient's service provider.

SMRP_YES
Setting this flag allows the recipient to reply through the sender's service provider.

SMSO_validityPeriod contains an **SMValidityPeriod** value which indicates the length of time that the message is considered valid for delivery. The cellular provider should interpret this value and if it is unable to connect with the recipient, drop the message and inform the sender that the message was not delivered. Possible values are:

SMVP_1_HOUR
SMVP_6_HOURS
SMVP_24_HOURS
SMVP_1_WEEK
SMVP_MAX (The actual value of SMVP_MAX will vary by service provider.)

SMSO_messageConversion contains an **SMMessageConversion** value, indicating that you wish to have the SMS message delivered to the recipient in a different format than that of an SMS message

(provided that the cellular provider offers this service). Possible values are:

SMMC_NORMAL
Deliver the message as a normal SMS message. This is the default value.

SMMC_FAX_GROUP_3
Deliver the message as a fax according to the fax group 3 standard.

SMMC_FAX_GROUP_4
Deliver the message as a fax according to the fax group 4 standard.

SMMC_VOICE
Deliver the message as a voice mail message.

SMMC_ERMES
Deliver the message according to the European Radio Messaging System standard.

SMMC_PAGING
Deliver the message as an alpha-numeric page.

SMMC_EMAIL
Deliver the message as an Internet e-mail message.

SMMC_X400
Deliver the mesage according to the X400 e-mail standard.

SMSO_scAddress stores the address of the service center (cellular provider). This number is stored as an ASCII text string and is limited to 22 characters. This number should be retrieved from the Nokia 9000 Communicator's .INI file; the number is stored under the **scNumber** field within the **SMS** category. This "address" in this case is really just a numerical phone number.

Reading a String From the .INI File

`initfile.h, InitFileReadStringBuffer()`

To read a string from the .INI file, call **InitFileReadStringBuffer()**. Pass "SMS" as the category and "scNumber" as the key. The routine will read the entry into a buffer passed to the routine. It will also read the size of the string into another buffer.



Code Display 3-3 Setting the SMSendingOptions Parameters

```
SMSendingOptions        *ptrSendingOptions
VMFileHandle            adminFile;
DBGroupAndItem          smsParams;
word                    sizeOfBuffer;

adminFile = MailboxGetAdminFile();

smsParams = DBAllocUngrouped(adminFile, sizeof(SMSendingOptions));

ptrSendingOptions = (SMSendingOptions *) DBLockUngrouped(adminFile, smsParams);

/* set desired sending options */
ptrSendingOptions->SMSO_replyPath = SMRP_NO;
ptrSendingOptions->SMSO_validityPeriod = SMVP_24_HOURS;
ptrSendingOptions->SMSO_messageConversion = SMMC_NORMAL;

/* be sure to initialize these fields as well, otherwise the app will crash! */
ptrSendingOptions->SMSO_dataCodingScheme = 0;
ptrSendingOptions->SMSO_userDataLength = 0;
ptrSendingOptions->SMSO_userDataLength = 0;

InitFileReadStringBuffer("SMS",
                        "scNumber",
                        ptrSendingOptions->SMSO_scAddress,
                        0,
                        &sizeOfBuffer);

DBDirty(ptrSendingOptions);
DBNUnlock(ptrSendingOptions);
```

3.3.2.4 Setting Up the MailboxRegisterMessageArgs

`MailboxRegisterMessageArgs, MailboxTransAddr`

By now, you have created a reference to the SMS message itself (within a **VMTreeAppRef** structure); you have also placed the SMS sending parameters within the mailbox administration file. You are now ready to load the **MailboxRegisterMessageArgs** structure with parameters. (Two of these parameters will be the location of the SMS message and its sending parameters.)

1. Indicate the format in which you wish the SMS message stored while it resides in the outbox. (The mailbox will call the appropriate mailbox data driver when the message is registered.)
2. Indicate the format of the actual message. This is where you tell the mailbox that this message is an SMS message.
3. Indicate the VM file in which the SMS message is stored.
4. Tell the mailbox to use the SMS transport driver when the mailbox sends the message.
5. Give the mailbox the address of the recipient.
6. Indicate the DB item where the **SMSendingOptions** are located.
7. Tell the mailbox when to begin delivery of the message and when to give up delivery attempts if unsuccessful.

All of these steps are accomplished by simply filling in a **MailboxRegisterMessageArgs** structure. This structure is large, but it is not altogether complicated. Each field is described in the following sections, though only SMS relevant information is shown. It is shown below:

```
typedef struct {
      MailboxStorage          MRA_bodyStorage;
      MailboxDataFormat       MRA_bodyFormat;
      const void              *MRA_bodyRef;
      word                    MRA_bodyRefLen;
      MailboxTransport        MRA_transport;
      MailboxTransportOption  MRA_transOption;
      MailboxTransAddr        *MRA_transAddrs;
      word                    MRA_numTransAddrs;
      dword                   MRA_transData;
      MailboxMessageFlags     MRA_flags;
      const char              *MRA_summary;
      GeodeToken              MRA_destApp;
      FileDateAndTime         MRA_startBound;
      FileDateAndTime         MRA_endBound;
} MailboxRegisterMessageArgs;
```




Data Format for Storage

As we mentioned, the Mailbox will need to store a message in a format that it can understand. Rather than have the Mailbox understand every data format that is (or will ever be) developed, it is easier to "package" the message in a small number of standard formats. You specify in what format the message will be wrapped in the *MRA_bodyStorage* field.

MRA_bodyStorage indicates under what format (of type **MailboxStorage**) the message will be stored in the internal Mailbox administration file. **MailboxStorage** is defined as follows:

```
typedef struct {
      word              MS_id;
      ManufacturerID    MS_manuf;
} MailboxStorage;
```

MS_id stores a word of data indicating the type of data driver that will be used to store the message. For SMS messages, we will use a Geoworks data driver referenced by a **GeoworksMailboxStorageID**. The following types are valid:

```
GMSID_FILE
GMSID_VM_TREE
```

As we indicated earlier, SMS messages should be stored in VM trees. You do have the option of storing them as GEOS files as well, but VM trees are easier to manipulate.

MS_manuf stores the **ManufacturerID** of the data driver being used. Since we will be using Geoworks VM trees, we will store MANUFACTURER_ID_GEOWORKS in this location.

Data Format of the Message

MRA_bodyFormat indicates the format of the actual data being stored (in its original source). This field takes an argument of type **MailboxDataFormat**. This is defined below:

```
typedef struct {
      word              MDF_id;
      ManufacturerID    MDF_manuf;
} MailboxDataFormat;
```

MDF_id stores a word of data indicating the data stored in the message. We will use a Geoworks data format referenced by a **GeoworksMailboxDataFormatID**. Pass GMDFID_SHORT_MESSAGE in this field to indicate that the message is a Geoworks SMS message.

MDF_manuf stores the **ManufacturerID** of the format. Since we will be using Geoworks SMS , we will store MANUFACTURER_ID_GEOWORKS in this location.

References to the VM Tree Structure

MRA_bodyRef stores a pointer to the reference of the SMS message. Since we will be storing the message in a VM tree, we will reference that message with a **VMAppTreeRef**.

MRA_bodyRefLen stores the length of the reference indicated in *MRA_bodyRef*. In this case, we will store the size of the **VMAppTreeRef** structure.

The Transport Driver

The Mailbox uses an applicable transport driver when it wants to deliver a message to the outside world. You specify this transport driver in the *MRA_transport* field.

MRA_transport takes an argument of type **MailboxTransport**. This is defined below:

```
typedef struct {
      word              MT_id;
      ManufacturerID    MT_manuf;
} MailboxTransport;
```

MT_id stores a word of data indicating the type of transport driver that will be used to deliver the message. For SMS messages, we will use a Geoworks transport driver referenced by a **GeoworksMailboxTransportID**.

To send an SMS message, we will use the SM transport driver referenced by GMTID_SM.

MT_manuf stores the **ManufacturerID** of the transport driver being used. Since we will be using Geoworks SM transport

driver, we will store MANUFACTURER_ID_GEOWORKS in this location.

MRA_transOption stores an unsigned value that the the transport driver uses. For SMS, set this to zero.

MRA_transAddrs stores the address of the eventual recipient. This address is stored as a pointer to a **MailboxTransAddr** structure; it is only important that the transport driver understand the format for this address.

If there are multiple recipients, this pointer references multiple **MailboxTransAddr** structures. This structure is defined below:

```
typedef struct {
      const void        *MTA_transAddr;
      unsigned          MTA_transAddrLen;
      const char        *MTA_userTransAddr;
} MailboxTransAddr;
```

MTA_transAddr stores a pointer to the recipient address, understood by the transport driver. (For SMS this is a simple ASCII text string of the recipient's phone number.)

MTA_transAddrLen stores the length of the address stored in *MTA_transAddr*.

MTA_userTransAddr stores a user-readable version of the address, as a null-terminated text string. Because the SMS address is already stored as an ASCII text value, this field will be the same as *MTA_transAddr*.

MRA_numTransAddrs stores the number of recipient addresses stored in *MRA_transAddrs*.

MRA_transData stores data understood by the transport driver. As mentioned in "Setting Up the SMSending Options" on page 75, the SMS transport driver expects a **DBItem** containing a structure of type **SMSendingOptions**.

MRA_flags stores a field of **MailboxMessageFlags.**
For SMS messages, the only flag you may care about is MMF_DELETE_BODY_AFTER_TRANSMISSION, which will

delete the VM file containing the SMS message after it is delivered. Do not set any other flags.

MRA_summary stores a text string that the Outbox displays to indicate the type of message being delivered. For example, if you are sending a chess move for a game in progress, you might set this field to display "Chess Move."

MRA_destApp stores the **GeodeToken** of the destination application.

MRA_startBound indicates a time and date (of type **FileDateAndTime**) to begin transmission of the message. You may also pass the value MAILBOX_NOW (0) for immediate delivery.

MRA_endBound indicates a time and date (of type **FileDateAndTime**) to end attempts to transmit the message to the service provider. You may also pass the value MAILBOX_ETERNITY (-1) to indicate attempt until successful.

Code Display 3-4 Setting Up the Mailbox Registration Arguments

```
MailboxRegisterMessageArgs      registerArgs;
MailboxTransAddr                addressList;
TCHAR                           number[MAX_ADDRESS_SIZE]

/*
 * We tell the mailbox that the data is stored within a VMTreeAppRef.
 */
registerArgs.MRA_bodyStorage.MS_id = GMSID_VM_TREE;
registerArgs.MRA_bodyStorage.MS_manuf = MANUFACTURER_ID_GEOWORKS;

/*
 * We tell the mailbox that this is an SMS message.
 */
registerArgs.MRA_bodyFormat.MDF_id = GMDFID_SHORT_MESSAGE;
registerArgs.MRA_bodyFormat.MDF_manuf = MANUFACTURER_ID_GEOWORKS;

/*
 * We tell the mailbox where the body text is stored.
 */
MRA_bodyRef = &vmRef;
registerArgs.MRA_bodyRefLen = sizeof(VMTreeAppRef);
```




```
/*
 * We tell the mailbox to use the SM transport driver when sending the message.
 */
registerArgs.MRA_transport.MT_id = GMTID_SM;
registerArgs.MRA_transport.MT_manuf = MANUFACTURER_ID_GEOWORKS;

registerArgs.MRA_transOption = 0;

/*
 * Normally, we would retrieve the phone number from an outside source (such as
 * the contact database). Here, we just set it to a specific value.
 */

number = “13125551212”;

/*
 * We need to stuff each address (in this case, a phone number) into a
 * MailboxTransAddr structure.
 */

addressList.MTA_transAddr = &number;
addressList.MTA_transAddrLen = strlen(number);
addressList.MTA_userTransAddr = &number;

registerArgs.MRA_transAddrs = &addressList;
registerArgs.MRA_numTransAddr = 1;

/*
 * We pass the previously set up SMSendingOptions in the DBGroupItem to the
 * transport driver.
 */
registerArgs.MRA_transData = smsParams;

registerArgs.MRA_flags = MMF_DELETE_BODY_AFTER_TRANSMISSION;

registerArgs.MRA_summary = “Chess Move”;

strcpy(registerArgs.MRA_destApp.GT_chars, “SKAA”);
registerArgs.MRA_destApp.GT_manufID = MANUFACTURER_ID_TBD;

registerArgs.MRA_startBound = MAILBOX_NOW;
registerArgs.MRA_endBound = MAILBOX_ETERNITY;
```

3.3.2.5 Registering the Message

`MailboxRegisterMessage(), MailboxDoneWithVMFile()`

Finally, you have everything set up. To send the message, all you need to do is register the message with the mailbox library.

MailboxRegisterMessage() takes an address (of type **MailboxMessage**) as a second parameter. The routine will write a token value into this address so that future mailbox routines can refer to this message by its token.

Do not confuse this token with the actual message (which is sent through the **MailboxRegisterMessageArgs**).

After you have successfully registered the message, call **MailboxDoneWithVMFile()** to tell the mailbox to get rid of the temporary VM file it had created.

Code Display 3-5 Using MailboxRegisterMessage

```
MailboxRegisterMessageArgs      registerArgs;
MailboxMessage                  msgToken;
MailboxError                    mailboxError;
VMFileHandle                    vmFile;

mailboxError = MailboxRegisterMessage(&registerArgs, &msgToken);

if (mailboxError == ME_SUCCESS) {

    /* If we want to do anything special upon success, here it would reside ... */

};

MailboxDoneWithVMFile(vmFile);
```

There is a possibility that **MailboxRegisterMessage()** may return an error (of type **MailboxError**) if something is not working correctly (or if the message was not set up correctly). The routine returns ME_SUCCESS if the registration was successful.



The following are valid mailbox error codes:

```
ME_SUCCESS = 0
ME_NOT_ENOUGH_MEMORY
ME_CANNOT_LOAD_DATA_DRIVER
ME_CANNOT_LOAD_TRANSPORT_DRIVER
ME_UNABLE_TO_CREATE_TRANSMIT_THREAD
ME_CANNOT_ENQUEUE_MESSAGE
ME_USER_CANCELED
ME_LOST_CONNECTION
ME_CANNOT_CREATE_MESSAGE_FILE
ME_CANNOT_RESIZE_MBOX_REF
ME_CANNOT_SAVE_MESSAGE_FILE
ME_CANCELLED_BY_RECEIVER
ME_MESSAGE_BLOCKS_ARE_MISATCHED
ME_DATA_DRIVER_CANNOT_STORE_MESSAGE_BODY
ME_INSUFFICIENT_DISK_SPACE
ME_CANNOT_CONNECT
```

The following are unrecoverable **MailboxError** error codes. (The high bit is set if this is the case.)

```
#define ME_UNRECOVERABLE 0x8000
ME_ADDRESS_INVALID
ME_DATA_DRIVER_CANNOT_ACCESS_MESSAGE_BODY
ME_UNSUPPORTED_BODY_FORMAT
ME_MESSAGE_BODY_INVALID
ME_INVALID_MESSAGE
ME_REPLY_ADDRESS_NOT_AVAILABLE
ME_DESTINATION_APPLICATION_UNKNOWN
ME_UNKNOWN_DISK_ERROR
ME_APP_REF_BUF_TOO_SMALL
ME_CANNOT_OPEN_MESSAGE_FILE
ME_CANNOT_READ_MESSAGE_FILE
ME_CAN_NEVER_CONNECT
```

3.4 Receiving an SMS Message

There are two ways the Nokia 9000 Communicator may receive an SMS message: standard, or targeted.

As a developer, you do not need to concern yourself with standard messages since a "standard" message is handled by the Nokia 9000 Communicator's built-in SMS application. This application handles standard messages that are either sent without an SMS prefix or that are sent specifically to the SM Receive application.

Targeted messages are those messages sent using the nine character prefix. These messages may not necessarily be sent using another Nokia 9000 Communicator; any device that uses the correct SMS prefix can contact a specific application if it so desires. In any case, the nine character prefix (containing the **GeodeToken**) will specify the application that is meant to receive the message.

3.4.1 SM Receive and the Mailbox Library

When the GSM network informs the unit that is has an SMS message pending for receipt, it launches the SM Receive application. This application retrieves the SMS message(s). If the message is a standard SMS message, it routes the message to the application's "Received messages" folder.

If the message is targeted to a specific application, the SM Receive application contacts the mailbox and places the message in the mailbox's *inbox*. The mailbox then contacts the application by sending it a message (MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE). If the mailbox cannot contact the application, it re-routes the message back to the SM Receive application.

You do not need to know how the SM Receive application works, but it is useful to know the process which occurs when an SMS message is received.

3.4.2 Receiving an SMS Message

When the inbox receives a message that it determines is destined for a specific application, the mailbox will contact that application's process class with MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE. This message passes a token identifer of the SMS message.

If you wish your application to handle an SMS message, you will need to write a message handler for MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE. It should perform the following steps:

1. Call **MailboxAcknowledgeReceipt()** to inform the mailbox that we're handling the message. Pass the token identifer of the SMS message that was passed with MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE. (See "Acknowledging Receipt of the Message" on page 88.)
2. Retrieve the VM reference to the message with **MailboxGetBodyRef()**. (See "Retrieving the Text from the VM Block" on page 89.)
3. Lock down the VM block and retrieve a pointer to the text.
4. Copy the text to a local buffer.
5. Unlock the VM block.
6. Call **MailboxDoneWithBody()** to instruct the mailbox to release the VM reference to the text. (See "Cleaning Up" on page 90.)
7. Call **Mailbox DeleteMessage()**, passing the token referring to this message.

These specific steps are described in more detail below:

3.4.2.1 Acknowledging Receipt of the Message

`MailboxAcknowledgeMessageReceipt()`

When your application receives an SMS message with MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE, your first order of business should be informing the mailbox that you have the message. To do this, send the mailbox **MailboxAcknowledgeMessageReceipt()**, passing it the message token.

Code Display 3-6 Acknowledging Receipt of an SMS Message

```
/*
 * Each application which wishes to receive SMS messages should create a
 * MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE handler in the app's main prcoess
 * class.
 */

@method MyProcessClass, MSG_META_MAILBOX_NOTIFY_MESSAGE_AVAILABLE {

    /*
     * First we inform the mailbox that we've received the message. We pass the
     * 'msg' token that was passed to this message in the first place.
     */

    MailboxAcknowledgeMessageReceipt(msg);

    /* To be continued ... */
```

3.4.2.2 Retrieving the Text from the VM Block

`MailboxGetBodyRef()`

As we indicated in"Storing the Text Within a VM File" on page 72, it is convenient for the SMS message itself to be wrapped in a small **VMChain** structure. In the following section, we assume that is indeed what was done on the sending end.

Use **MailboxGetBodyRef()** to retrieve this reference to the actual text. Pass the message token, a pointer to a buffer to store the body reference, and the length of that buffer.

Code Display 3-7 Retrieving the Text

```
    MailboxError        errorCode;
    VMAppTreeRef        vmRef;
    word                refSize;
    MemHandle           handleToText;
    TCHAR               smsText[FOAM_MAX_SMS_TEXT_SIZE+1];
    char                *ptrToText;
```

```
/*
 * First, retrieve the reference to the body of the SMS text from the mailbox
 * library. Place that reference into our vmRef variable.
 */

refSize = sizeof(VMTreeAppRef);
errorCode = MailboxGetBodyRef(msg, &vmRef, &refSize);

if (errorCode == ME_SUCCESS) {

    /*
     * Lock the block and retrieve a pointer to the SMS text.
     */
    ptrToText = (char *) VMLock(vmRef.VMTAR_vmFile,
                            VMCHAIN_GET_VM_BLOCK(vmRef.VMTAR_vmChain),
                            &handleToText);

    /*
     * Advance the pointer beyond the carriage return to get beyond the
     * prefix.
     */

    while ( *ptrToText != '\r')
        ptrToText++;

    /* Advance over the '\r' ...*/
        ptrToText++;

    strcpy(smsText, ptrToText);

    VMUnlock(handleToText);

    /* To be continued ... */
```

3.4.2.3 Cleaning Up

`MailboxDoneWithBody(), MailboxDeleteMessage()`

Finally, we need to notify the mailbox that it can delete both the text of the message (and the reference to that text) and the message itself. To perform each of these steps, call **MailboxDoneWithBody()** and **MailboxDeleteMessage()**.

Code Display 3-8 Cleaning Up the Mailbox

```
        /* Now remove the external text and the reference to that text. */

        MailboxDoneWithBody(msg, &vmRef, refSize)

        /*
         * Finally, delete the “message” including the message token and any
         * information included with it.
         */

        MailboxDeleteMessage(msg);

    }

}
```



N9000C

Contacts



4

4.1 Setting up a Contact List 97
4.1.1 Multiple-Selection ContactLists 100
4.1.2 Remembering the Search 100
4.1.3 Detecting Unknown Phone Numbers 100
4.2 Choosing Contacts From a Log 101
4.3 Using a Dedicated Contact 103
4.4 Logging Calls 106
4.5 To Get More Info on a Contact 108





The Nokia 9000 Communicator has a Contact database which acts as a central repository of "address book" information. Every time the user enters information about how a place may be reached by phone, fax, or e-mail that information is stored in the Contact database.

Whenever you prompt the user for information about where to send something, you'll probably use the Contact database. This will allow the user to choose which of their contacts should receive.

There are two libraries which exist to keep track of the Contact database and provide an API for the applications which share access to it.

- The *ContDB* library keeps track of the contact information, and may be used to look up or edit contact information. It also provides a controller which allows the user to choose a contact from a list of existing contacts.
- The *ContLog* library provides an API for logging what sorts of communication has been made with the entities described in the Contact database. It also provides a controller which allows the user to select a recently contacted contact.

The Contact database is a Foam database, and some operations upon the database will make use of Foam database commands. We will not explore the meanings of these commands closely here, but will show how they are used in the context of the Contact database.

Before reading this chapter, you should be familiar with controllers, Object-Oriented Programming, and the GEOS messaging system. It helps if you try out the "Contacts" built-in application, which demonstrates much of the functionality we'll discuss here.

Most programmers first encounter the ContDB and ContLog libraries when they need to intercept a message associated with a **ContactList** or perhaps a **RecentContactsControl** object. These object classes provide the user with a standard way to choose a contact.

There are other controllers which allow the user to choose an SMS contact. **SMAddressControlClass** allows the user to choose a recipient for a SMS message. This class should cover some applications' needs when it comes to allowing the user to choose a contact. However, if you need a custom control,

you may wish to use a **ContactList** and/or a **RecentContactsSMSControl**.

A **ContactList**, as shown below (Figure 4-1), presents the user with a list of known contacts. The user may choose a name from the list. In section 4.1 on page 97, we show how to set up such an object and get the phone number of the contact the user selects.



Figure 4-1 *ContactList Object*
The ContactList object provides a list of contacts and a Search bar.

The **RecentContactsSMSControl**, shown below (Figure 4-2) presents the user with a list containing names and numbers associated with contacts that have recently had SMS transactions with the user. The user may choose a name or number from the list. In section 4.2 on page 101, we show how to set up such an object and get the phone number of the contact the user selects.

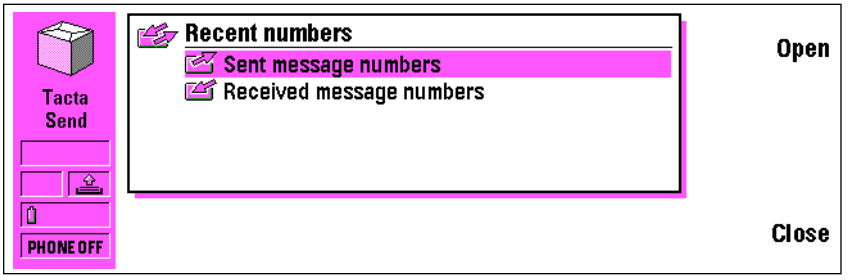


Figure 4-2 *RecentContactsSMSControl*
The RecentContactsSMSControl objects provides a list of recently contacted contacts. This gives the user a quick way to send messages to their most common contacts.

4.1 Setting up a Contact List

`ContactListClass`,
`CONTACT_LIST_NOTIFY_CONTACT_SELECTED_MSG`,
`ContactGetDBHandle()`, `FoamDBGetRecordFromID()`,
`FoamDBGetFieldData()`, `FoamDBDiscardRecord()`,
`ContactReleaseDBHandle()`

To present the user with a list of Contacts, do the following:

Make sure that your .gp file contains lines requiring the **foamdb** and **contdb** libraries:

```
library contdb
library foamdb
```

In your source code, create a **ContactList** object. The following **ContactList** object declaration allows the user to pick a contact. If the contact has more than one GSM phone number, the controller will make the user choose one of the fields.

- When the user chooses a contact, the ContactList will send a message to an object The **ContactList**'s *GCI_output* and `ATTR_CONTACT_LIST_NOTIFY_CONTACT_SELECTED_MSG` specify the object and message. In the example below, the application's process object will handle a message. We will discuss the message in more detail below.
- To make the **ContactList** SMS-specific, set its ATTR_CONTACT_LIST_CALL_TYPE to CCT_SMS as shown below; also, set the CLF_SELECT_CONTACT_NUMBER feature flag in ATTR_GEN_CONTROL_REQUIRE_UI.
- To include the **ContactList**'s Search bar, set the CLF_SEARCH feature flag in ATTR_GEN_CONTROL_REQUIRE_UI; to exclude the Search bar, exclude the UI.
- Optionally, you can use the standard Contact List bitmap by making your **ContactList** a **ComplexMoniker** and setting its *CMI_iconBitmap* field to CMB_CONTACT_LIST.



Putting this together, we end up with the following:

```
@object ContactListClass MyContactList = {
  GI_visMoniker = “I want to talk to…”;
  GCI_output = (TO_PROCESS);
  ATTR_CONTACT_LIST_CALL_TYPE = CCT_SMS;
  ATTR_GEN_CONTROL_REQUIRE_UI = CLF_SEARCH |
                        CLF_SELECT_CONTACT_NUMBER ;
  HINT_EXPAND_HEIGHT_TO_FIT_PARENT;
  ATTR_CONTACT_LIST_NOTIFY_CONTACT_SELECTED_MSG =
                         MSG_MYPROCESS_SEND_MESSAGE ;
}
```

With this configuration, the **ContactList** will send a MSG_MYPROCESS_SEND_MESSAGE message to your process object when the user chooses a contact. A sample handler for this message is shown below (section Code Display 4-1 of chapter 4). The message will conform to the prototype:

```
@prototype void
    CONTACT_LIST_NOTIFY_CONTACT_SELECTED_MSG(
                                        dword recordID,
                                        word fieldID);
```

The message handler should carry out the following steps:

1. Get the Contact database's handle. To do this, call **ContactGetDBHandle()**. This routine takes no arguments, and returns the Contact database's handle.

2. Get the handle of the record by calling **FoamDBGetRecordFromID()**. This routine takes the Contact's database handle and the record's **RecordID** value. It allocates a block of memory to hold the record's data and returns the record's handle.

3. To extract the contact's GSM phone number, call **FoamDBGetFieldData()**. Pass the database handle, the record handle, the **FieldID** associated with the GSM number field (this number is supplied by the **ContactList** when it sends this message), a buffer to write the name to, and the size of that buffer. The function fills in the buffer with the phone number string, and returns the length of that string. The returned string might not be null-terminated; thus you will either need to keep track of the returned string length, or else terminate the string.

4. Now that you're done with the record, let the database know by calling **FoamDBDiscardRecord()**. This routine takes the Contact's database handle and the record's handle as arguments.

5. Now that you're done with the database, release its handle by calling **ContactReleaseDBHandle()**.

The example below shows a way of carrying out the above set of steps.

Code Display 4-1 Handling the ContactList's "Selected" Message

```
@class MyProcessClass, GenProcessClass;
        @message (CONTACT_LIST_NOTIFY_CONTACT_SELECTED_MSG)
                                                MSG_MYPROCESS_SEND_MESSAGE;
@endc

@method MyProcessClass, MSG_MYPROCESS_SEND_MESSAGE {
        TCHAR           theNumber[31];
        TCHAR           theName[MAX_NAME_DATA_LEN+1];
        word            numLen;
        VMFileHandle    CDBHandle;
        MemHandle       theRecord;

                /* Get the Contact database's handle */
        CDBHandle = ContactGetDBHandle();

                /* Get the handle to the record we're interested in */
        theRecord = FoamDBGetRecordFromID( CDBHandle, recordID );

                /* Get the GSM phone number string; null-terminate it. */
        FoamDBGetFieldData( CDBHandle, theRecord, fieldID, theNumber, 30);
        theNumber[numLen] = '\000'; /* add null terminator */

                /* We'll copy the name to theName buffer. */
        ContactGetName(theRecord, theName);

                /* If you need more information about the contact
                 * (e.g., business name, job title), this is a good place
                 * to retrieve it. */

        /* Do what you like with theNumber and theName */

                /* We're done with this record. */
        FoamDBDiscardRecord(CDBHandle, theRecord );
```




```
            /* We're done with the database. */
        ContactReleaseDBHandle();
}
```

4.1.1 Multiple-Selection ContactLists

Recent versions of the ContDB library support ContactLists which allow the user to select more than one contact at a time. This feature is available on devices with version numbers "Responder Build 4...." and higher. See "Update," Chapter 13 for information about finding out the software version of the user's device.

To create such a ContactList, give it an ATTR_CONTACT_LIST_NON_EXCLUSIVE_LIST variable data field. To query such a list to find out which records are selected, use MSG_CONTACT_LIST_GET_IDS_OF_SELECTED_RECORDS.

4.1.2 Remembering the Search

Normally, a ContactList will lose track of the user's current search if the focus moves to another object. To force the list to remember the current search, send it a MSG_CONTACT_LIST_PRESERVE_SEARCH_ON_NEXT_LOST_FOCUS before it loses the focus. (This message is available on devices with version numbers "Responder Build 4...." and higher. See "Update," Chapter 13 for information about finding out the software version of the user's device.)

4.1.3 Detecting Unknown Phone Numbers

If the user enters a valid phone number in the search box of aContactList, but that number is not found, an application mayrequest that the ContactList send out a notification using a variable data field. (This variable data field is available on devices with version numbers "Responder Build 4...." and

higher. See "Update," Chapter 13 for information about finding out the software version of the user's device.)

Give the ContactList a ATTR_CONTACT_LIST_NOTIFY_NUMBER_ENTERED_MSG variable data field. This field's data is a message.

The message should have parameters:

```
void MSG_MY_CL_NOTIFY( MemHandle phNumber = cx);
```

The passed block contains the phone number string. The message's recipient is responsible for freeing the passed block.

4.2 Choosing Contacts From a Log

`RecentContactsSMSControlClass, CONTACTS_CONTROL_NOTIFY_ENTRY_SELECTED_MSG`

For a list of recent SMS contacts, use a **RecentContactsSMSController**. You will need the following line in your **.gp** file:

```
library     contlog
```

The following setup would allow the user to pick a phone number from a list of recently called SMS numbers, sending a MSG_MYPROCESS_SEND_RECENT message to the application's process object:

```
@object RecentContactsSMSControlClass MyLog = {
        GCI_output = ( TO_PROCESS );
        ATTR_RECENT_CONTACTS_CONTROL_NOTIFY_ENTRY_SELECTED_MSG =
                                        MSG_MYPROCESS_SEND_RECENT;
}
```

With this setup, when the user chooses a phone number from the log, a MSG_MYPROCESS_SEND_RECENT message will be sent to the process object, which should handle the message. The message should conform to the CONTACTS_CONTROL_NOTIFY_ENTRY_SELECTED_MSG prototype.

```
@prototype void
    CONTACTS_CONTROL_NOTIFY_ENTRY_SELECTED_MSG(
                                RecentContactsData *data)
typedef struct {
      dword               RCD_contactID;
      NameOrNumber        RCD_number;
} RecentContactsData;
```

The *data* argument's *RCD_number* field contains the GSM phone number of the selected contact. The code example below shows handler that extract's the name associated with the selected contact. It makes use of functions described in section 4.1 on page 97.

Code Display 4-2 Handling the RecentContacts "Selected" Message

```
@class MyProcessClass, GenProcessClass;
        @message (CONTACTS_CONTROL_NOTIFY_ENTRY_SELECTED_MSG)
                                              MSG_MYPROCESS_SEND_RECENT;
@endc

@method MyProcessClass, MSG_MYPROCESS_SEND_RECENT {

        /* Note: all of the code in this example is only useful if you need
         * the contact's name. If all you need is the GSM phone number,
         * none of this code is necessary. The phone number string is in
         * data->RCD_number.
         *
         * If you do need the following code, you will need to make sure
         * that your .gp file includes the lines
               library contdb
               library foamdb
         */

        TCHAR           theName[MAX_NAME_DATA_LEN+1];
        VMFileHandle    CDBHandle;
        MemHandle       theRecord;

               /* If the user chose a phone number that doesn't have a contact
                * associated with it, the contact's RecordID value is -1,
                * signalling an invalid contact. */
        if ( data->RCD_contactID != LECI_INVALID_CONTACT_ID) {
         CDBHandle = ContactGetDBHandle();
         theRecord = FoamDBGetRecordFromID( CDBHandle, data->RCD_contactID );
```

```
        ContactGetName(theRecord, theName);

               /* If you need more information about the contact
                * (e.g., business name, job title), this is a good place
                * to retrieve it. */

        FoamDBDiscardRecord(CDBHandle, theRecord );
        ContactReleaseDBHandle();
       }

       /* do something clever with data->RCD_number and theName */
}
```

4.3 Using a Dedicated Contact

`ContactMatchName(), ContactEnsureField()`

Perhaps you're writing a special-purpose application that is supposed to communicate with just one contact. Perhaps a carry-out restaurant chain has contracted you to write an application that the user can use to place an order over the phone. The program would only need to make contact with one place: the local restaurant's GSM phone number.

In this case, you wouldn't want to provide any gadgetry for choosing a contact—your program is only interested in one dedicated contact. Still, it's worthwhile to store the contact in the Contact database—if the restaurant's phone number changes, the user can enter the corrected number using the Contact Manager built-in application.

To find the Contact record for a particular contact, you will need to make sure that your **.gp** file contains the following lines:

```
library contdb
library foamdb
```




Code Display 4-3, below, shows an example of how you could find a dedicated contact in the Contact database, and extract the phone number associated with that contact.

To locate a dedicated contact and extract its phone number, carry out the following steps:

1 To get the **RecordID** of the contact's record, call **ContactMatchName()**. This routine takes a search string (pass the contact's name), a **ContactFieldType** (pass CFT_NAME, since we're searching by name), a flag (pass a non-zero value to specify that you're only interested in one matching record), a pointer to a **FieldID** buffer to fill in and a pointer to a word value to fill in. It returns the **RecordID** of the contact (or -1 if the contact wasn't found). It also fills in the **FieldID** buffer with the contact's name field **FieldID** and fills in the word value with the number of matches found; neither of these values is important to our purpose.
If the contact is not found, there are a couple of approaches the application can take:
• Create a new record with the proper name and a guess at the phone number. The code example below illustrates this approach.
• Signal to the user that an error has occured and halt the current operation.

2 Get the Contact database's handle. To do this, call **ContactGetDBHandle()**. This routine takes no arguments and returns the database's handle.

3 Get the record's handle by calling **FoamDBGetRecordFromID()**. This routine takes the Contact database's handle and the record's **RecordID**. It returns the record's memory handle.

4 Get the **FieldID** of the record's GSM phone number field. In the example below, we assume that the record has only one phone number field, and that this is the field we want. For a more rigorous approach, we could examine each field to make sure it was named "Tel (GSM)". For an example of how one might examine all the fields of a record, see section 4.5 on page 108.
For our shortcut, we use the **ContactEnsureField()** function. This function takes as arguments a record handle, a field name string (in our example, we pass a null optr to specify that we should use the default field name, and the **ContactFieldType** of the field to find (pass CFT_PHONE). The function returns a **FieldID**.

5 To extract the phone number data, call **FoamDBGetFieldData()**. Pass the database handle, the record handle, the **FieldID**, a buffer to write the name to, and the size of that buffer. The function fills in the buffer with the phone number string, and returns the length of that string. The returned string might not be null-terminated; thus you will either need to keep track of the returned string length, or else terminate the string.

6 Now that you're done with the record, let the database know by calling **FoamDBDiscardRecord()**. This routine takes the Contact's database handle and the record's handle as arguments.

7 Now that you're done with the database, release its handle by calling **ContactReleaseDBHandle()**.

Code Display 4-3 Finding the "Stuckey's Snack Shack" Contact

```
TCHAR           theName[]=      "Stuckey's Snack Shack";
TCHAR           gsmNum[MAX_NUMBER_FIELD_DATA_LEN] = "1.800.788.2539"
word            gsmNumLen;
FieldID         theField;
word            dummyNumMatches;
RecordID        theRecordID;
Memhandle       theRecord;
VMFileHandle    CDBHandle;

theRecordID = ContactMatchName(theName, CFT_NAME,1, &theField, &dummnyNumMatches);
if ( theRecordID == LECI_INVALID_CONTACT_ID) { /* If we didn't find the record…*/
  theRecord = ContactCreateRecordFromTemplate(); /* … then we create it */

  /* Fill in the name field; here @TelGSMStr is a chunk holding the string
   * "Tel(GSM)" */
  theField = ContactEnsureField( theRecord, NullOptr, CFT_NAME );
  CDBHandle = ContactGetDBHandle();
  FoamDBSetFieldData( CDBHandle, theRecord,theField, theName, sizeof(theName)-1 );
  /* Fill in the fax field of new record with default GSM number string */
  /* Fill in the name field; here @TelGSMStr is a chunk holding the string
   * "Tel(GSM)" */
  theField = ContactEnsureField( theRecord, @TelGSMStr, CFT_PHONE );
  FoamDBSetFieldData( CDBHandle, theRecord, theField, gsmNum, 14 );

  ContactReleaseDBHandle();
  /* Save the newly created record to the Contact DB */
  theRecordIIndex = ContactSaveRecord( theRecord );
  theRecord = FoamDBGetVisibleRecord( CDBHandle, theRecordIndex );
}
```




```
else { /* If we successfully found the record, then get its handle */
CDBHandle = ContactGetDBHandle();
theRecord = FoamDBGetRecordFromID( CDBHandle, theRecordID);
}

/* Whether we found the record or just now created it, we now have
 * its handle and can extract the information that we want. */
  /* Fill in the name field; here @TelGSMStr is a chunk holding the string
   * “Tel(GSM)” */
theField = ContactEnsureField( theRecord, @TelGSMStr, CFT_PHONE );
/* Copy the fax number into our buffer */
gsmNumLen = FoamDBGetFieldData( CDBHandle, theRecord, theField, gsmNum,
                                                MAX_NUMBER_FIELD_DATA_LEN);
gsmNum[gsmNumLen]= ‘\000‘

        /* If we need more information about the contact (e.g., street address,
         * fax number, etc.) this would be a good place to get it. */

FoamDBDiscardRecord( CDBHandle, theRecord );
ContactReleaseHandle();

/* Do something clever with theName and gsmNum */
```

To allow the user to edit the contact information for this single contact, use a **ContactEdit** object. To specify the contact that the user will work with, you will need its handle. Pass the handle as an argument to the MSG_CONTACT_EDIT_DISPLAY_RECORD message. (Don't call **ContactSaveRecord()**, **FoamDBDeleteRecord()**, or **FoamDBDiscardRecord()** when done with this handle; the controller will do so.)

4.4 Logging Calls

`LogAddEntry()`

If you wish to log incoming or outgoing messages, you will use the **LogAddEntry()** function, provided by the **contlog** library. Make sure that you your **.gp** file contains the line

```
library contlog
```



When it comes time to make the call that you want to log, carry out the following steps:

1. Set up a **LogEntry** structure and call **LogAddEntry()**. The code shown in Code Display 4-4 would be appropriate for logging an outgoing message; to log an incoming message, substitute LED_RECEIVED for LED_SENT.
2. Make the call.
3. When the call is done, call **LogAddEntry()** again, passing the same **LogEntry** structure as before, after writing the duration (in seconds) in the **LogEntry**'s *LE_duration* field. (The previous invokation of **LogAddEntry()** will have filled in the **LogEntry**'s *LE_flags* field with correct values.)

Code Display 4-4 Logging an SMS Call

```
LogEntry MyEntry ;
dword       duration;
TimerDateAndTime tdat;
TimerGetDateAndTime(&tdat);
duration = TimerGetCount();

strcpy( MyEntry->LE_number, theNumber);
MyEntry.LE_contactID = recordID;
MyEntry.LE_type = LET_SMS;
MyEntry.LE_direction = LED_SENT;
MyEntry.LE_duration = 1;
MyEntry.LE_dateTime.DAT_year = tdat.TDAT_year;
MyEntry.LE_datetime.DAT_month = tdat.TDAT_month;
MyEntry.LE_datetime.DAT_day = tdat.TDAT_day;
MyEntry.LE_datetime.DAT_hour = tdat.TDAT_hours;
MyEntry.LE_datetime.DAT_minute = tdat.TDAT_minutes;

LogAddEntry( &MyEntry );

/* Send the message, and when done...*/

MyEntry.LE_duration = TimerGetCount() - duration;

LogAddEntry( &MyEntry );
```




When using the Contact Log library to log a contact, if you don't know the phone number, you can pass LEF_WILDCARD_NUMBER to **LogAddEntry()** to specify that this call's number should match any phone number. This is only allowed on devices with version numbers "Responder Build 4...." and higher. See "Update," Chapter 13 for information about finding out the software version of the user's device.

4.5 To Get More Info on a Contact

`FoamDBFieldEnum(), FoamDBGetFieldType(), FoamDBGetFieldName()`

In the examples above, we've shown how to get a selected contact's name and GSM phone number. It is possible to get more information about a contact given its **RecordID** number.

In the examples shown in Code Display 4-1, Code Display 4-2, and Code Display 4-3 there is a comment noting a good place to get more information about a contact. If, for some reason, you need to find out information about a Contact in a context other than shown in one of those examples,

1. You will need the record's **RecordID** number. Presumedly, you've stored away this number after using code like that shown in one of the examples shown above.
   To query a **ContactList** object for the **RecordID** of its selected record, use the MSG_CONTACT_LIST_GET_ID_OF_SELECTED_RECORD message. This message takes no arguments, and returns a **RecordID**.
2. Gain access to the Contact database by calling **ContactGetDBHandle()**, storing the returned handle.
3. To access the database record corresponding to the selected contact, call **FoamDBGetRecordFromID()**. This returns the record's handle.
4. Get the contact information you need. Information about how you might do this is described in the paragraphs following this checklist.
5. When you're done with the record handle, call **FoamDBDiscardRecord()** to free the handle.

6 Be sure to call **ContactReleaseDBHandle()** to signal you're done with the database. You shouldn't keep the handle around for long; doing so prevents the file from being closed.

To find out any other sort of information about the contact, you'll need to access one of the fields of the record. Fields within a Foam DB record (the Contact DB is a Foam DB) are identified by a **FieldID** number.

The **FoamDBFieldEnum()** function provides a way to scan all the fields of a record. The following code snippet shows how you might use **FoamDBFieldEnum()** to search for all of a contact's voice phone numbers. **FoamDBFieldEnum()** calls a callback routine for each of the record's fields. **FoamDBFieldEnum()** takes as arguments a record handle, a pointer to a data buffer (this pointer will be passed to the callback routine; it gives you a way to pass data to the callback; in our example, we pass the record handle in the buffer), and a pointer to a callback function. The callback function is passed a **FieldHeader** structure and a pointer to the data buffer that was passed to **FoamDBFieldEnum()**. From the FieldHeader, it can get the **FieldID** from the **FieldHeader**'s *FH_id* field. To get information about each field, use the **FoamDBGetFieldType()**, **FoamDBGetFieldName()**, and/or **FoamDBGetFieldData()** routines. Each of these routines takes the contact database handle, a record handle, a **FieldID**, and a pointer to a buffer to fill with a return value.

Code Display 4-5 Checking All Fields of a Contact

In this example, we check all fields of a contact, and build up an array with the names and data for those fields that correspond to voice phone numbers.

In this example, all we know at the start is the record's RecordID number.

```
void BuildNumberList( RecordID recordID ) {
        /* Scan all fields of the Contact record */
        /* For each one that corresponds to a voice phone number, store
         * the field's name (e.g., "Phone") and data (e.g., "555-1212") in
         * the global Data array. */

        VMFileHandle    CDBHandle;
        MemHandle       curRecord;

        CDBHandle = ContactGetDBHandle();
        curRecord = FoamDBGetRecordFromID( CDBHandle, recordID );
        NumberOfNumbers = 0     /* Global variable:
                                        number of elements in Data array*/
```




```
        FoamDBFieldEnum( curRecord, &curRecord, &BuildNumberListCallback );
        FoamDBDiscardRecord( CDBhandle, curRecord) ; /* Done with the record */
        ContactReleaseDBHandle(); /* Done with the Contact DB for now */
}

Boolean _pascal BuildNumberListCallback( FieldHeader *field, void *enumData ) {
        VMFileHandle    CDBHandle;
        byte            fieldType;
        MemHandle       curRecord = *((MemHandle *) enumData );

        CDBandle = ContactGetDBHandle();
        FoamDBGetFieldType( CDBHandle, curRecord, field->FH_id, &fieldType);
        if (fieldType == CFT_PHONE ) { /* For each voice phone number */
                                /* To look for something other than a phone
                                 * number, we would use a different
                                 * ContdbFieldType value. */
                                        /* Grab the field name */
          FoamDBGetFieldName( CDBHandle, curRecord, field->FH_id,
                        Data[NumberOfNumbers].fName, MAX_FIELD_LABEL_LEN);
                                        /* Grab the field data */
                                        /* If the data has zero length, func will
                                         * return zero. */
          if (FoamDBGetFieldData( CDBHandle, curRecord, field->FH_id,
                        Data[NumberOfNumbers].fData, MAX_FIELD_NUMBER_DATA_LEN)) {
                                        /* If the data was not of zero length
                                         * (wasn't just a blank field), then
                                         * up our NumberOfNumbers. */
            NumberOfNumbers++;
          }
        }
        ContactReleaseDBHandle(); /* We're done with the database */
                                /* if we return non-zero, we'll stop Enum'ing
                                 * through the fields. Since we only have room
                                 * for 10 fields in our array… */
        return( NumberOfNumbers > 10) ;
}
```

As shown above, to get the field data and field name associate with a field, use the **FoamDBGetFieldData()** and **FoamDBFGetFieldName()** functions. Many users will leave blank fields in their contact records; to test for a blank field, check **FoamDBGetFieldData()**'s return value; if it's zero, then the user has not filled in that field of the contact information.

4.6 The User's Own Data

`ContactGetUserDataName(), ContactGetUserDataID()`

Recent versions of the ContDB library provide routines which access the user's own contact information. These routines are only available on devices with version numbers "Responder Build 4...." and higher. See "Update," Chapter 13 for information about finding out the software version of the user's device.

The ContDB library provides two routines useful for accessing the contact information the user has specified for themself. The **ContactGetUserDataName()** routine returns the user's name string. **ContactGetUserDataID()** returns the record ID number of a record which contains the user's contact information.

112



Distributing Your Application



5.1 Package Description File 115
5.2 Setting Up a PC Server 118
5.3 Setting Up a Web Server 119

5





Once you've finished coding and testing your application, you'll want to distribute it to users. You'll need to take some time to figure out exactly what files make up your package—what files does the user need to use your product? Is it just an application geode? Perhaps an application geode plus a data file? The files that make up your software are known as a software package.

Once you've figured out what files make up your software package, there are two approaches to making your package available:

PC Download
: If all users will have access to a 486 (or better) PC running MS Windows 3.1 or MS Windows 95 and a serial or IR connection, and if you have some way to get files onto that PC, you can have users install your software with the built-in **Install/Remove software** application.
You will provide a package description file (described in the "Package Description File" section, below), and will need to set up server software on the PC (described in section 5.2 on page 118).

Web Site
: You can make the files available on a web site. Users with the ability to browse the web can download your program easily, thanks to installation savvy in the built-in **WWW** application. You will need to provide a package description file (described in the "Package Description File" section, below), and a bit more to get your server and files ready (described in section 5.3 on page 119). You will bundle each software package into an Add-On Software file (a specially tagged archive).

No matter how you're making your software available, you'll need to create its Package Description File.

5.1 Package Description File

No matter how you're making your program available, you'll need to provide a "package description file," sometimes known as the **.INS** file. This is an



ASCII text file which specifies important information about the files that make up your program:

- Name of the package, which the **Install/Remove software** application displays to the user
- Description of the package, which the **Install/Remove software** application displays to the user
- Order in which files should be installed.
- Names and locations of the files on the host machine
- Where the files should be copied to on the client phone

This installation order is important in case of interrupted installation—if file *A* depends upon file *B*, but installation was interrupted after file *A* was brought over, but before file *B* was, then file *A* won't work, since it depends on an absent file. If another package is relying upon an older, successfully installed version of file *A*, then you've just broken that other file.

Use the following format when creating the package description file:

- The file name must have a .**INS** suffix.
- Whitespace at the end of a line is ignored; blank lines are ignored.
- The first line must say "GEOS Package Description File v1.0"
- The second line must contain the package name. This is the name by which the package will be identified to the user in the **Install/Remove software** application. The application doesn't have space for a long description, though—about 20 characters.
- The third line must contain a description of the package. When the user views the package in the **Install/Remove software** application, they may read this description before deciding whether to download/install the package. The application doesn't have space for a long description, though—about 16 characters.
- The fourth line must contain the size of the package, (in bytes). This is the combined size of all files in the package. This string must be at most 14 characters long. (Hopefully, this will be sufficient!) When computing the size, don't forget to include the size of the package description file itself. It's all right to over-estimate the size.

- The fifth line must consist of a period (.). (This delimiter separates information about the package as a whole from information about the individual files.)
- The rest of the file consists of pairs of lines:
  The first line of each pair contains the path of a file on the host machine (or within the Add-On Software file, which will be described later);
  The second line of each pair contains the destination of the file on the client phone.

  The host-machine location is relative to the location of the package description file. The destination location is relative to the base of the GEOS tree.

The line-pairs describing file locations must be ordered based upon the order in which the files should be installed. Libraries, fonts, etc. must be listed before the applications that depend on them.

Code Display 5-1 Sample Package Description File

This example shows a package description file for the Santa package. On the host machine, the package description file is in the C:\NSERVER directory, and the files making up the package are in the C:\NSERVER\SANTA directory. The Santa application depends on the Sleigh library, and the Rudy font; the Sleigh library depends on the Donner and Blitzen libraries.

```
GEOS Package Description File v1.0

Santa v1.0
Gives gifts.
15769
.

santa\rudy.fnt
userdata\font\rudy.fnt

santa\donner.geo
system\donner.geo

santa\blitzen.geo
system\blitzen.geo

santa\sleigh.geo
system\sleigh.geo

santa\santa.geo
world\extrapps\santa.geo
```

5.2 Setting Up a PC Server

The Nokia 9000 Communicator comes with a disk containing the software by which a PC may be made to act as a server machine for the **Install/Remove software** application. The phone can make either a serial or IR connection with the PC. (For an IR connection, the PC must be running MS Windows 95 and the IR monitoring software which comes with it.)

1 From MS Windows 3.1 or Windows 95, run the program called **server** on the floppy disk to install a program called **NServer** onto the PC.

2 Run **NServer** to turn your PC into a server for the Nokia 9000 Communicator. This program monitors one of the PC's COM ports.

3 Make sure that the **NServer** program is monitoring the right COM port—check its Settings.

If the user makes a serial or IR connection between a Nokia 9000 Communicator and a PC running the NServer, then runs the **Install/Remove software** application on the phone, they can install packages from the PC to their phone.

The **Install/Remove software** application needs to know which directory to scan for Package Definition files. By default, the Install/Remove software application will look in the PC's A:\NSERVER directory—allowing for easy installation directly from floppy. If the files reside on the PC's hard drive, they should probably reside in C:\NSERVER; if they don't, make sure that the user knows where to look for them. To avoid clutter, you might want to put each "package" of files into its own directory under the directory containing the package description files.

5.3 Setting Up a Web Server

To make Nokia 9000 Communicator software packages available on a web site, the web site administrator will do some set-up work so that the server knows how to tag Add-On Software files. Each software package must be bundled up into an Add-On Software file—a special kind of archive.

Standard Add-On Software files will have a **.AOS** suffix, and should be identified by the MIME-type *application/x-nokia-9000-communicator-add-on-software*. Exactly how this is set up depends upon your server software. NCSA's httpd daemon may use a file called **mime.types**, to which you would add a line reading

```
application/x-nokia-9000-communicator-add-on-software aos
```

Once the server has been set up correctly, each software package needs to be bundled up into an Add-On Software file.

1 Pack the package descriptor file and all files in the software package into one **tar** archive file. (This is done using the **tar** program. **Tar** is a standard Unix program; there is a public domain DOS **tar** program. The



host-file-location lines in the package description file should describe locations within the tar file's directory structure.) The .**INS** file must be in the top level directory of the tar file so that it may be found. Read your **tar** documentation to find out how to create the archive. An appropriate invocation for one version of **tar** might be:

```
tar cvf santa2.aos santa2.ins santa2/santa2.geo
```

2 Give the tar file's filename the suffix .**AOS** so that it will be recognized as an Add-On Software file.

3 Put the file up on your web server.

Users of the Nokia 9000 Communicator using the **WWW** built-in application will be able to use **WWW** to install the software package from your site. The **WWW** program will download the Add-On Software file, extract the files from the archive, and put them into the proper directories as directed by the package description file.

Memory Levels



6.1 Memory Levels 123
6.2 Modifying Files on the FFS 123
6.3 Operations that Consume FFS Space 124
6.4 Utility Space Routines 125

6





This chapter collects various utility routines and concepts that may not adequately be addressed within other chapters.

If you are writing an application that requires use of memory, you will want to keep track of how much memory is available on the Nokia 9000 Communicator's flash file system (FFS).

6.1 Memory Levels

Operations that require writing to the FFS should almost always check "disk space" both before and after the operation. There are three defined levels of available disk space on the device. These levels are:

- Not full (more than 50kb available)
- Nearly full (between 25 and 50kb available)
- Full (less than 25 kb available)

You can check for these conditions by using the following constants:

- FOAM_DISK_SPACE_NEARLY_FULL_LEVEL
- FOAM_DISK_SPACE_FULL_LEVEL

FFS checking is usually needed at the following times:

- Writing files to disk
- Engaging in an operation that consumes memory

6.2 Modifying Files on the FFS

`FoamWarnForSaving(), FoamSaveAction`

Whenever you perform an operation that modifies a file on the FFS, you should make sure that you have enough space to save the file. For example,



you may bring up a document, modify it, and then wish to close the app, saving the document as it now exists.

In those case, use **FoamWarnForSaving()** to make sure you can proceed with the operation. Pass this routine the number of bytes required for the operation. The routine checks if there is enough memory available for the operation. If there is, it returns FSA_SAVE_DATA. Otherwise, it initiates a dialog box that asks you whether you wish to either throw away the changes or cancel the operation.

FoamWarnForSaving() returns one of the following **FoamSaveAction** values:

FSA_SAVE_DATA
: The operation is well within the limits.

FSA_THROW_AWAY_CHANGES
: The operation could not proceed due to memory constraints, and the user chose to discard the changes. Proceed accordingly. (You might, for example, allow the app to close.)

FSA_DO_NOTHING
: The operation could not proceed due to memory constraints, and the user chose to cancel the operation. Proceed accordingly. (You should not allow the app to close, for example.)

6.3 Operations that Consume FFS Space

`FoamWarnIfNotEnoughSpace(), FoamCheckIfOutOfSpace(), FoamDiskSpaceStatus`

Some operations may consume memory once they are begun, and it might be difficult to stop them once they start. For example, receiving a fax, reading or receiving e-mail, downloading something from the Web or printing a document may all consume FFS space.

FoamWarnIfNotEnoughSpace() takes a numeric value corresponding to the bytes required for the operation. If the operation would result in memory

falling below the critical level, the routine will notify the user that it has insufficient memory to complete the operation (via a dialog box) and will return FDDS_FULL.

If the operation can proceed otherwise, it will return the appropriate **FoamDiskSpaceStatus** value:

FDDS_NOT_FULL
FDDS_NEARLY_FULL
FDDS_FULL

FoamCheckIfOutOfSpace() also returns a **FoamDiskSpaceStatus** but does not initiate any dialog box.

You should call **FoamWarnSpaceAfter Operation()** after any operation that consumes FFS space. If the operation resulted in an FDDS_NEARLY_FULL status, the routine brings up a dialog alerting the user that the device is nearly out of FFS space.

6.4 Utility Space Routines

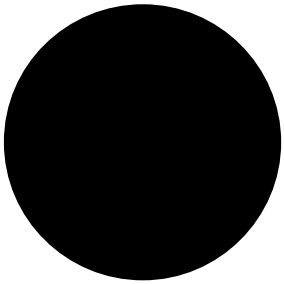
`FoamGetFreeSpace(), FoamCheckIfOutOfSpace()`

FoamGetFreeSpace() returns the number of bytes of free space.

FoamCheckIfOutOfSpace() also returns a **FoamDiskSpaceStatus** but does not initiate any dialog box.

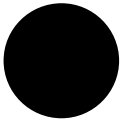
126

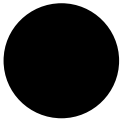
The Socket Library



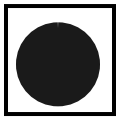
7

7.1 Introduction 129
7.2 Simple Example 134
7.3 Addresses 139
7.3.1 Letting the User Choose an Address 140
7.3.2 Managing Addresses 144
7.3.3 Hardware Ports 144
7.4 Making a Connection 145
7.5 Accepting a Connection 146
7.6 Loading on Connection 149
7.7 Sending and Receiving Data 150
7.7.1 Urgent Data 150
7.7.2 SocketSend() and SocketRecv() 151
7.7.3 Options 154
7.8 Closing the Connection 155
7.9 Checking the Connection 156
7.10 Domain-Specific Information 158
7.10.1 TCP/IP—GEOS Specific 158
7.10.2 TCP/IP—Standard 163
7.10.3 Loopback 164





This chapter describes interactions with the Socket library, used for managing communications between threads — threads on different machines or two GEOS threads on the same machine (via the "loopback" communication domain). It provides an API which is mostly independent of the medium used for inter-machine communication.



It is common to spawn a separate thread to poll a socket for incoming data; See Appendix B, "Threads and Semaphores" on page 913 to learn about managing threads. Sample applications are located in APPL\SDK_C\MultThrd. Some socket operations require knowledge of the communications network used.

7.1 Introduction

The Socket library manages communications both between machines and between threads on the same machine. It provides a level of abstraction above individual communication protocols (*e.g.*, IrLMP, TCP/IP). Programs using the Socket library can send and receive information packets without knowing much about the communications protocol by which those packets are being transmitted. Also, programs using the Socket library need not know about the hardware communication ports used — whether communication is being carried out over an IR serial port, a modem, or a network connection — the API is the same.

The Socket library supports different "delivery types"— different levels of reliability in transmitting data. There is a trade-off: the more reliable delivery types require more set-up work by the programs using them. The following delivery types are available:

Datagram Unreliable transmission of discrete packets. Packets may arrive out of order or not at all. It is up to the data sender to determine if re-transmission is necessary. This is the least demanding delivery type to set up, because it doesn't require any two-way connection.



Sequenced Packet
Sequenced Packet delivery means reliable transmission of discrete packets. The packets arrive in the order sent. The Socket library manages error checking and re-transmissions, as necessary. The program using a sequenced packet socket is responsible for establishing a two-way communications connection.

Stream Stream delivery means the reliable transmission of a stream of bytes, as opposed to other delivery types which require that data be broken up into packets. The Socket library manages a stream of bytes, breaking it up into packets, transmitting the packets, and reconstructing the byte stream from packets on the other side. The Socket library also manages error-checking and re-transmission, as necessary. The program using a stream socket is responsible for establishing a two-way communications connection.

7.1

Addresses

Each side of the connection is represented by an address. An address is made up of several parts:

Domain The network over which the connection is being made. Domains include TCP/IP, IRLAP, and "loopback" (used for connections between two threads on the same machine).

Address Data
This data identifies a machine on a network domain. The format of the address depends on the domain. In the case of TCP/IP, a sample address might be the string "geoworks.com". IrLAP addresses are 32-bit numbers. If the domain is "loopback", then no address information is necessary — the sending and receiving machines are one and the same.

Port This token identifies one side of a connection within one machine. Ports allow multiple data sockets to operate over one connection. Thus, connecting to "geoworks.com", one might send mail messages using port 25 and file-transfer commands via port 21. Port numbers are either 16-bit or 32-bit, depending on the domain.

The amount of overhead associated with sending and receiving data depends upon the devices involved, the domain, and the delivery type. When testing,

experiment with different domains (if available), delivery types, and packet sizes to find a good balance of reliability and rapidity.

For any domain, there may be more than one address format. For instance, the TCP/IP address "geoworks.com" is conveniently human-readable, but cannot be used at the socket level to actually make a connection. Instead, you must resolve "geoworks.com" to a 32-bit address number. All socket operations require "primitive" addresses — addresses resolved to their simplest form. To resolve an address, call **SocketResolve()**. We will discuss the details of working with addresses in more detail below.

7.1

Errors

Many of the Socket library routines return error values. Sometimes these error values are return values of routines; if you call a routine that can return an error value, be sure to check the return value and react appropriately to errors. Some routines don't return an error value directly, instead returning a flag to indicate that something has gone wrong. To find out exactly what went wrong, call **ThreadGetError()**.

Those Socket library routines that return error values do not set the thread error value. Calling **ThreadGetError()** after calling such a routine would not detect any error encountered by that routine.

ThreadGetError() returns a word-sized value. The low byte of this word is a **SocketError** value; the high byte is a **SpecSocketDrError** value. If the returned word is zero, that means that there was no error.

In general, a **SpecSocketDrError** value indicates a low-level error, and a **SocketError** value indicates a high-level error. In many cases, there will be a **SocketError** in the low byte, but the high byte will be clear — there was no problem with the low-level communication medium, but a protocol exception may have occurred.

In general, only the low byte (the **SocketError** value) is useful in determining how to handle the exception. The high byte might affect what error message is displayed to the user. For example, it probably doesn't matter to your program whether the high byte was SSDE_LINE_BUSY or SSDE_NO_DIALTONE. In either case, it just means that communication could not be established. However, users might want to know the nature of the problem so that they can figure out how fix it.



Some of the more common **SocketError** values are listed below. In addition, certain routines may make use of other **SocketError** values, which will be discussed with those routines.

SE_NORMAL
: There is no error. SE_NORMAL is guaranteed to equal zero.

SE_OUT_OF_MEMORY
: The Socket library ran out of memory. Perhaps it was trying to receive or send a data packet that was too large.

7.1

SE_SOCKET_IN_USE
: The socket was busy doing something else. Perhaps you tried to connect it, but it was already connected.

SE_SOCKET_BUSY
: The socket was temporarily busy doing something else. Perhaps you should try again a few times.

SE_MEDIUM_BUSY
: The hardware port which would be used to carry out the requested operation was busy. You might wait for the hardware port to become free; you could call **SocketCheckMediumConnection()** to see if the port is being used for a socket connection.

SE_ALL_PORTS_BUSY
: To carry out the requested operation, the Socket library tried to use a port number; however all port numbers were already busy.

SE_SOCKET_NOT_BOUND
: The socket was not bound (*i.e.*, associated with a port number), but needed to be bound to carry out the requested operation. To bind a socket, call **SocketBind()**.

SE_SOCKET_NOT_LISTENING
: The socket wasn't listening for incoming connections, and needed to be in this state to carry out the requested operation. To make a socket listen, call **SocketListen()**.

SE_SOCKET_LISTENING
: The socket was busy listening for incoming connections when you asked it to carry out another operation. Remember that

you often want separate sockets to handle listening and data transfer.

SE_SOCKET_NOT_CONNECTED
: The socket was not connected, but a connection is required for the operation you requested. To connect the socket, call **SocketConnect()**.

SE_SOCKET_CLOSED
: The socket was closed, making the requested operation impossible.

7.1

SE_CONNECTION_CLOSED
: The other side of your socket's connection has closed that connection; to close up your side of the connection, call **SocketClose()**.

SE_CONNECTION_FAILED
: The connection was lost, probably due to some low-level link failure. There may have been a problem with the physical connection or with the domain-specific software.

SE_CONNECTION_RESET
: The connection was lost, probably due to this side of the connection intentially making a low-level disconnection; *e.g.*, the sort of disconnection that results from **SocketCloseDomainMedium()** or the Socket's owning thread exiting.

SE_TIMED_OUT
: The Socket library was unable to carry out the requested operation before the time-out time was reached. Beware: Depending on the domain, for certain phases of certain operations, there may be low-level time-outs which can occur even if you have specified a longer or an infinite time-out.

SE_CANT_LOAD_DRIVER
: The Socket library was unable to load a driver necessary to carry out the requested operation.

SE_UNKNOWN_DOMAIN
: The Socket library didn't recognize the domain name.



SE_DESTINATION_UNREACHABLE
: The requested destination address was unreachable over the network.

SE_LINK_FAILED
: The low-level link used for this connection failed or could not be established. There may have been a problem with the physical connection or with the domain-specific software.

7.2

SE_SYSTEM_SHUTDOWN
: This error signals that GEOS is shutting down. Your program should probably either end its connections gracefully or attempt to delay the shutdown. Note that it is possible for a shutdown to occur without generating this signal.

SE_INFO_NOT_AVAILABLE
: The requested information was not available.

SE_DOMAIN_REQUIRES_16BIT_PORTS
: You tried to use a 32-bit port number in a domain that does not support them. Use MANUFACTURER_ID_SOCKET_16BIT_PORT as the **ManufacturerID** of your port number.

SE_INTERRUPT
: The operation was stymied by **SocketInterrupt()**.

SE_INTERNAL_ERROR
: The Socket library did something wrong. You should take advantage of whatever developer support to which you have access. They will want a back-trace and other information necessary to duplicate the error.

7.2 Simple Example

The sample application Appl/SDK_C/Talk provides an example of a simple application using the Socket library.

To understand how the program works, you should, of course, look at its source code. The following is a high-level overview of what the Talk example illustrates: a simple client-server connection in which the server awaits connections, the client connects, and then the two exchange data. Note: this application assumes that the host device (the client) has already connected

to the server. Remote connections are usually made through PPP or Point-to-Point Protocol. See **SocketOpenDomainMedium()** discussed in section 7.3.3 on page 144.

The example performs the following steps:

1 **Allocate memory in which to store data.**
When sending or receiving data, you need to provide a pointer to a buffer containing the data to be sent or received. Thus, the buffer should be in a block which can either be locked down as needed or is in fixed memory. 7.2
Programs often spawn a separate thread to receive data from the socket; if this thread needs to lock down a memory block in which to store the data, then make sure that the thread can lock down the memory block. See Appendix B, "Threads and Semaphores" on page 913 to learn about managing threads.
(If you follow the multi-threaded approach, be aware that when the Socket's owning thread exits, it frees the Socket. To allow a Socket to survive the exit of its thread, use **SocketSetIntSocketOption()** to change its owning thread.)
For this simple example, we're using a small buffer to handle the receipt of data, so we declare it as a global variable so it resides in the application's fixed memory:

```
char recvBuf[16];
```

2 **Client Side: Make Sure Connected to Domain**
Depending on the nature of the device and application, you may need to make sure that the device is connected to the communication domain. In the Talk sample application, we make sure that the device is dialed in to an ISP: an **AccessPointControl** object prompts the user to choose an ISP, and a call to **SocketOpenDomainMedium()** opens a PPP connection to that ISP.

3 **Client Side: Get Address For Other Side of Connection.**
The Talk application allows the user to connect with another machine by specifying that machine's address. The Talk application creates a controller gadget by which the user can specify an address. **SocketGetAddressController()** returns the controller class' pointer; **ObjInstantiate()** creates an object of the class.
The following code is called when the application opens. It creates the controller gadget, initializes its data, and places it in a dialog box.

```
ctrlClass =
  SocketGetAddressController(theAddress.SA_domain));
```

```
ctrlOb =
  ObjInstantiate(
    HandleOf(GeodeGetOptrNS(@AddressDialog)),
    ctrlClass);
@call ctrlOb::MSG_SOCKET_ADDRESS_CONTROL_SET_ACTION(
  ConstructOptr(0,TO_PROCESS), MSG_CTP_VALID);
@call
  GeodeGetOptrNS(@AddressDialog)::MSG_GEN_ADD_CHILD(
    ctrlOb, CCO_FIRST);
@call ctrlOb::MSG_GEN_SET_USABLE(VUM_NOW);
```

7.2

When the user enters an address, the following code is called. It gets the selected address from the address controller and packages it up into a form the Socket library can use.

`MSG_SOCKET_ADDRESS_CONTROL_GET_ADDRESSES` returns a chunk with raw address data based on the address entered by the user. **SocketResolve()** reduces this raw address data to a primitive form that can be used for making connections.

```
adArray = @call
ctrlOb::MSG_SOCKET_ADDRESS_CONTROL_GET_ADDRESSES();
address1 = ChunkArrayElementToPtrHandles(
  HandleOf(ctrlOb), adArray, 0,0);
theAddress.SA_addressSize =
  SocketResolve(
    theAddress.SA_domain, (byte *)&address1[1],
    address1->SACA_opaqueSize, addressBuffer,
    MAX_ADDRESS_SIZE);
LMemFreeHandles(HandleOf(ctrlOb), adArray);
```

4 Client Side: Create and Connect Socket.
The Client side now creates a socket for its side of the connection and attempts to connect to the address extracted from the address controller. The socket will be of the stream delivery type.

```
dataSocket = SocketCreate(SDT_STREAM);
```

```
SocketConnect(
  dataSocket, (SocketAddress *) &theAddress,
  SOCKET_NO_TIMEOUT);
```

5 Server Side: Create Socket and Accept Connection(s)

On the server side, the application spawns a separate thread to handle incoming connections.

There are two "sockets" created on this, the server side of the connection. The first socket is bound to a particular port, and "listens" for incoming connections. It is possible to accept more than one connection over the listening socket—each accepted connection is represented by another socket to represent this side of the new particular connection. 7.2

```
listenSocket = SocketCreate(SDT_STREAM);

SocketBind(listenSocket, theAddress.SA_port, 0);

SocketListen(listenSocket, 5);

dataSocket = SocketAccept(
  listenSocket, SOCKET_NO_TIMEOUT);
```

6 Either Side: Send Data

In this example, the data to send is taken from a text object. The program loads the text into a block of memory, locks the block down to get a pointer to it, then transmits the data.

```
textBlock = @call
  GeodeGetOptrNS(@InText)::
  MSG_VIS_TEXT_GET_ALL_BLOCK(NullHandle);

textPtr = MemLock(textBlock);

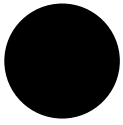
textSize = LocalStringSize(textPtr);

SocketSend(
  dataSocket, textPtr, textSize, 0,
  (SocketAddress *)0);

MemFree(textBlock);
```

7 Either Side: Wait For and Receive Data

The application's listening thread waits for data to arrive and stores it away in a buffer when it does arrive. It processes the incoming data and gets ready for the next piece of data.



```
while ((
  datasize = SocketRecv(
    dataSocket, recvBuf, sizeof recvBuf,
    SOCKET_NO_TIMEOUT, 0, (SocketAddress *)0))
  != 0)
{ @call
  GeodeGetOptrNS(@OutText)::MSG_VIS_TEXT_APPEND_PTR(
    recvBuf, datasize); }
```

7.2

8 Either Side: Close Connection

The application has a Close button. When the user presses this button, the application shuts down its side of the connection. The **SocketCloseSend()** routine partially shuts down the socket so that it can receive data, but cannot send it; also, the routine sends a signal to the other side that the connection is closing.

```
if (listenSocket) {
  SocketClose(listenSocket);
  listenSocket = 0; }
SocketCloseSend(dataSocket);
```

9 Either Side: Detect Closed Connection and Close Socket

When receiving data, if either side seems to have received a zero-length packet, it checks to see if an exception has occurred, specifically to see if the connection has closed. (When **SocketRecv()** detects an error, it sets the thread error value, which may be retrieved via **ThreadGetError()**.) If the connection has closed, then Talk closes the sockets on this side of the connection.

```
if (ThreadGetError() == SE_CONNECTION_CLOSED) {
  if (listenSocket) {
    SocketClose(listenSocket);
    listenSocket = 0; }
  SocketClose(dataSocket); }
```

10 Both Sides: Sockets Freed When Thread Exits

Lest you are worried about these Sockets you have created but not freed, know that they will be freed automatically when the owning thread exits. (To allow a Socket to survive its thread's exit, you must assign the Socket to a new thread.)

7.3 Addresses

In the example above, the application queried the user for a connection address. This is a fairly common case. However, some programs need to use hard-wired addresses. Perhaps you want to allow connections over all available domains. Thus, a few words about addresses may be in order.

Addresses are usually represented via the **SocketAddress** structure:


```
typedef struct {
      SocketPort  SA_port;
      word        SA_domainSize;
      char *      SA_domain;
      word        SA_addressSize;
} SocketAddress;
/* domain-specific address data here */
```

SA_port Port numbers identify a particular line of communication within a machine. Port numbers may be 32-bit or 16-bit, depending on the domain. They are specified via the **SocketPort** structure, made up of a 16-bit **ManufacturerID** and 16-bit arbitrarily-chosen token number. If the domain uses 16-bit socket numbers, then use MANUFACTURER_ID_SOCKET_16BIT_PORT as the **ManufacturerID**. If the domain allows 32-bit socket numbers, then you may use MANUFACTURER_ID_SOCKET_16BIT_PORT or some other **ManufacturerID**, probably your own ID or that of some standard service provider. This numbering scheme helps to avoid overlapping port numbers.

SA_domain, ***SA_domainSize***
The domain identifies the protocol of the network by which the addressed machine may be reached. The domain is specified by a string. In a **SocketAddress** structure, *SA_domain* is a pointer to the null-terminated domain name string and *SA_domainSize* is the size of the buffer containing the domain name string.

Many communications protocols are hard-wired to work with a particular domain. To find out all domains available to the user's device, call **SocketGetDomain().** (See section 7.3.1 on page 140.)



***SA_addressSize*, Address Data**

The format of the address data used to identify a machine within a domain depends on that domain. The *SA_addressSize* field of the **SocketAddress** structure contains the size of the address data. The buffer containing the address data should fall immediately after the **SocketAddress** structure. Read the documentation for a given domain to find out the format for its addresses.

7.3

You'll want to make sure that space for this data is, in fact, allocated at the end of the structure. If you declare a socket address, you will define a struct, probably based on **SocketAddress**, that has the fields of **SocketAddress**, but also has a buffer to hold the resolved address data. More likely than not, you won't need to declare or define such a structure — you're more likely to use **SocketCreateResolvedAddress()** or bypass the creation of this structure altogether.

Before using an address, make sure that it is in its primitive form. For instance, an IP address like "geoworks.com" is not in its primitive form — the IP address must be translated into a four-byte number before it can be used. Use **SocketResolve()** (described below) to transform an address into its primitive form. Note that the address passed to **SocketResolve()** is not in a **SocketAddress** structure.

7.3.1 Letting the User Choose an Address

```
SocketAddressControllerClass, SocketGetDomains(),
SocketGetAddressController(),
SocketCreateResolvedAddress(), SocketResolve(),
SocketInterruptResolve()
```

Given a domain, you can request the Socket library to provide UI gadgetry so that the user can choose an address within that domain. Many programs only support socket connections over one domain. However, if you need a list of all domains supported in the user's environment, call **SocketGetDomains()**.

The **SocketGetDomains()** routine takes the optr of a chunk array and fills in that array with elements. Each element contains a (non-null-terminated)

string which is the name of a supported domain. This chunk array has a standard **ChunkArrayHeader** header and variable-sized elements.

If you want the routine to allocate the chunk array for you, pass an optr that contains a NullChunk (*i.e.*, a null ChunkHandle):

```
MemHandle   myHandle;
Optr        myOptr;
myHandle = OptrToHandle(myHandle);
myOptr = ConstructOptr(myHandle, NULL);
```

7.3

For more information about working with chunk arrays, see "Local Memory," Chapter 16. In short, to manage the chunk array of domain names:

1. Lock down the local memory heap with **MemLock()**.
2. Use **LMemDerefHandles()** to get a pointer to the chunk array.
3. To get the number of domains, call **ChunkArrayGetCount()**.
4. To get a pointer to the nth string in the chunk array, call **ChunkArrayElementToPtr()** or **ChunkArrayElementToPtrHandles()**.
5. When finished reading domain strings, call **MemUnlock()** on the local memory heap.

Given a domain name, you can create some UI gadgetry for choosing addresses within that domain. As shown in the Appl/SDK_C/Talk example, call **SocketGetAddressController()** to get the class of an appropriate controller. This class is guaranteed to support the messages shown in Code Display 7-1.

Code Display 7-1 SocketAddressControllerClass

```
/* Domain-specific subclasses of SocketAddressControllerClass all support
 * the following messages. MSG_SOCKET_ADDRESS_CONTROL_GET_ADDRESSES is probably
 * the most commonly used message. */

/*
 * Build the block of data that holds the addresses selected by the user
 * to pass to the transport driver. If OK, returns ChunkArray of SACAddress
 * structures in same block as controller. Else, returns 0.
```

```
 * As of this writing, all address controllers return at most one address
 * in the list of addresses.

 * The returned address(es) are not in its primitive form; use
 * SocketResolve() to transform the address into something usable.
 */
@message word MSG_SOCKET_ADDRESS_CONTROL_GET_ADDRESSES();

typedef struct {
        word            SACA_opaqueSize;
        /* label byte   SACA_opaque;    Pass this to SocketResolve */
        /* after the opaque address, there is a null-terminated
         *  human-readable form of the address. */
} SACAddress;

/*
 * Inform the subclass of the type of medium selected by the user
 * so it can customize its appearance appropriately, should the controller
 * handle media that have different ways of addressing.
 */
@message void MSG_SOCKET_ADDRESS_CONTROL_SET_MEDIUM (
                                MediumType mediumType);

/*
 * Set the current user-editable address to the passed address.
 */
@message void MSG_SOCKET_ADDRESS_CONTROL_SET_ADDRESSES(optr addr);

/*
 * Inform the controller of the message and destination object it should use to
 * notify the invoker that a valid address exists.
 */
@message void MSG_SOCKET_ADDRESS_CONTROL_SET_ACTION (
                                optr dest,
                                word actionMsg);

/*
 * Let the containing dialog know whether the address we've got is a
 * valid one, so it can decide whether to allow the user to send the
 * message.
 */
@message void MSG_SOCKET_ADDRESS_CONTROL_SET_VALID_STATE(Boolean valid);
```

7.3

You cannot use an address with Socket library routines unless that address is in its primitive form. This primitive form is created by taking a

higher-level form of the address and passing it to the **SocketCreateResolvedAddress()** routine or the **SocketResolve()** routine. Exactly how the Socket library resolves the address depends upon the domain.

Recall that addresses returned by the address controller are not in their primitive form. The primitive form of an address can be volatile: feel free to re-use them if making several connections over a short period of time, but re-resolve the address if it's been a long time since it was last resolved. Note that there can be a large overhead when it comes to resolving addresses. Also note that IRLAP numeric addresses are especially volatile, changing every time the protocol connection starts, *i.e.*, every time the IRLAP driver is loaded, either for the first time or after more than a minute of inactivity.

The **SocketCreateResolvedAddress()** routine takes a pointer to a domain name and raw address data and returns a buffer with a **SocketAddress** structure and the address data. Depending on what you're going to use the address for, you may need to fill in the structure's *SA_port* field.

SocketResolve() is the same as **SocketCreateResolvedAddress()** except that **SocketResolve()** takes a buffer to hold the resolved address data. If the buffer isn't big enough, **SocketResolve()** returns SE_BUFFER_TOO_SMALL. Depending on the domain, calling **SocketResolve()** on an address already in its primitive form may have undesired results.

The resolution process may or may not require network access, depending on the domain and the input format. Depending on the domain, the addressing system may be volatile. Under these conditions, put off resolving the address as long as possible, until just before using the address; there will be less time for the address to go bad. If the network goes down, the Socket library might not be able to resolve addresses. In this case, **SocketCreateResolvedAddress()** or **SocketResolve()** returns SE_TEMPORARY_ERROR.

If you wish to interrupt the resolution process (perhaps you allow the user to cancel the operation), use the **SocketInterruptResolve()** function. This function attempts to halt the resolution process. It is, however, unreliable. It may not succeed in interrupting the resolution process; it may also interrupt other resolves happening at the same time.



7.3.2 Managing Addresses

`SocketGetAddressSize(), SocketGetSocketName(), SocketGetPeerName()`

Most programs blindly use addresses supplied by the user or use only hard-wired addresses. Only a few programs need to access and manipulate addresses. The Socket library provides a few utility routines for this purpose.

To find out the maximum address data size for a given domain, call **SocketGetAddressSize()**. This can be helpful when allocating memory to handle a full address, such as that returned by the **SocketGetSocketName()** and **SocketGetPeerName()** functions.

If a socket is connected, call **SocketGetPeerName()** to get the address of the socket at the other end of the connection. To find out the address used to represent the socket's own side of the connection, call **SocketGetSocketName()**.

7.3.3 Hardware Ports

`SocketGetDomainMedia(), SocketGetAddressMedium(), SocketCheckMediumConnection(), SocketGetMediumAddress(), SocketOpenDomainMedium(), SocketCloseDomainMedium()`

Most socket connections involve the use of some hardware "port" — some physical connection to the outside world. To determine which sort of hardware ports the user's machine uses to connect to a given domain, call **SocketGetDomainMedia()**. This returns a chunk array of **MediumType** structures. Given a choice of domains to make a given connection, consider which medium would be used to make the connection.

To find out which port the Socket library would use to connect to a given address, call **SocketGetAddressMedium()**. Depending on the medium, you might try to connect to the other site via some other domain or decide not to make the connection at all.

To check a given hardware port is being used for an active connection, call **SocketCheckMediumConnection()**. If a point-to-point connection exists, this routine returns the address of the remote site. If there is a connection,

but not a specific address, then the routine returns a null address. If there is no connection through the hardware port, the routine returns a **SocketError** value.

To find out the address of the local machine on a connection over a particular medium, use the **SocketGetMediumAddress()** routine. This is the address another machine would use to connect to this machine via that medium.

The **SocketOpenDomainMedium()** routine opens a "raw" connection using the specified medium. Exactly what is meant by a "raw" connection depends upon the domain involved. In the case of TCP, this routine will dial the modem and create a PPP connection to the PPP server but will not send any TCP packets.

It is possible to "hang up" a hardware port. Call **SocketCloseDomainMedium()** to force a domain to close a medium. You may request that the medium only be closed if it is not being used (*i.e.*, there are no active connections using the medium). If any socket connections are using the port, they will receive the error SE_CONNECTION_RESET. Later, we will discuss orderly ways to close a connection, being careful to send proper notification to the other side of the connection.

7.4 Making a Connection

`SocketCreate(), SocketConnect(), SocketInterrupt()`

To make a connection, a program creates a socket and specifies an address to which to connect that socket.

To create a new socket, call **SocketCreate()**. You must specify the socket's delivery type: datagram, sequential packet, or stream.

Before sending or receiving data over a sequential packet or stream socket, you must "connect" it to the place it will be sending the data to or receiving it from.

Depending on the medium and device, you may need to make sure that the device is connected to a network. For example, for a Nokia 9000 Communicator to make a TCP/IP connection, it must first have made a PPP



connection. Use the **SocketOpenDomainMedium()** routine to make the "raw" network connection.

To make a connection, call **SocketConnect()**. Pass the socket created via **SocketCreate()**, the address to connect to, and a time-out value. If it returns SE_NORMAL, then the connection was established successfully. If it returns SE_CONNECTION_REFUSED, the remote site wasn't prepared to accept the connection. If it returns SE_CONNECTION_ERROR, the connection didn't go through for some miscellaneous reason. If it returns SE_NON_UNIQUE_CONNECTION, then the attempted connection would have been identical to another connection: same source address, source port, destination address, destination port, and delivery type. There would be no way to tell these connections apart. If you try to open the same connection from both sides simultaneously, they may both fail with SE_NON_UNIQUE_CONNECTION.

To cancel the connection operation, call **SocketInterrupt()**.

Datagram sockets cannot form a true connection — they are unreliable. Sockets must be able to send and receive datagram packets without relying on a permanent connection. However, using **SocketConnect()** with a datagram socket specifies a default destination address to use when sending packets.

7.5 Accepting a Connection

`SocketBind(), SocketBindInDomain(), SocketListen(), SocketAccept()`

There's a bit of work involved in setting up a program to receive connections from other sockets. The program must create a socket and associate it with a port. If there is to be a sequenced-packet or stream delivery connection, then the program needs to do some more work to listen for and accept these connections.

If a program creates a socket that accepts connections, the program should probably spawn a separate thread for each accepted connection. If this is not practical, then see **SocketCheckReady()** for a way to monitor several sockets with a single thread.

Call **SocketCreate()** to create a socket for this side of the connection. If the socket is to receive datagram packets, then it must be of type datagram. If the other socket in the connection will be a sequenced packet or stream delivery type, then the socket accepting the connection must be either sequenced packet or stream delivery type; however, the two sockets need not be of the same type.

(If you follow the multi-threaded approach, be aware that when the Socket's owning thread exits, it frees the Socket. To allow a Socket to survive the exit of its thread, use **SocketSetIntSocketOption()** to change its owning thread.) 7.5

The other side of the connection attempts to hook up to a port number on this side of the connection. If a socket is to intercept connections to a certain port, it must somehow associate itself with that port. Use the **SocketBind()** or **SocketBindInDomain()** routines to associate a socket with a port number. **SocketBind()** associates the socket with a port number. **SocketBindInDomain()** associates the socket with a port number but only within a specific domain.

For any given domain, normally only one socket on the machine may be bound to any port number. Thus, if you call **SocketBindInDomain()** to bind socket A to port 3 in the TCP/IP domain, then:

- You could use **SocketBindInDomain()** to bind socket B to port 3 in the IRLAP domain, but
- You couldn't use **SocketBindInDomain()** to bind socket C to port 3 in the TCP/IP domain, and
- You couldn't use **SocketBind()** to bind socket D to port 3.

There are some ways around this restriction:

- To force **SocketBind()** or **SocketBindInDomain()** to re-use some port that another socket has bound, then pass the SBF_REUSE_PORT flag. Otherwise, be prepared to receive SE_PORT_IN_USE or SE_BIND_CONFLICT. Even the SBF_REUSE_PORT flag won't allow you to use both **SocketBind()** and **SocketBindInDomain()** on a single port. E.g., in the example above, you could use **SocketBindInDomain()** with SBF_REUSE_PORT to bind socket C to port 3 in the TCP/IP domain, but you could not use **SocketBind()** with SBF_REUSE_PORT to bind socket

D to port 3. You may not use SBF_REUSE_PORT with datagram-based sockets.

- If there are only non-datagram-based sockets bound to a port, you may bind a datagram-based socket to that port without setting SBF_REUSE_PORT. The reverse is also true. Recall that only a datagram socket can connect with another datagram socket; non-datagram sockets connect only with other non-datagram sockets.

7.5

- A given socket may only be bound to one port. Trying to bind it to another port results in SE_SOCKET_ALREADY_BOUND. Create another socket to bind to the other port.

Always check for bind conflicts. Bind conflicts may occur even though a program is well-behaved: other programs may have bound ports to a socket. For reasons described in “Loading on Connection” on page 149, sometimes the Socket library itself may monitor a port.

When setting up a sequenced packet or stream delivery connection, a bit more set-up work is necessary. These delivery types require a “connection.”

To listen for incoming sequenced packet or stream connections, call **SocketListen()**. The socket must be bound to a port before it can listen. To allow more than one connection through the port, specify the maximum number of connections as an argument to **SocketListen()**. The **SocketListen()** routine causes a socket to listen for incoming connections. If another socket is listening at the port, **SocketListen()** returns SE_PORT_ALREADY_LISTENING.

To make a socket wait for and accept a connection, call **SocketAccept()**. The socket must be listening for connections (processed by **SocketListen()**). Because the thread blocks until a connection is made or **SocketAccept()** times out, programs normally don’t call this routine in their main execution thread. As in the Appl/SDK_C/Talk sample application, a separate thread handles accepting a connection and receiving data from the connection.

If a connection is made, **SocketAccept()** returns a **Socket** used to represent the connection. Do not confuse this connected socket with the listening socket. You might be able to accept more connections from the listening socket.

If **SocketAccept()** returns SE_LISTEN_QUEUE_EMPTY, then there were no connections waiting to be accepted.

7.6 Loading on Connection

`SocketAddLoadOnMsg(), SocketAddLoadOnMsgInDomain(), SocketRemoveLoadOnMsg(), SocketRemoveLoadOnMsgInDomain()`

It's all very well that you can tell a running program to accept packets and connections over sockets. However, it seems unreasonable to ask the user to run these programs by hand any time there might be a pending connection.

You can tell the Socket library to load a particular geode when it receives data on a particular port, or on a particular port via a particular domain. For an example, see the Appl/SDK_C/Talk sample application: if you run Talk and press the Autoload button, the Socket library knows to start up the Talk program because it senses that it is receiving an incoming Talk connection.

When the program starts up, it should probably create a socket, bind it to the appropriate port, and make that socket listen. If the listening socket is then destroyed, then any unaccepted connections are canceled.

The Socket library writes out an entry to the GEOS.INI file in the [socket] category and with key "LoadOnMsg". The Socket library consults these GEOS.INI file entries when it starts up and will act accordingly when it receives packets.

To make the Socket library load a program when receiving packets on a particular port, call **SocketAddLoadOnMsg()**. Specify the program by means of its path. To load a program when receiving packets on a particular port on a particular domain, call **SocketAddLoadOnMsgInDomain()**.

To remove the entries from the GEOS.INI file and tell the Socket library not to try to load the program when receiving packets, call either **SocketRemoveLoadOnMsg()** or **SocketRemoveLoadOnMsgInDomain()**.

To listen for incoming data in this manner, the Socket library effectively binds a socket to monitor the port. Thus

- **SocketAddLoadOnMsg()** and **SocketAddLoadOnMsgInDomain()** can run into bind conflicts just as **SocketBind()** and **SocketBindInDomain()** do.



- **SocketAddLoadOnMsg()** interferes with **SocketAddLoadOnMsgInDomain()** and **SocketBindInDomain()** on the same port.
- **SocketAddLoadOnMsgInDomain()** interferes with **SocketAddLoadOnMsg()** and **SocketBind()** on the same port.

7.7

7.7 Sending and Receiving Data

`SocketSend(), SocketRecv(), SocketSetIntSocketOption(), SocketGetIntSocketOption(), SocketSetSocketOption(), SocketGetSocketOption()`

Once both sides of a socket communication are set up, they're ready to send data back and forth. The **SocketSend()** and **SocketRecv()** routines, described below, allow a socket to transmit and receive data, respectively.

The amount of overhead associated with sending and receiving data depends on the devices involved, the domain, and the delivery type. Be certain that if just one byte of data is sent at a time, time performance will suffer.

Whenever possible, the sender should gather up small pieces of data and send them in one large packet. The optimal size of a packet depends on the device, domain, and delivery type. When designing a communications protocol, it would be worthwhile to figure out which sorts of data may be combined into larger blocks. Note that in some domains, the driver will try to do this gathering of small blocks into large packets for you.

7.7.1 Urgent Data

The Socket library supports the notion of urgent data. This allows a short packet of data to cut in front of other packets. To send a packet marked urgent, pass the SSF_URGENT flag to **SocketSend()**.

In some domains, only one byte of data in a packet can be marked urgent. In such a domain, passing SSF_URGENT with a 32-byte packet to **SocketSend()** results in two packets being sent: a 1-byte urgent packet and a 31-byte non-urgent packet. The data of the 1-byte urgent packet is the last

byte of the original 32-byte packet. Check your domain documentation for size limits on urgent packets.

Urgent data may be delivered out of order. It may arrive out of sync with non-urgent packets; it may arrive out of order with other urgent packets. There is no queue for urgent data. Each urgent data packet received causes the previous urgent data to be discarded.

The **SocketRecv()** routine, in charge of receiving incoming packets, may be instructed to receive only urgent data or only non-urgent data. If there is pending urgent data and **SocketRecv()** is told to poll for normal data, it will return an error value, SE_URGENT, signalling that the program should call **SocketRecv()** again, this time to receive urgent data. (If, instead, you call **SocketRecv()** telling it to receive normal data again, it will return any pending normal data; it will not generate the SE_URGENT exception for the pending piece of urgent data — unless *another* piece of urgent data has arrived in the meantime.)

7.7

Datagram connections do not support urgent data; **SocketRecv()** generates an SE_URGENT error on a datagram connection, it's really generating a SE_EXCEPTION value. If you receive such an error, it is a sign that the network is having troubles and packets *may* not be getting through.

You may force incoming urgent data to be treated in the same manner as normal data by setting the socket's "in-line" option. See page 154 for information about socket options.

7.7.2 SocketSend() and SocketRecv()

The **SocketSend()** routine sends data over a socket. It takes several arguments:

Socket
: The socket through which to send data

Buffer
: A buffer containing the data to send

Buffer Size
: The size of the data buffer.



Optional Address

Address to which to send data. If you pass an address here, make sure that you set the Address flag in the Flags field. If you don't set the Address flag, then this Optional Address argument may be a null pointer.

For sequenced packet or stream delivery type sockets, optional address is ignored.

7.7

For datagram delivery type sockets, specify the destination address of the packet in this parameter unless the socket has a default destination address as described below.

Datagram sockets can use the **SocketConnect()** routine to specify a default destination address. If they have no default address, you must specify the destination address for the packet. If there is a default destination address and you pass this optional address, then the optional address overrides the default for this send.

The following flags are available:

SSF_ADDRESS

To pass an optional address.

SSF_URGENT

To mark the data packet as urgent.

SSF_OPEN_LINK

To open the link if it is closed or to close the current link if it is idle and open it to a different location. If you do not set this flag, the datagram will only be sent if the link is opened to the desired remote link address.

Note that in some domains, the driver may delay before sending small packets of data because it is waiting for more packets to combine into one large packet. In many cases, this leads to more efficient communication. You may turn off this behavior by setting the "no delay" option. See page 154 for information about setting socket options.

The **SocketRecv()** routine receives data from the socket. After filling the passed buffer with data, the socket discards its copy of the received data to make room for the next piece of incoming data.

Sequenced packet or datagram delivery sockets can only receive whole packets. If you grab data from the socket but don't grab all the data in the packet, the remaining data is lost. Thus, it's a good idea to establish a maximum packet size for sockets using these delivery types. If you're not sure how much room you'll need to receive a packet, call **SocketRecv()** with the SRF_PEEK flag. This allows you to "peek" at the incoming data without causing the socket to discard its copy.

The **SocketRecv()** routine takes the following arguments: 7.7

Socket
: The socket from which to grab the data.

Buffer
: Buffer to fill with data.

Buffer Size
: Size of the data buffer. A packet-based socket will not retain data that does not fit in the buffer. A stream socket, however, will.

Time-out
: The number of 1/60 second ticks to wait for incoming data. Pass SOCKET_NO_TIMEOUT to wait forever.

Optional Address
: If you pass the Address flag of the Flags argument, then pass an empty **SocketAddress** buffer. (The buffer's *SA_domain*, *SA_domainSize*, and *SA_addressSize* fields must be initialized.) Make sure the buffer has room for the address data after the **SocketAddress** structure. Note that if either the domain name buffer or the address buffer isn't long enough to hold its string, the resulting truncated string isn't null-terminated.

  If you do not set the Address flag, this Optional Address argument may be a null pointer.

The following flags are available:

SRF_ADDRESS
: Return the source address in the Optional Address buffer.



SRF_URGENT
Return the first packet which has been marked urgent. (See "Urgent Data" on page 150.)

SRF_PEEK
Force the socket to not discard the data received.

The **SocketRecv()** routine returns the size of the received buffer. If this size is zero, there may be an error in the connection. Use **ThreadGetError()** to check for an error. If **ThreadGetError()** returns SE_NORMAL, then there was no error and you received a zero-length packet. For a datagram connection, an SE_EXCEPTION error indicates that the network is having problems and some datagrams *may* not be getting through.

7.7

7.7.3 Options

To customize a socket's sending and receiving behavior, use **SocketSetIntSocketOption()** (or **SocketSetSocketOption()** if you are using Assembly language). The following options are available:

SO_RECV_BUF: **Receive Buffer** Size
To change the size of the buffer used to hold incoming data.

SO_SEND_BUF: **Send Buffer** Size
To change the size of the buffer used to hold outgoing data.

SO_INLINE: **In-line** urgent data
To force urgent data to be treated the same as normal data. This allows you to poll for both urgent and normal data with a single **SocketRecv()** command. Of course, setting this option means that you will not receive urgent data as soon as possible.

SO_NODELAY: **No Delay** for small packets
To force a driver to send small packets without waiting for other packets to bundle together. Drivers often delay sending small packets, waiting for other packets to bundle together in one larger packet. This is oftentimes more efficient.

SO_OWNER: **Owning Thread** for socket
To specify a new owning thread for a socket. When the owning thread detaches from the Socket library (this happens automatically when the thread exits), the socket will be freed;

any attempts to use the freed socket will result in errors. To make a socket survive its thread, you must assign it to a new thread. This option does not actually affect how the socket sends or receives data.

You may find out the current options for a socket by calling **SocketGetIntSocketOption()** (**SocketGetSocketOption()** if you're using Assembly language).

7.8 Closing the Connection

`SocketInterrupt(), SocketClose(), SocketCloseSend(), SocketReset(), SocketCloseDomainMedium()`

To halt either an accept or receive operation in progress, call **SocketInterrupt()**. If no such operation is in progress but the socket is listening or connected, the next such operation will be interrupted. Look out for the error SE_SOCKET_NOT_INTERRUPTIBLE; this signals that a non-interruptible operation is taking place on the socket.

When it comes time to close the connection and/or free a socket, call **SocketClose()** or **SocketCloseSend()**.

SocketCloseSend() works only with connected sockets. It partially shuts down the socket so that it can no longer send data; it can, however, still receive data. (Passing a partially closed socket to **SocketSend()** results in an SE_SOCKET_CLOSED error.) **SocketCloseSend()** sends an SE_CONNECTION_CLOSED exception packet to the other side of the connection to let it know that it's starting to shut itself down. Upon receipt of this exception packet, the other side of the connection will probably want to begin closing itself down as well.

Not all domains support closing the "send" part of a socket. In this case, the routine will return SE_CLOSE_SEND_NOT_SUPPORTED and you must use **SocketClose()** instead to close the connection and free the socket.

SocketClose() closes the socket (shutting down both incoming and outgoing data) and frees it. This routine may be used on a socket in any state. You must use **SocketClose()** to close (and free) a datagram socket.



To close a connection with extreme prejudice, call **SocketReset()**. Use this routine with caution as it bypasses all normal protocol-level handshaking and unilaterally kills the connection. When using this routine, you should still call **SocketClose()** to free the socket data structures and flush any unread data.

The **SocketCloseDomainMedium()** routine can close all connections over a given medium without sending any notifications. It effectively hangs up the connection. You pass a flag to specify whether the routine should only hang up if there are no active connections through the medium, or hang regardless. For more information about socket media and this routine, see “Hardware Ports” on page 144.

7.9

7.9 Checking the Connection

`SocketCheckListen(), SocketCheckReady()`

The Socket library provides utility routines that query the status of a connection. This can be helpful when figuring out why a connection may not be working or confirming that a connection is working.

Use **SocketCheckListen()** on a port to find out the domain and medium for the first connection request on the port; that is, the first connection request which has not yet been accepted. Think of this as a limited form of “caller ID” — a chance to find out about the incoming connection before accepting it. If **SocketCheckListen()** returns SE_PORT_NOT_LISTENING, it means there is no socket that is bound to the port and listening.

To “peek” at the next packet of incoming data, call **SocketRecv()** and pass the SRF_PEEK flag. This allows you to get the size of the next packet of incoming data and to look at its contents without destroying it.

To check one or more sockets to see if they have received data, received connection requests, or are ready to write, call **SocketCheckReady()**. It can check connections for incoming data or just incoming urgent data.

For programs which need to poll many sockets, **SocketCheckReady()** provides a tidy means to do this without spawning a thread for each socket.

The **SocketCheckReady()** routine takes an array of **SocketCheckRequest** structures. Each one of these structures contains a socket and a condition. **SocketCheckReady()** looks at each **SocketCheckRequest** structure in the array and returns the index of the first structure whose socket meets the condition.

Thus, to determine if a given socket is properly set up so that you can send data through it, pass **SocketCheckReady()** a one-element array (represented in pseudo-code):

7.9

```
{ mySocket, SC_WRITE, 0 }
```

To check several sockets to see whether any of them had received any connection requests, pass to **SocketCheckReady()** an array of the form (represented in pseudo-code):

```
{ socket1, SC_ACCEPT, 0 },
{ socket2, SC_ACCEPT, 0 },
{ socket3, SC_ACCEPT, 0 }
```

For any socket, you may check for one or more of the following conditions, as represented by **SocketCondition** values:

SC_ACCEPT
: If a socket is listening for a connection, this condition indicates that another socket is trying to connect to the listening socket.

SC_READ
: If a socket is connected, this condition indicates that a packet of data has come in and is ready to be read.

SC_EXCEPTION
: If a socket is connected, this condition indicates that the socket has discovered something wrong with its connection.

SC_URGENT
: If a socket is connected, this condition indicates that it has received a packet of data that was marked urgent.

SC_WRITE
: This condition indicates that data may be sent through the socket.



If you query a socket about a condition that does not apply to its current state (*e.g.*, ask a non-listening socket if it is ready to accept connections), then **SocketCheckReady()** returns SE_IMPROPER_CONDITION.

7.10 Domain-Specific Information

7.10

Some portions of the Socket library API are domain specific; depending on which domain you're using, the domain name, address formats, port numbering schemes, etc. may differ.The following sections describe information specific to particular domains. Not all domains are available on any given device.

7.10.1 TCP/IP—GEOS Specific

The TCP/IP domain is a popular standard used for internet communications. The GEOS-specific version supports 32-bit port numbers. GEOS TCP/IP data may be transmitted via a regular TCP/IP network, though both the sending and receiving machines must support GEOS TCP/IP.

Domain Name:
"TCPIP_GEOS"

Port Numbers:
32-bit values.

Opaque Address Format:
This address will be either a **TcpAccPntExtendedAddress**,a **TcpOnlyExtendedAddress**, or a **TcpNonAccPntExtendedAddress**.

TcpAccPntExtendedAddress: Use this structure when referring to an address identified by an access point ID number.

TcpOnlyExtendedAddress: Use this structure to identify a TCP address by its address instead of by its ID.

TcpNonAccPntExtendedAddress: Use this structure to identify a TCP address that is not a known access point. You will need to

specify a link address (a phone number) as well as an IP address.

Notes:

- On some devices (including the Nokia 9000 Communicator), you cannot make a TCP/IP connection or send packets until the device has a "raw" connection to the network—a PPP connection. Calling **SocketOpenDomainMedium()** with the TCP medium creates a PPP connection. The modem will dial, and the machine will connect to the PPP server, but no TCP level packets will be sent. If the PPP connection is already made, this routine will return SE_NORMAL, just as if the connection had been just now made. If the modem is already in use, an SE_MEDIUM_BUSY error will be returned. Use the **SocketCloseDomaninMedium()** to hang up the phone when done. 7.10
- The Talk sample application shows how a device can make a raw PPP connection with the Internet Service Provider (ISP), getting the necessary information about the ISP from the built-in Access Point library. (Notice how the object AccpntControl is declared; notice the sample's TalkAddressClass handler for MSG_GEN_GUP_INTERACTION_COMMAND, which queries the AccessPointControl for the selected ISP and extracts the useful information about that ISP; finally, the MSG_CTP_CONNECT handler dials the phone to make the PPP connection.)

Code Display 7-2 Making the Raw TCP/IP Connection

Here we see three snippets of code from the Talk sample application The AccessPointControl allows the user to choose an ISP:

```
@chunk char accpntMkr[] = "Access List";

@object AccessPointControlClass AccpntControl = {
        GI_states = GS_USABLE|GS_ENABLED;
        HINT_ACCESS_POINT_CONTROL_MINIMIZE_SIZE;
        ATTR_ACCESS_POINT_CONTROL_LIST_MONIKER = @accpntMkr; }
```

In @method TalkAddressClass, MSG_GEN_GUP_INTERACTION_COMMAND, we get information we'll need about the ISP

```
point = @call \
GeodeGetOptrNS(@AccpntControl)::MSG_ACCESS_POINT_CONTROL_GET_SELECTION();
```

```
/* store link info into address buffer */
rawAddress.UTA_link.TAPEA_linkSize = 3;
rawAddress.UTA_link.TAPEA_linkType = LT_ID;
rawAddress.UTA_link.TAPEA_accPntID = point;

/* the text of the address follows the link info */
alen = @call GeodeGetOptrNS(@IPText)::MSG_VIS_TEXT_GET_ALL_PTR(
                                                (char *)&(rawAddress.UTA_ip[0]));

if (alen > MAX_IP_ADDR_STRING_LENGTH) FatalError(0);  /* too much text */
```

7.10

```
/* resolve the raw address into a SocketAddress */
theAddress.SA_addressSize = SocketResolve(theAddress.SA_domain,
                                          (byte *)(&rawAddress),
                                          sizeof(TcpAccPntExtendedAddress)+alen,
                                          (byte *)(&addressBuffer),
                                          MAX_ADDRESS_SIZE);
```

In MSG_CTP_CONNECT's handler, we make the PPP connection:

```
rval = SocketOpenDomainMedium((SocketAddress *) &theAddress, SOCKET_NO_TIMEOUT);
```

- TCP/IP only supports one byte of urgent data in a packet. If you send a multi-byte packet marked urgent, it will be divided into two packets (on the receiving side). For example, a 32-byte packet marked urgent would be broken up into a 31-byte normal packet and a one-byte urgent packet.

  If you set the SSF_URGENT flag to **SocketSend()** when sending a multi-byte packet, the last byte of the packet will be marked urgent.

  Recall that you can set the in-line option to specify that urgent data should be treated as normal data. In this case, urgent data will be treated as a normal 1-byte packet.

- If you're not familiar with TCP/IP, *TCP/IP illustrated* by W. Richard Stevens provides a good introduction.

  The TCP/IP standard is defined in a number of RFCs. The RFCs listed below may be of interest. If you research these documents, note that the first is an index to the others.

  1720 J. Postel, I. Architecture Board (IAB), "INTERNET OFFICIAL PROTOCOL STANDARDS", 11/23/1994. (Obsoletes RFC1610) (Obsoleted by RFC1780) (STD 1)

1700 J. Reynolds, J. Postel, “ASSIGNED NUMBERS”, 10/20/1994. (Obsoletes RFC1340)

1122 R. Braden, “Requirements for Internet hosts - communication layers”, 10/01/1989.

1123 R. Braden, “Requirements for Internet hosts - application and support”, 10/01/1989.

0791 J. Postel, “Internet Protocol”, 09/01/1981. (Obsoletes RFC0760)

0950 J. Mogul, J. Postel, “Internet standard subnetting procedure”, 08/01/1985.

0919 J. Mogul, “Broadcasting Internet datagrams”, 10/01/1984.

0922 J. Mogul, “Broadcasting Internet datagrams in the presence of subnets”, 10/01/1984.

0792 J. Postel, “Internet Control Message Protocol”, 09/01/1981. (Obsoletes RFC0777)

0768 J. Postel, “User Datagram Protocol”, 08/28/1980.

0793 J. Postel, “Transmission Control Protocol”, 09/01/1981. (Updates RFC0761)

1034 P. Mockapetris, “Domain names - concepts and facilities”, 11/01/1987. (Obsoletes RFC0973) (Updated by RFC1101)

1035 P. Mockapetris, “Domain names - implementation and specification”, 11/01/1987. (Obsoletes RFC0973) (Updated by RFC1348)

0974 C. Partridge, “Mail routing and the domain system”, 01/01/1986.

1531 R. Droms, “Dynamic Host Configuration Protocol”, 10/07/1993. (Obsoleted by RFC1541)

1533 S. Alexander, R. Droms, “DHCP Options and BOOTP Vendor Extensions”, 10/08/1993. (Obsoletes RFC1497)

1534 R. Droms, “Interoperation Between DHCP and BOOTP”,

7.10



10/08/1993.

1144 V. Jacobson, “Compressing TCP/IP headers for low-speed serial links”, 02/01/1990.

1547 D. Perkins, “Requirements for an Internet Standard Point-to-Point Protocol”, 12/09/1993.

7.10

1662 W. Simpson, “PPP in HDLC-like Framing”, 07/21/1994. (Obsoletes RFC1549)

1334 B. Lloyd, W. Simpson, “PPP Authentication Protocols”, 10/20/1992.

1661 W. Simpson, “The Point-to-Point Protocol (PPP)”, 07/21/1994. (Obsoletes RFC1548)

1570 W. Simpson, “PPP LCP Extensions”, 01/11/1994. (Updates RFC1548)

1663 D. Rand, “PPP Reliable Transmission”, 07/21/1994.

1332 PS G. McGregor, “The PPP Internet Protocol Control Protocol (IPCP)”, 05/26/1992. (Obsoletes RFC1172)

- Some (16-bit) constants (defined in **sockmisc.h**) are available with the port numbers for common TCP services:

```
#define ECHO            7  /* TCP/UDP */
#define DISCARD         9  /* TCP/UDP */
#define FTP_DATA        20 /* TCP */
#define FTP             21 /* TCP */
#define TELNET_SERVER   23 /* TCP */
#define NAME_SERVER     42 /* UDP */
#define WHOIS           43 /* TCP */
#define DOMAIN_SERVER   53 /* TCP/UDP */
#define FINGER          79 /* TCP */
```

- The address data information with a **SocketAddress** structure takes the following form for the TCP domain:

```
typedef struct {
      word    TOEA_linkSize;  /* 0 */
      byte    TOEA_ipAddr[4]; /* IP address */
} TcpOnlyResolvedAddress;
```

A variation on this structure is used for storing the address information for a TCP access point address. Along with the address information, there are three bytes of information about the nature of the link: the first of these bytes should have value LT_ID, so that the driver will know that the next two bytes represent a link ID; the other two bytes should be the access point's ID.

7.10

```
typedef struct {
      word    TAPRA_linkSize; /* 3 */
      byte    TAPRA_linkType; /* LinkType (LT_ID) */
      word    TAPRA_accPntID;
      byte     TAPRA_ipAddr[4]; /* IP address */
} TcpAccPntResolvedAddress;
```

7.10.2 TCP/IP—Standard

This is the standard TCP/IP domain, popular protocol for internet communications.

Domain Name:
"TCPIP"

Port Numbers:
16-bit values.

Opaque Address Format:
This address will be either a **TcpAccPntExtendedAddress**,a **TcpOnlyExtendedAddress**, or a **TcpNonAccPntExtendedAddress**.

TcpAccPntExtendedAddress: Use this structure when referring to an address identified by an access point ID number.

TcpOnlyExtendedAddress: Use this structure to identify a TCP address by its address instead of by its ID.



TcpNonAccPntExtendedAddress: Use this structure to identify a TCP address that is not a known access point. You will need to specify a link address (a phone number) as well as an IP address.

Notes:

- The notes for GEOS TCP/IP (starting on page 160) apply to the TCP/IP standard as well.
- You cannot use the sequential packets delivery type with standard TCP/IP.

7.10

7.10.3 Loopback

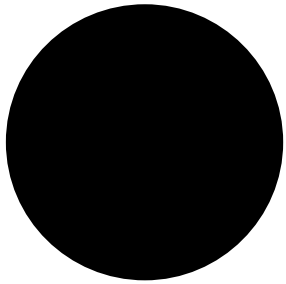
The loopback domain can only be used for communications between sockets on the same machine.

Domain Name:
"loopback"

Port Numbers:
32-bit values.

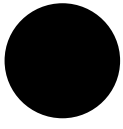
Opaque Address Format:
There is no address data: The loopback domain is used for intra-machine communication.

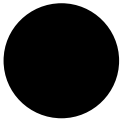
Access Point Library



8

8.1 **Access Points** 167
8.2 **Letting The User Choose** 168
8.3 **Searching For an Access Point** 169
8.4 **Locking and Unlocking** 171
8.5 **Adding an Access Point** 171
8.6 **Deleting an Access Point** 171





The Access Point library acts as a sort of special address book—it keeps track of phone numbers and other information about Internet Service Providers. It can also store information about Terminal and Telnet access points. The library provides UI gadgetry by which the user may choose and edit access point entries; the library also provides routines by which programs may do the same.



To use the Access Point library, you need only basic GEOS programming knowledge. If you plan to use certain advanced functionality, you will need to learn about chunk arrays, which are documented in "Local Memory," Chapter 16.

8.1 Access Points

Access points, loosely defined, are collections of data useful for making communications connections. The Access Point library keeps track of access points by storing their information in the .INI file.

The library defines a few kinds of access points, identified by the **AccessPointType** enumerated type:

APT_INTERNET
: This access point holds data useful for connecting to an ISP. Applications making a PPP connection can use this information to make the connection.

APT_TERMINAL
: This access point holds data useful for connecting to a dialup terminal.

APT_TELNET
: This access point holds data useful for making a telnet connection.

For each access point, the AccessPoint library may keep track of any number of properties. While you may store all sorts of information about any access point, there are a number of standard properties, enumerated as



AccessPointStandardProperty values, including name, phone number, user ID, address, and password.

Access points are referenced by word-length ID numbers.

8.2 Letting The User Choose

8.2

The Talk sample application shows how an application can use an AccpntControl object to provide UI for picking an access point. This example is discussed in "Socket," Chapter 7. The application also shows how to use the selected access point (in this case, an ISP) to establish a PPP connection; this part of the application is discussed further in the Socket chapter.

In this example, the controller is declared:

```
@chunk char accpntMkr[] = “Access List”;

@object AccessPointControlClass AccpntControl = {
    GI_states = GS_USABLE | GS_ENABLED;
    ATTR_ACCESS_POINT_CONTROL_LIST_MONIKER = @accpntMkr;
    HINT_ACCESS_POINT_CONTROL_MINIMIZE_SIZE;
}
```

To present the user with a different type of access points (telnet connections, for example) set the type in the *APCI_type* instance data field:

```
@object AccessPointControlClass AccpntControl = {
    …
    APCI_type = APT_TELNET;
}
```

You may wish to set up your program's UI such that there is a gadget which will not be enabled (will be grayed out) if there are no access points of the proper type defined. To do this, turn off the object's GS_ENABLED flag in its declaration, and use the object's optr as the value of the *APCI_enableDisable* instance field:

```
@object … MyObject = {
    …
    GI_states = @default & ~GS_ENABLED;
}
```

```
@object AccessPointControlClass AccpntControl = {
    …
    APCI_enableDisable = @MyObject;
}
```

To find out the ID number of the controller's currently selected access point, use MSG_ACCESS_POINT_CONTROL_GET_SELECTION.

To find out when the user has selected an access point (changed their selection), you have to be a bit trickier.

- Subclass **AccessPointControlClass**.
- Write a method for your subclass, intercepting MSG_ACCESS_POINT_CONTROL_UPDATE_SELECTION. Be sure to use @callsuper to invoke the default handler for this message.
- Use an object of your subclass instead of a regular AccessPointControl object.

While the application is using the selected access point, it can "lock" the access point to prevent the access point's parameters from being modified while it's in use. (See section 8.4 on page 171.)

8.3 Searching For an Access Point

Perhaps instead of allowing the user to choose an access point, the program should use a known access point. The following code example shows how a program might search for an access point, given its type and name.

Code Display 8-1 Searching for an Access Point

```
@start  TemplateStringsResource, data;
    @chunk  TCHAR TargetName[] = "Stuckeys Sales Server";
@end    TemplateStringsResource;

@method TemplateProcessClass, MSG_TP_SEND {
    MemHandle     aBlock;
    ChunkHandle   theList;  /* Chunk Array of entry IDs of access points */
    word          theID = ACCESS_POINT_INVALID_ID ;
```




```
    aBlock = MemAllocLMem( LMEM_TYPE_GENERAL, 0 );
    MemLock( aBlock );
    theList = AccessPointGetEntries( aBlock , 0, APT_INTERNET );

    ChunkArrayEnumHandles( aBlock, theList, &theID, FunkyCallback);

    MemUnlock( aBlock );
    MemFree(aBlock);

    if (theID == ACCESS_POINT_INVALID_ID ) {
      /* didn't find the access point--deal with it */
    } else {
      /* found the access point--use it */
    }
}

Boolean _pascal FunkyCallback( void *el, void *retVal ) {
    char nomen[128];
    int  nomenSize;
    int  cmpResult;
    char *accessPointName;

    AccessPointGetStringPropertyBuffer( *((word *)el), APSP_NAME,
                                        nomen, &nomenSize );

    MemLock( OptrToHandle(@TargetName) );
    cmpResult = LocalCmpStrings( LMemDeref( @TargetName ) , nomen, 0);
    MemUnlock( OptrToHandle(@TargetName) );

    if (cmpResult) return(FALSE);        /* if they don't match, return FALSE so
                                            we can keep searching. */

    *(word *)retVal = *(word *)el;       /* We found a match. grab its ID */
    return(TRUE);                        /* …and we can stop searching now. */
}
```

8.3

The **AccessPointGetEntries()** routine returns a chunk array containing a list of access point IDs.

We then use the **ChunkArrayEnumHandles()** routine to call a callback routine to examine each ID in turn. There are many other ways we could have cycled through these entries—see the documentation of chunk arrays in "Local Memory," Chapter 16 to find out which is best for your purpose.

Given an access point's ID number, we find out the name associated with that ID by calling **AccessPointGetStringPropertyBuffer()**. To specify that

we're interested in the name, we pass APSP_NAME; we could also have passed some other **AccessPointStandardProperty** value, or even a pointer to a string to search for the value of some custom property.

While the application is using the selected access point, it can "lock" the access point to prevent the access point's parameters from being modified while it's in use. (See section 8.4 on page 171.)

8.4 Locking and Unlocking

If something were to alter an access point's parameters while your application was using that access point, bad things might happen. To prevent anything from changing the access point's parameters, use the **AccessPointLock()** routine. When done using the access point, use the **AccessPointUnlock()** routine to unlock it.

8.5 Adding an Access Point

To add a new access point:

- Use the **AccessPointCreateEntry()** routine to create the access point. To create an internet access point, use the syntax:

```
newID = AccessPointCreateEntry( 0, APT_INTERNET );
```

- Using the ID returned by **AccessPointCreateEntry()**, set the appropriate properties for the access point.

8.6 Deleting an Access Point

To delete an existing access point:

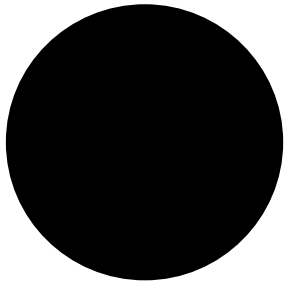
- Call **AccessPointDestroyEntry()**.



8.6

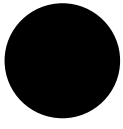
Concepts book

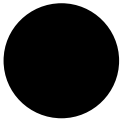
Modem Connections



9

9.1 **Introduction** 175
9.2 **Connecting** 176
9.3 **Sending Data** 178
9.4 **Receiving Data** 180
9.5 **Disconnecting** 183





This chapter discusses how to set up your Nokia 9000 Communicator to emulate a Hayes-compatible modem to establish a direct serial connection.

On the Nokia 9000 you connect through a “virtual” serial port and use a stream to read and write data to this port just as you would with an actual serial port. (For more information on streams, see "Using Streams," Chapter 21 of the Concepts Book.) The modem driver handles data notifications from the port.

9.1

The ModemC Library provides an interface to the modem driver while the VirtualSerial Library provides a stream interface to the “virtual” serial port. In addition, the **modem.goc** module provides a set of basic routines for making and using a modem connection. You can drop this module into any application that uses a modem.

9.1 Introduction

Sending and receiving data over a serial line is similar to sending and receiving data through a socket connection. You configure your hardware, make the connection, open streams for sending and receiving data, and close the connection (*i.e.*, hang up).

This chapter discusses the MTalk sample application; this application illustrates a simple “talk” exchange between two devices over a modem connection. It provides user interface for taking text input (which it sends to the other end of the connection) and for displaying data received from the other end of the connection.



modem.goc

You can drop this module into any application that uses a modem.

MTalk uses routines from the **modem.goc** module which provides basic functionality for making and using a modem connection.



9.2 Connecting

To establish a connection, MTalk does the following:

- allocates a buffer to be used (later) for holding received data
- prompts the user for the phone number to dial
- calls **ModemConnect()** (defined in modem.goc) to make the connection

9.2

Code Display 9-1 Making A Serial Line Connection

```
/*
 * phoneNum - Phone number to dial.
 * retVal - Return value of ModemConnect().
 */
char        phoneNum[MAX_PHONE_NUM_LENGTH + 1];
Boolean     retVal;

/*
 * Allocate a global memory block to be used as a buffer to hold received data.
 * A local buffer variable in the read function would use up
 * precious stack space, and a global buffer variable
 * would increase the size of the application's fixed data segment,
 * which degrades overall memory manager performance.
 *
 * Allocating it to be DISCARDABLE has further advantages.
 * Because we don't care what happens to the contents after they
 * have been processed, we'll allow the memory manager to throw
 * the block out if it decides it needs the heap space. This
 * means a little extra work for us whenever we want to use the
 * block, since we have to check for this condition, and
 * reallocate if necessary.
 */

recvBuffer = MemAlloc( INPUT_BUFFER_SIZE, HF_DISCARDABLE, 0 );

/*
 * If the buffer was not successfully allocated, memory must be getting low
 * so shut down.
 */
if ( recvBuffer == NullHandle )
```

```
{
    @send application::MSG_META_QUIT();
    return;
}

/*
 * Get the phone number to dial.
 */
@call MTalkPhoneNumberText::MSG_VIS_TEXT_GET_ALL_PTR( phoneNum );

/*                                                                    9.2
 * Create the modem connection and register the object and messages
 * it will receive when data is received or when the call has ended.
 * Note: in this example, oself refers to the process object.
 * Remember that ModemConnect() returns FALSE if it successfully makes
 * the connection.
 */
retVal = ModemConnect( phoneNum,
                       oself,
                       MSG_MTALK_PROCESS_READ_DATA,
                       MSG_MTALK_PROCESS_CALL_ENDED );

/*
 * If the connection failed, free the buffer and return. (Note: after freeing
 * any allocated memory, it is good practice to re-set handles and pointers to
 * Null so that you can test for a Null value later.)
 */
if ( retVal )
{
    MemFree( recvBuffer );
    recvBuffer = NullHandle;
    return;
}
```

ModemConnect() takes care of the “housekeeping” details involved in making a connection. Specifically, this routine takes care of the following:

- checks to make sure the phone is available for use
- opens and configures the port (on the Nokia 9000, the “virtual” serial port is COM1)
- initializes the modem



- sets up to receive notification when data arrives on the serial port or when the call ends
- dials the phone number

ModemConnect() takes four parameters:

phoneNumber
Char pointer to the phone number to dial.

9.3

dataReceiver
Optr to the object that will receive data or call end notification.

dataReceiptMessage
Message sent when data arrives on the port.

endCallMessage
Message sent when the call has ended.

ModemConnect() returns FALSE if the attempt to connect is successful. If the attempt is unsuccessful, it displays a dialog box notifying the user of the error. Possible error messages include:

"Connectivity Problem:
Data calls cannot be made while another call is active."

"Connectivity Problem: Line is busy. Try again later."

"Connectivity Problem:
No answer. Check the phone number and try again later."

"Connectivity Problem: General Error."

"Connectivity Problem: Error initializing modem."

9.3 Sending Data

To send data, MTalk does the following:

- retrieves the text the user has typed in and puts it into a memory block
- locks the block down (to get a pointer to the text)
- gets the size of the text
- calls **ModemSend()** (defined in modem.goc) to send the data

- unlocks the memory block

Code Display 9-2 Sending Data Over A Serial Connection

```
/*
 * textBlock - Handle of block containing text to send.
 * textPtr - Pointer to text to send.
 * textSize - Size of text to send.
 */
MemHandle   textBlock;
char *      textPtr;
int         textSize;

/*
 * Get text that user has typed and put it into a newly created memory block.
 * Lock the block down on the heap and obtain a pointer to it.
 */
textBlock = @call MTalkSendText::MSG_VIS_TEXT_GET_ALL_BLOCK( NullHandle );
textPtr = MemLock( textBlock );

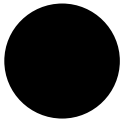
/*
 * Get the size of the text then send it.
 */
textSize = LocalStringSize( textPtr );
if ( ModemSend( textSize, (byte*)textPtr ) != STREAM_NO_ERROR )
{
    /* handle the error */
}

/*
 * Once the text has been sent, free its block.
 */
MemFree( textBlock );
```

9.3

ModemSend() calls the VirtualSerial Library routine, **VirtualSerialWrite()**, to send the passed data. **VirtualSerialWrite()** writes the passed data to stream and, if necessary, blocks until enough space in the stream becomes available. (If you don't want to block, pass STREAM_NO_BLOCK in **VirtualSerialWrite()**.)

ModemSend() takes two arguments:



dataLength
Length of the string (not counting the Null character).

data
Pointer to the data to be sent.

ModemSend() returns a **StreamError** value:

STREAM_NO_ERROR
if it successfully sends the data.

9.4

STREAM_CLOSED
if the stream is not open.

STREAM_SHORT_READ_WRITE
if the required amount of space was not available and STREAM_NO_BLOCK was specified.

9.4 Receiving Data

When the modem driver receives notification that data has arrived on the port, it sends a pre-registered message to a specified object. (You specify both the object and the message when you call **ModemConnect()**.)

In the MTalk example, the driver sends MSG_MTALK_PROCESS_READ_DATA to the process object when data has arrived on the port. The handler for this message is shown in Code Display 9-3.

To receive data, MTalk does the following:

- checks to make sure the connection is still open
- locks the input buffer down (to get a pointer to the text)
- calls **ModemReceive()** (defined in modem.goc) to read incoming data
- displays read data
- unlocks the block

Code Display 9-3 Receiving Data Over A Serial Connection

```
@method MTalkProcessClass, MSG_MTALK_PROCESS_READ_DATA
{
        /*
         * recvDataSize - Amout of data actually read in.
         * recvDataPtr - Pointer to data to read in.
         * retVal - Return value of ModemReceive().
         */
    word        revDataSize;
    char *      recvDataPtr;
    StreamError retVal;

        /*
         * Check to see if connection is still open. Because of message queueing
         * delays, we may have received this data notification after the connection
         * has been closed.
         *
         * ConnectionOpen - global flag that keeps track of whether
         * the connection is open or closed.
         */
    if ( ConnectionOpen == FALSE )
    {
        return;
    }

        /*
         * Lock the input buffer down on the heap, and obtain a pointer to it.
         * (Note: This buffer is a memory block that was allocated in the
         * “Connecting” example.)
         */
    recvDataPtr = MemLock( recvBuffer );

        /*
         * A NULL pointer means that the memory manager has gone ahead
         * and discarded the buffer block since the last time it was
         * unlocked.  So we need to reallocate it here.  Note that the
         * MemHandle is still in use; it just isn't bound to any block of memory.
         */
    if ( recvDataPtr == NULL )
    {
            /*
             * Reallocate the block, locking it immediately,
             * so it doesn't get discarded again. If the block cannot
             * be re-allocated, memory must be low so close the connection.
             */
```

```
        if (( MemReAlloc( recvBuffer, RECV_BUFFER_SIZE, HAF_LOCK ) == NullHandle)
        {
                @send self::MSG_MTALK_PROCESS_CLOSE();
                return;
        }

            /*
             * After re-allocating the locked block, dereference its handle
             * to obtain a pointer to it.
             */
9.4     recvDataPtr = MemDeref( recvBuffer );
    }

        /*
         * Read available data up to the size of our buffer then display it.
         */
        if ( ModemReceive( RECV_BUFFER_SIZE, (byte*)recvDataPtr, &recvDataSize )
                                                        == STREAM_NO_ERROR )
        {
            @call MTalkOutText::MSG_VIS_TEXT_APPEND_PTR( recvDataPtr,
                                                    recvDataSize );
        }

        /*
         * If the read data filled the entire buffer, there's a good
         * chance more data is available so send the message through the
         * process object's queue again.  (We do not want to keep reading
         * in a loop in case the other side dumped a very large stream
         * of data; this would keep the process object from handling
         * other messages in its queue.)
         */

    if ( RECV_BUFFER_SIZE == recvDataSize ) {
        @send ,forceQueue self::MSG_MTALK_PROCESS_READ_DATA();
    }

        /*
         * Once all the data has been read, unlock the buffer block.
         */
    MemUnlock( recvBuffer );

} /* end of MSG_MTALK_PROCESS_READ_DATA */
```

Like **ModemSend()**, **ModemReceive()** calls a VirtualSerial routine, **VirtualSerialRead()**, to read incoming data. Because no blocking is

specified, **VirtualSerialRead()** reads only data that is currently available in the stream. (If you want the routine to block until there is enough data in the stream to fill the entire buffer, pass STREAM_BLOCK in **VirtualSerialRead()**.)

ModemReceive() takes three arguments:

recvBufLength
: Size of allocated buffer (in bytes).

recvBuf
: Byte pointer to the buffer.

bytesReceived
: Word pointer. **ModemReceive()** writes the number of bytes actually read in to this parameter.

ModemReceive() returns a **StreamError** value:

STREAM_NO_ERROR
: if it successfully reads the data.

STREAM_CLOSED
: if the stream is not open.

STREAM_SHORT_READ_WRITE
: if the required amount of data was not available and STREAM_NO_BLOCK was specified.

9.5 Disconnecting

When a call has ended, the modem driver sends a pre-registered message to a specified object. (You specify both the message and the receiving object in **ModemConnect()**.)

In the MTalk example, the modem driver sends MSG_MTALK_PROCESS_CALL_ENDED to the process object when the call has ended for any reason. This message checks if the connection is still open and if it is, calls yet another message, MSG_MTALK_PROCESS_CLOSE, to close it. The handler for MSG_MTALK_PROCESS_CLOSE is shown in Code Display 9-4.



Code Display 9-4 Ending A Serial Line Connection

```
@method MTalkProcessClass, MSG_MTALK_PROCESS_CLOSE
{
        /* close the connection */
    ModemDisconnect();

        /*
         * Free the buffer and re-set the (global) "connection" flag
         * to indicate the connection is closed.
         */
    MemFree( recvBuffer );
    recvBuffer = NullHandle;
    ConnectionOpen = FALSE;

} /* end of MSG_MTALK_PROCESS_CLOSE */
```

9.5

ModemDisconnect(), defined in modem.goc, takes care of the following:

- hangs up the phone
- closes the “virtual” serial port

ModemDisconnect() takes no parameters and returns void.

Virtual Serial Library



10

10.1 Overview 188
10.2 Function Reference 189
10.3 Differences from Normal Serial Driver 206
10.3.1 Differences in assembly language 206
10.3.2 Differences in C -language interface 207





This chapter discusses the external interface of *VSer*, the Virtual Serial library. VSer offers its client a stream interface similar to that of the GEOS Serial library, although not all the functionality of Serial library is implemented in VSer. For more information about GEOS Serial library see its chapter in the Concepts book.

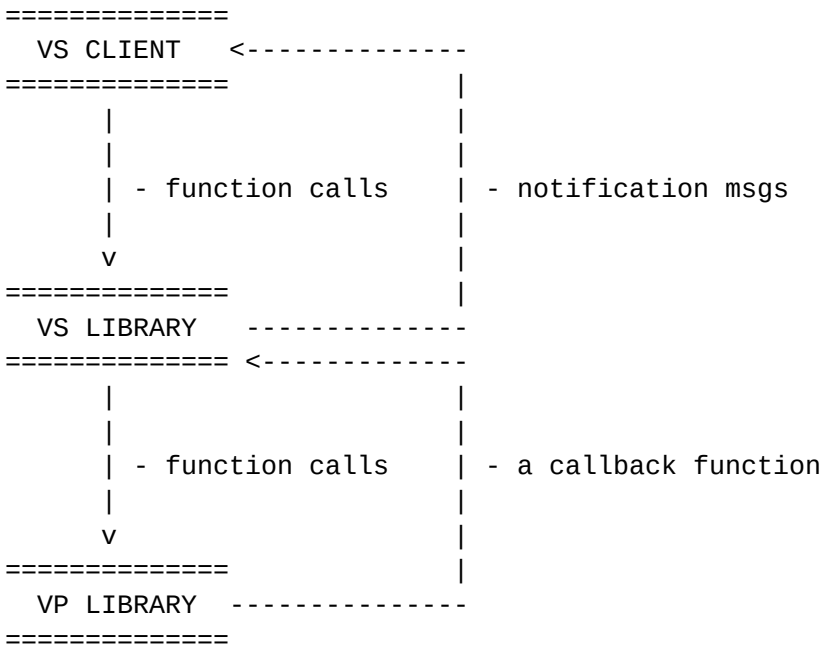
The VSer client writes data to a stream offered by VSer, which sends this data to Virtual Phone library as data messages to be forwarded to the phone unit. The functions VSer client can use are described in this document. All the functions can be called directly or by using a “strategy” function which takes the actual function number as a parameter (in register di). This strategy function is offered to ease the using of this library from assembly language code.

A VSer client may request the VS library to notify it of incoming data. These notifications are handled by sending a GEOS message to the client application.

Virtual Serial library provides all the necessary buffering needed for incoming data.

10.1 Overview

Virtual Serial library handles all needed flow control activities by itself. The calling application may select the used flow control method (software or hardware). Any flow control characters the client sends are ignored (i.e. they are treated as normal data).



When Virtual Serial client wants to start using VSer services, it has to first open the stream to VS. This is done by calling **VirtualSerialOpen()** function, which requests an access to Virtual Phone library's data resource. Because the Virtual Phone library allows only one data resource client to exist at any given time, this request fails if there is already an active data resource client. In this case the **VirtualSerialOpen()** function reports failure in its return value.

VirtualSerialWrite() is called when client wants to send data to the phone. This function takes a pointer to the data to send as parameter, also the data length is given as a parameter. VSer forms RBUS frame sized data packets

from the given data and stores them in its local slots. A seperate thread will then handle sending from these slots to the virtual phone. If no empty slots are available **VirtualSerialWrite()** either waits for a slot to become available or returns an error to the caller. The action chosen depends on blocking mode parameter given to **VirtualSerialWrite()** function.

When incoming data arrives, the Virtual Phone calls the **ReceiveData()** function with pointer to data (as well as the size of data) as a parameter. If the incoming data buffer is empty i.e. client has read all the previous incoming data, a notification message is sent to the client application (provided that notifications are enabled). This incoming data can be then read by client calling the **VirtualSerialRead()** function, which takes a pointer to the buffer to contain the data and amount of data to read as parameters.

When Virtual Serial client wants to stop using VSer services, it must call **VirtualSerialClose()** function, which flushes the incoming data buffer and releases the Virtual Phone's data resource.

10.2 Function Reference

VirtualSerialStrategy

```
void VirtualSerialStrategy( void)
```

Parameters:

parameters are passed in registers: ax, bx, cx, dx, ds, si and bp. di -register is used for passing the function number (DR_VIRTUAL_SERIAL_*)

Return value:

void (Return values are passed in registers)

This function serves as a common entry point for VirtualSerial -functions, when VS library is called from assembler code. Registers are used for passing return values. VirtualSerialStrategy is exported as the first function in the library and it is located in fixed memory block.

Notes:

See also: **CallVirtualSer()**



CallVirtualSer macro

```
CallVirtualSer macro func
```

Parameters:

func
number of the function to be called (DR_VIRTUAL_SERIAL_*)
ax, bx, cx, dx, ds, si, bp -registers used registers and their meaning depends on function to be called

Return value:

ax, bx, cx, dx, ds, si, bp -registers depends on function

This macro is used as common entry point for calls from assembler language code.

```
CallVirtualSer macro func
  mov di, func
  call VirtualSerialStrategy
endm
```

Notes: This macro is not necessarily needed anymore since VirtualSerialStrategy takes now parameters in registers.

See also: VirtualSerialStrategy

CallVirtualSer DR_VIRTUAL_SERIAL_OPEN

```
CallVirtualSer DR_VIRTUAL_SERIAL_OPEN
```

Parameters:

ax
StreamOpenFlags: SOF_NOBLOCK or SOF_TIMEOUT
cx
input buffer size (in bytes 1...4096)
dx
output buffer size, ignored: fixed 4352 bytes is used
bp
timeout value if SOF_TIMEOUT (in ticks 1...32768)

Return value:

carry
carry set if error
ax
error code describing the reason for failure:
VS_OK open successful

VS_DATA_RESOURCE_RESERVED VP data resource is already in use
VS_STREAM_ALREADY_OPEN stream has already opened by another client
VS_CANNOT_ALLOC memory for the streams buffer cannot be allocated

The client calls this function when it wants to open the stream to the phone. If somebody is already using VP data resource, this function

1 fails immediately, if SOF_NOBLOCK was specified in ax

2 tries again until successful or timeout expires (in bp), if SOF_TIMEOUT was specified in ax

Notes: This function must return VS_OK before client can start using other Virtual Serial -functions. If input buffer size is set > 4096 bytes, only 4096 bytes size buffer is used and no error is returned.

See also: CallVirtualSer DR_VIRTUAL_SERIAL_CLOSE
VirtualSerialOpen

CallVirtualSer DR_VIRTUAL_SERIAL_OPEN_FOR_DRIVER

`CallVirtualSer DR_VIRTUAL_SERIAL_OPEN_FOR_DRIVER`

Synonym for DR_VIRTUAL_SERIAL_OPEN

See also: CallVirtualSer DR_VIRTUAL_SERIAL_OPEN

CallVirtualSer DR_VIRTUAL_SERIAL_CLOSE

`CallVirtualSer DR_VIRTUAL_SERIAL_CLOSE`

Return value:
carry
carry set if error
ax
VS_OK if successfully closed
VS_ALREADY_CLOSED if stream was already closed

Description: Flushes data in incoming data buffer and closes connection to the phone. Outgoing buffer is sent.

Notes: No Virtual Serial function, except **VirtualSerialOpen()**, should be called after this function.



See also: CallVirtualSer DR_VIRTUAL_SERIAL_OPEN

CallVirtualSer DR_VIRTUAL_SERIAL_SET_NOTIFY

`CallVirtualSer DR_VIRTUAL_SERIAL_SET_NOTIFY`

Parameters:

ax
specifies which events are to be notified and how
VSNT_READER notify reader
VSNT_WRITER notify writer (not supported)
VSNE_ERROR error notification (not supported)
VSNE_DATA data/space available notification
VSNE_MODEM notify when modem input signals change
VSNM_NONE disable notifier
VSNM_ROUTINE notify by calling fixed routine (not supp.)
VSNM_MESSAGE notify by sending a message cx:dx
destination of output bp method to send

Return value:

carry
carry set if error
ax
VS_OK successfully registered
VS_NOT_SUPPORTED notifier not supported
VS_CLOSED stream not open
VS_NO_NOTIFIER tried to disable non-existing notifier

Registers a notifier for any of the possible events. Event and notification method is specified in ax register. Client must choose one VSNT_, one VSNE_ and one VSNM_ parameter and OR them together.

Currently supported combinations are:

- VSNT_READER | VSNE_DATA | VSNM_MESSAGE
  notifies reader for data available using messaging
- VSNT_READER | VSNE_DATA | VSNM_NONE
  turns off data notifications
- VSNT_READER | VSNE_MODEM | VSNM_MESSAGE
  notifies reader for changed modem status using messaging

- VSNT_READER | VSNE_MODEM | VSNM_NONE
  turns off modem notifications

Currently not-supported parameters are: VSNT_WRITER, VSNE_ERROR and VSNM_ROUTINE

See also: **VirtualSerialSetNotify()**

CallVirtualSer DR_VIRTUAL_SERIAL_FLUSH

`CallVirtualSer DR_VIRTUAL_SERIAL_FLUSH`

Parameters: ax
operation direction
STREAM_READ flushes incoming data buffer
STREAM_WRITE ignored
STREAM_BOTH same as STREAM_READ

Return value:
carry
carry set if error
ax
VS_OK successfully flushed
VS_CLOSED stream not open

Flushes data from input buffer.

See also: **VirtualSerialFlush()**

CallVirtualSer DR_VIRTUAL_SERIAL_READ

`CallVirtualSer DR_VIRTUAL_SERIAL_READ`

Parameters:
ax
blocking mode
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK return as many bytes as are currently available
cx
number of bytes to read (in bytes 1...32768)
ds:si buffer to which to read

Return value:
carry
carry set if return value is not VS_OK or VS_SHORT_READ_WRITE



cx
number of bytes read
ax
VS_OK all required data successfully read
VS_SHORT_READ_WRITE the required amount of data was not available and STREAM_NO_BLOCK was specified
VS_CLOSED stream not open

Fills the buffer specified in ds:si with required amount of incoming data. If there is less than required amount of data in VS's buffer and STREAM_BLOCK was specified, this function will block until enough data comes available. If STREAM_NO_BLOCK was specified, this function returns immediately with VS_SHORT_READ_WRITE in ax.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not send enough data this function will block forever.

See also: **VirtualSerialRead CallVirtualSer DR_VIRTUAL_SERIAL_READ_BYTE**

■ CallVirtualSer DR_VIRTUAL_SERIAL_READ_BYTE

`CallVirtualSer DR_VIRTUAL_SERIAL_READ_BYTE`

Parameters:

ax
blocking mode
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK return as many bytes as are currently available

Return value:

carry
carry set if no byte was available and ax was STREAM_NO_BLOCK
al
byte read

Reads one byte from incoming data buffer. If no data is available and STREAM_BLOCK was specified will block wait until enough data is received. In STREAM_NO_BLOCK mode this function returns immediately.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not send enough data this function will block forever.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_READ**

CallVirtualSer DR_VIRTUAL_SERIAL_WRITE

`CallVirtualSer DR_VIRTUAL_SERIAL_WRITE`

Parameters:

ax
blocking mode
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK write as many bytes as could be written
cx
number of bytes to write (in bytes 1...32768)
ds:si buffer from which to write

Return value:

carry
carry set if return value is not VS_OK nor VS_SHORT_READ_WRITE
cx
number of bytes written
ax
VS_OK successfully written
VS_CLOSED stream not open
VS_SHORT_READ_WRITE the required amount of space was not available and STREAM_NO_BLOCK was specified

Writes the specified amount of data from given buffer to stream. If STREAM_BLOCK mode was specified and there is not enough space in stream, will block wait until more space becomes available. If STREAM_NO_BLOCK was specified, returns immediately with VS_SHORT_READ_WRITE in ax register.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not read its data, stream may be constantly full and this function will block for maximum of two minutes. After two minutes function returns with VS_SHORT_READ_WRITE as return value and discards the given data.

See also: **VirtualSerialWrite CallVirtualSer DR_VIRTUAL_SERIAL_WRITE_BYTE**



CallVirtualSer DR_VIRTUAL_SERIAL_WRITE_BYTE

`CallVirtualSer DR_VIRTUAL_SERIAL_WRITE_BYTE`

Parameters:

ax
blocking mode
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK write as many bytes as could be written
cl
byte to write

Return value:

carry
carry set if byte could not be written and STREAM_NO_BLOCK was specified (i.e. return value is not VS_OK)
ax
VS_OK successfully written
VS_CLOSED stream not open
VS_SHORT_READ_WRITE the required amount of space was not available and STREAM_NO_BLOCK was specified

Writes the given byte to the stream. If STREAM_BLOCK mode was specified and there is not enough space in stream, will block wait until more space becomes available. If STREAM_NO_BLOCK was specified, returns immediately with VS_SHORT_READ_WRITE in ax register.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not read its data, stream may be constantly full and this function will block for maximum of two minutes. After two minutes function returns with VS_SHORT_READ_WRITE as return value and discards the given data.

See also: **VirtualSerialWriteByte CallVirtualSer DR_VIRTUAL_SERIAL_WRITE**

CallVirtualSer DR_VIRTUAL_SERIAL_SET_FLOW_CONTROL

`CallVirtualSer DR_VIRTUAL_SERIAL_SET_FLOW_CONTROL`

Parameters:

ax type of flow control desired
VSFC_SOFTWARE use software flow control
VSFC_HARDWARE use hardware flow control

Return value:

carry
carry set if error
ax
VS_OK successfully set
VS_CLOSED stream not open

Selects the flow control method used between modem in the phone and Virtual Serial library.

Notes: Both hardware and software flow control may be in effect at the same time. In software flow control mode no flow control characters (XON/XOFF) are passed to the client within data. Neither is flow control characters expected to receive from client within outgoing data.

See also: **VirtualSerialSetFlowControl()**

■ CallVirtualSer DR_VIRTUAL_SERIAL_SET_FORMAT

```
CallVirtualSer DR_VIRTUAL_SERIAL_SET_FORMAT
```

Parameters:

ax
Data format
cx
Baud rate

Return value:

carry
carry set if error
ax
VS_OK successfully set
VS_CLOSED stream not open

Sets format and speed.

Notes: Sets desired format and speed for modem to use. Even if there would be data in output buffer modem format can be changed.

See also: **VirtualSerialSetFormat()**



CallVirtualSer DR_VIRTUAL_SERIAL_GET_MEDIUM

```
CallVirtualSer DR_VIRTUAL_SERIAL_GET_MEDIUM
```

Return value:
ax
GMID_CELL_MODEM
dx
MANUFACTURER_ID_GEOWORKS

Returns medium and manufacturer id.

VirtualSerialOpen()

```
VirtualSerialStatus VirtualSerialOpen(
        VSOpenFlags         stream_open_mode,
        word                input_buffer_size,
        word                output_buffer_size,
        word                timeout )
```

Parameters:
StreamOpenFlags stream_open_mode
SOF_NOBLOCK or SOF_TIMEOUT
word input_buffer_size
input buffer size (in bytes 1...4096)
word output_buffer_size
ignored (4352 bytes size output buffer is always used)
word timeout
timeout value if SOF_TIMEOUT (in ticks 1...32768))

Return value: VirtualSerialStatus
VS_OK open successful
VS_DATA_RESOURCE_RESERVED VP data resource is already in use
VS_STREAM_ALREADY_OPEN stream has already opened by another client
VS_CANNOT_ALLOC memory for the streams buffer cannot be allocated

The client calls this function when it wants to open the stream to the phone. If somebody is already using the Virtual Phone data resource, this function

1. fails immediately, if SOF_NOBLOCK was specified
2. tries again until successful or timeout expires, if SOF_TIMEOUT was specified

Notes: This function must return VS_OK before client can start using other Virtual Serial -functions. Outgoing data is not buffered in VirtualSerial library, but sent directly from calling client's buffer.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_OPEN VirtualSerialClose**

VirtualSerialClose()

```
VirtualSerialStatus VirtualSerialClose( void )
```

Return value: VirtualSerialStatus
VS_OK if successfully closed
VS_CLOSED if stream was already closed

Waits until all the outgoing data is send to remote unit and then flushes data in incoming data buffer and closes connection to remote unit.

Notes: No Virtual Serial function, except **VirtualSerialOpen()**, should be called after this function.

See also: CallVirtualSer DR_VIRTUAL_SERIAL_CLOSE
VirtualSerialOpen

VirtualSerialSetNotify()

```
VirtualSerialStatus VirtualSerialSetNotify(VSNotifyType notify_type,
  optr destination, word method )
```

Parameters: VSNotifyType notify_type specifies which events are to be notified and how optr destination pointer to destination object to receive the notification word method message to send to destination object

Return value: VirtualSerialStatus
VS_OK successfully registered
VS_NOT_SUPPORTED notifier not supported
VS_CLOSED stream not open

Registers a notifier for any of the possible events. Event and notification method is specified in ax register. Client must choose one VSNT_, one VSNE_ and one VSNM_ parameter and OR them together e.g. notify_type = VSNT_READER | VSNE_DATA | VSNM_MESSAGE notifies reader for data available using messaging.



Notes: Currently only VSNT_READER, VSNE_DATA, VSNE_MODEM and VSNM_MESSAGE are supported.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_SET_NOTIFY**

VirtualSerialFlush()

```
VirtualSerialStatus VirtualSerialFlush(
        word flushMode)
```

Parameters:

word flushMode
operation direction:
STREAM_READ flushes incoming data buffer
STREAM_WRITE ignored
STREAM_BOTH same as STREAM_READ

Return value: VirtualSerialStatus
VS_OK successfully flushed
VS_CLOSED stream not open

Flushes data from incoming buffer.

See also: C**allVirtualSer DR_VIRTUAL_SERIAL_FLUSH**

VirtualSerialRead()

```
VirtualSerialStatus VirtualSerialRead(
        VSBlockingMode      blocking_mode,
        word                buffer_size,
        byte                *buffer,
        word                *bytes_read )
```

Parameters:

VSBlockingMode blocking_mode
Specify whether to block if there is not enough data waiting to be read.
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK return as many bytes as are currently available
word buffer_size
Size of passed buffer (i.e. amount of data to read from stream) (in bytes 1...32768)
byte *buffer
Pointer to buffer where data from stream will be written.
word *bytes_read

This function will write to this variable the number of bytes actually read from the stream.

Return value: VirtualSerialStatus
VS_OK all required data successfully read
VS_SHORT_READ_WRITE the required amount of data was not available and STREAM_NO_BLOCK was specified
VS_CLOSED stream not open

Fills the buffer specified in 'buffer' with required amount of incoming data. If there is less than required amount of data in VS's buffer and STREAM_BLOCK was specified, this function will block until enough data comes available. If STREAM_NO_BLOCK was specified, this function returns immediately with VS_SHORT_READ_WRITE as a return value.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not send enough data this function will block forever.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_READ VirtualSerialReadByte**

■ VirtualSerialReadByte()

```
VirtualSerialStatus VirtualSerialReadByte(
        VSBlockingMode      blocking_mode,
        byte                *data_byte )
```

Parameters:
VSBlockingMode blocking_mode
Specify whether to block if there is not enough data waiting to be read.
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK return error if no incoming byte was available
byte *data_byte
Read a byte from the stream, and write it to this variable.

Return value: VirtualSerialStatus
VS_OK all required data successfully read
VS_SHORT_READ_WRITE the required amount of data was not available and STREAM_NO_BLOCK was specified
VS_CLOSED stream not open



Reads one byte from incoming data buffer. If no data is available and STREAM_BLOCK was specified will block wait until enough data is received. In STREAM_NO_BLOCK mode this function returns immediately.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not send enough data this function will block forever.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_READ_BYTE VirtualSerialRead**

■ VirtualSerialWrite()

```
VirtualSerialStatus VirtualSerialWrite(
          VSBlockingMode      blocking_mode,
          word                buffer_size,
          const byte          *buffer,
          word                *bytes_written)
```

Parameters:

VSBlockingMode blocking_mode
Specify whether to block if there is not enough room to write all the data.
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK write as many bytes as could be written
word buffer_size
Size of passed data buffer (i.e. amount of data to write to stream) (in bytes 1...32768)
const byte *buffer
Pointer to data to write to stream.
word *bytes_written
This function will write to this variable the number of bytes actually written to the stream.

Return value: VirtualSerialStatus
VS_OK successfully written
VS_CLOSED stream not open
VS_SHORT_READ_WRITE the required amount of space was not available and STREAM_NO_BLOCK was specified

Writes the specified amount of data from given buffer to stream. If STREAM_BLOCK mode was specified and there is not enough space in stream, will block wait until more space becomes available. If STREAM_NO_BLOCK was specified, returns immediately with VS_SHORT_READ_WRITE as a return value.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not read its data, stream may be constantly full and this function will block for maximum of two minutes. After two minutes function returns with VS_SHORT_READ_WRITE as return value and discards the given data.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_WRITE**, **VirtualSerialWriteByte()**

■ VirtualSerialWriteByte

```
VirtualSerialStatus VirtualSerialWriteByte(
        VSBlockingMode      blocking_mode,
        byte                data_byte )
```

Parameters:

VSBlockingMode blocking_mode
Specify whether to block if there is not enough room to write all the data.
STREAM_BLOCK block waiting for the full request to be filled
STREAM_NO_BLOCK return error if byte could not have been written
byte data_byte
Write this byte to the stream.

Return value: VirtualSerialStatus
VS_OK successfully written
VS_CLOSED stream not open
VS_SHORT_READ_WRITE the required amount of space was not available and STREAM_NO_BLOCK was specified

Writes the given byte to the stream. If STREAM_BLOCK mode was specified and there is not enough space in stream, will block wait until more space becomes available. If STREAM_NO_BLOCK was specified, returns immediately with VS_SHORT_READ_WRITE as a return value.

Notes: STREAM_BLOCK mode is dangerous to use: if the opposite end does not read its data, stream may be constantly full and this function will block for maximum of two minutes. After two minutes function returns with VS_SHORT_READ_WRITE as return value and discards the given data.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_WRITE_BYTE**, **VirtualSerialWrite()**



VirtualSerialSetFlowControl()

```
VirtualSerialStatus VirtualSerialSetFlowControl(
          VSFlowControl flow_control_mode)
```

Parameters:

word flow_control_mode type of flow control desired
VSFC_SOFTWARE use software flow control
VSFC_HARDWARE use hardware flow control

Return value: VirtualSerialStatus VS_OK successfully set VS_CLOSED stream not open

Selects the flow control method used between modem in RU and Virtual Serial library.

See also: **CallVirtualSer DR_SERIAL_SET_FLOW_CONTROL**

VirtualSerialSetFormat()

```
VirtualSerialStatus VirtualSerialSetFormat(
          VSSerialFormat      format,
          VSSerialBaud        baud)
```

Parameters:

VSSerialFormat format Data format
VSSerialBaud baud
Baud rate

Return value: VirtualSerialStatus
VS_OK successfully set
VS_CLOSED stream not open

Sets format and speed.

Notes: Sets desired speed for modem to use.

See also: **CallVirtualSer DR_VIRTUAL_SERIAL_SET_FORMAT**

VirtualSerialReceiveData()

```
void VirtualSerialReceiveData(
          byte destination,
          byte msglength,
          char *data )
```

Parameters:

byte destination

target of the message, may be:
PDA_DATA
PDA_DATA_SIGNALS
PDA_DATA_FLOW_CTRL
byte msgLength
length of the data message pointed by 'data'
char *data
pointer to the data message

VP calls this function when there is data coming from RU to data client.

Notes: This function should never be called directly from client application.

MSG_MS_DATA_NOTIFY prototype

```
@prototype word MSG_VS_DATA_NOTIFY_PROTO(
          word number_of_bytes,
          word direction)
```

Parameters: word number_of_bytes
number of bytes received to incoming
buffer word
direction
SERIAL_READ always, only incoming buffer is notified

This is the prototype for the data notification message Virtual Serial Library sends to the application.

MSG_VS_MODEM_NOTIFY_PROTO prototype

```
@prototype word MSG_VS_MODEM_NOTIFY_PROTO(
          word modem_status,
          word dummy)
```

Parameters:
word modem_status
modem status bit field (lsb)
SMS_CTS_CHANGED Clear to send changed
SMS_DSR_CHANGED Data-Set Ready changed
SMS_RING_CHANGED Activated only when the SMS_RING indicator changes from active to inactive (signals the end of a RING signal on the incoming phone line)
SMS_DCD_CHANGED Carrier Detect changed
SMS_CTS Clear to send (modem will accept data)
SMS_DSR Data-Set Ready (modem is happy)



SMS_RING Ring Detect
SMS_DCD Carrier Detect (msb) word dummy always 0

This is the prototype for the modem notification message Virtual Serial Library sends to the application.

MSG_VS_FCLASS_NOTIFY_PROTO prototype

```
@prototype word MSG_VS_FCLASS_NOTIFY_PROTO(
         word fclass,
         word dummy)
```

Parameters: word fclass Informs which fax class is used Return value: none

Sends a data notify to application. It tells application the fclass type.

Notes: There is no asm interface for this.

10.3 Differences from Normal Serial Driver

First, and most importantly, Virtual Serial library is implemented as library, not a driver as GEOS serial driver is.

10.3.1 Differences in assembly language

Unimplemented functions

DR_STREAM_GET_DEVICE_MAP,
DR_STREAM_GET_ERROR,
DR_STREAM_SET_ERROR,
DR_STREAM_SET_THRESHOLD,
DR_STREAM_QUERY,
DR_SERIAL_GET_FORMAT,
DR_SERIAL_SET_MODEM,
DR_SERIAL_GET_MODEM,
DR_SERIAL_DEFINE_PORT,
DR_SERIAL_STAT_PORT,

DR_SERIAL_CLOSE_WITHOUT_RESET,
DR_SERIAL_REESTABLISH_STATE,
DR_SERIAL_PORT_ABSENT

Differences in parameters

- bx -register used for passing unit number in GEOS serial driver is ignored in Virtual Serial library (VS supports only one “port”).
- In DR_SERIAL_OPEN output buffer size (cx-register) is ignored, outgoing data is not buffered.
- In DR_SERIAL_CLOSE ax-register is ignored as STREAM_LINGER mode is only supported (not STREAM_DISCARD).
- In DR_SERIAL_FLUSH operation mode STREAM_WRITE is ignored and STREAM_BOTH is treated as STREAM_READ.
- In DR_STREAM_SET_NOTIFY VSNT_WRITER and VSNM_ROUTINE are not supported, passing them as parameters causes VS_NOT_SUPPORTED to be returned.
- In DR_SERIAL_SET_FLOW_CONTROL cl- and ch- register values ignored also in hardware flow control mode.

Differences in return values

- ax -register contains always error code describing the reason for failure. Exception to this is DR_SERIAL_READ_BYTE function, which returns byte read in ax -register (in al) and does not return error value.
- All functions set carry on exit if function is not successful, except DR_SERIAL_READ and DR_SERIAL_WRITE which treat VS_SHORT_READ_WRITE as VS_OK.

10.3.2 Differences in C -language interface

Unimplemented functions

SerialGetDeviceMap(),
SerialGetError(),
SerialSetError(),



SerialSetThreshold(),
SerialCloseWithoutReset(),
SerialGetFormat(),
SerialGetModem(),
SerialQuery(),
SerialEscLoadOptions(),
SerialSetModem(),
SerialDefinePort(),
SerialStatPort()

Differences in parameters of implemented functions

- None of the VirtualSerial -functions use driver and unit parameters.
- VirtualSerialOpen() function does not use outBuffSize parameters.
- VirtualSerialClose() function does not use any parameters.
- VirtualSerialSetFlowControl() function does not use modem and status parameters.

E-mail



11

11.1 Example: Sending a Text Message 211
11.1.1 Essentials 211
11.1.2 Variations 218
11.2 The Preparation Sequence 221
11.3 Mailbox Library 224
11.4 Communicator-Specific Features 227
11.5 Useful Mailbox Messages 228





This chapter describes how to write an application that sends e-mail on the Nokia 9000 Communicator. Your application will use the Mailbox Library, which is a general-purpose message management system; it has been customized for use on the Communicator.

Before reading this document, you should already understand the platform-independent basics of sending e-mail. The examples use VM files and Huge Arrays; you can find more information about these data storage types in the Virtual Memory chapter of the Concepts book. **MailboxSendControlClass** is subclassed from **GenControlClass**; controllers are discussed in the Generic UI Controllers chapter of the Objects book. You might consider whether another type of message (SMS, for example) is more appropriate or easier to implement in your application; information on SMS messages is found in "SMS Guide," Chapter 3.

11.1 Example: Sending a Text Message

The sample application **email.goc**, included with your SDK, demonstrates how to send the contents of an editable text object as an e-mail message.

11.1.1 Essentials

In **email.goc**, we use a **MailboxSendControl** object, attached to the application's generic UI tree, to inform our application when the user is ready to send an e-mail message. After she composes an e-mail message, the user presses the *send* trigger, which activates the **MailboxSendControl** object. It sends our process object a system message (MSG_META_MAILBOX_CREATE_MESSAGE) that we handle by storing the e-mail message body. We then pass a pointer to this body (and other registration information) back to the send controller via MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE. We can inspect the



success of the registration attempt by handling MSG_META_MAILBOX_MESSAGE_REGISTERED.

To register the message in our MSG_META_MAILBOX_CREATE_MESSAGE handler, we:

- Store the message body in a format understood by the e-mail transport driver.
- Fill an **MSCRegisterMessageArgs** structure with information about the message, including the body data type, the body format, a pointer to the body location, delivery flags, and delivery time constraints.
- Register the message arguments with our send controller via MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE; the controller in turn registers the message with the Mailbox Library after getting the destination internet address from the e-mail driver's **MLTDAddressControl** object.

A static **MailboxSendControl** object is therefore declared in one of our object resource blocks, as shown in Code Display 11-1. (Here, we subclass **MailboxSendControlClass** to allow future message interception.)

Code Display 11-1 EMailSendControl Declaration

```
@class EMailSendControlClass,
                MailboxSendControlClass;
@endc;
@classdecl EMailSendControlClass;

....
@chunk TCHAR EMailSendControlTitle[] = “Send”;
@object ComplexMonikerClass EMailSendControl = {
        ComplexMoniker = EMailSendControlClass;
        CMI_topText = @EMailSendControlTitle;

        GII_visibility = GIV_POPUP;
        GCI_output = process;
        ATTR_MAILBOX_SEND_CONTROL_SEND_DOCUMENT;
        ATTR_MAILBOX_SEND_CONTROL_SINGLE_TRANSPORT = {
         { GMID_CELL_MODEM, MANUFACTURER_ID_GEOWORKS},
         { GMTID_SMTP, MANUFACTURER_ID_GEOWORKS },
         0
        };
```

```
        HINT_SEEK_MENU_BAR;
        HINT_SEEK_REPLY_BAR;
        HINT_SEEK_SLOT = 1;
}
```

The **MailboxSendControl** object must be on the notification list MGCNLT_ACTIVE_LIST in order to be built properly at application start-up. This is done in the declaration of the application object, shown in Code Display 11-2.

Code Display 11-2 Application Object Declaration

```
@object FoamSubApplicationClass EMailApp = {
       GI_visMoniker = list { @EMailTextMoniker };
       GI_comp = @EMailPrimary;
       FSAI_launcherApp = FA_EXTRAS_APP;

       gcnList( MANUFACTURER_ID_GEOWORKS, GAGCNLT_WINDOWS ) = @EMailPrimary;
       gcnList( MANUFACTURER_ID_GEOWORKS, MGCNLT_ACTIVE_LIST ) = @EMailSendControl;
}

@visMoniker EMailTextMoniker = “SDK:\rE-Mail”;
```

All objects sent on the Communicator are assumed MOT_DOCUMENT, so we declare the controller as supporting that type with ATTR_MAILBOX_SEND_CONTROL_SEND_DOCUMENT. We also specify e-mail as our only transport choice with the ATTR_MAILBOX_SEND_CONTROL_SINGLE_TRANSPORT declaration. (GMTID_SMTP requests the e-mail transport.) If we did not specify this attribute, the send controller would conduct a dialog with the user to ask for the transport type (e-mail, SMS, print job, etc.).

The send controller will invoke our MSG_META_MAILBOX_CREATE_MESSAGE method, shown in Code Display 11-3, when it's time to package the message body for sending.



Code Display 11-3 Handler for MSG_META_MAILBOX_CREATE_MESSAGE

```
#define MVMAIL_MAILBOX_MESSAGE_FLAGS \
        ( MMF_SEND_WITHOUT_QUERY | \
          MMF_DELETE_BODY_AFTER_TRANSMISSION | \
        ( MMP_FIRST_CLASS << MMF_PRIORITY_OFFSET ) | \
        ( MDV_READ << MMF_VERB_OFFSET ) )

@method EMailProcessClass, MSG_META_MAILBOX_CREATE_MESSAGE
/*
 * RETURNS:     void;
 * PARAMETERS:  optr sendControl,
 *              word transHandle;
 */
{
        word                    mailboxVMStatus;
        VMBlockHandle           arrayVMBlock;
        VMTreeAppRef            myAppRef;
        MSCRegisterMessageArgs  msgArgs;
        TCHAR                   mySubjectStr[] = “Email Sample Msg”;
        MailboxDataFormat       formats[2];
        MailboxDataFormat       reqFormat;
        MSCSetAddressesArgs     setAddr;
        MailboxTransAddr        transAddr;
        TCHAR                   sampMsgStr[] = “Sample mail message body.\n”;
        MemHandle               msgBodyHandle;
        TCHAR *                 msgBodyPtr;
        dword                   msgBodyLength;

        /*
        * Verify format with MMSendControl.
        */
        formats[0].MDF_id = GMDFID_TEXT_ARRAY;
        formats[0].MDF_manuf = MANUFACTURER_ID_GEOWORKS;
        formats[1].MDF_id = GMDFID_INVALID;
        formats[1].MDF_manuf = MANUFACTURER_ID_GEOWORKS;

        reqFormat = @call sendControl::MSG_MAILBOX_SEND_CONTROL_CHOOSE_FORMAT(
                formats, transHandle );

        if ( ( 0 == reqFormat.MDF_id ) && ( 0 == reqFormat.MDF_manuf ) ) {
                FoamDisplayError( @EMailNoFormatErrStr );
                return;
        }
        /*
```

```
* OPTIONAL:
* Set default address. This address will appear in the
* To: and Subject: fields when the address controller comes up.
*/
transAddr.MTA_transAddr = “user@internet.com”;
transAddr.MTA_transAddrLen = strlen( transAddr.MTA_transAddr );
transAddr.MTA_userTransAddr = “Your Subject Here”;
setAddr.MSCSAA_transAddrs = &transAddr;
setAddr.MSCSAA_numTransAddrs = 1;
setAddr.MSCSAA_transportAndOption.MTAO_transport.MT_id = GMTID_SMTP;
setAddr.MSCSAA_transportAndOption.MTAO_transport.MT_manuf =
                                        MANUFACTURER_ID_GEOWORKS;
setAddr.MSCSAA_transportAndOption.MTAO_transOption = 0;

@call sendControl::MSG_MAILBOX_SEND_CONTROL_SET_ADDRESSES( &setAddr );

/*
* Get the Mailbox VM file.
*/
mailboxVMFile_g = MailboxGetVMFile( 0, &mailboxVMStatus );
if ( NullHandle == mailboxVMFile_g ) {

        @call sendControl::MSG_MAILBOX_SEND_CONTROL_CANCEL_MESSAGE(
                transHandle, -1 );

        FoamDisplayError( @EMailMBoxAllocFailedStr );
        return;
}

/*
* Create huge array; returns handle to itself.
*/
arrayVMBlock = HugeArrayCreate( mailboxVMFile_g,
        sizeof( TCHAR ),
        0 );

/*
* Fill in the block with some form of message.
*/

HugeArrayAppend( mailboxVMFile_g,
                arrayVMBlock,
                LocalStringLength( sampMsgStr ),
                sampMsgStr );

/*
```




```
        * Register a VM text array with the mailbox system.
        */
        msgArgs.MSCRMA_bodyStorage.MS_manuf = MANUFACTURER_ID_GEOWORKS;
        msgArgs.MSCRMA_bodyStorage.MS_id =    GMSID_VM_TREE;
        msgArgs.MSCRMA_bodyFormat.MDF_manuf = MANUFACTURER_ID_GEOWORKS;
        msgArgs.MSCRMA_bodyFormat.MDF_id =    GMDFID_TEXT_ARRAY;
        msgArgs.MSCRMA_bodyRef =    &myAppRef;
        msgArgs.MSCRMA_bodyRefLen = sizeof( myAppRef );
        myAppRef.VMTAR_vmChain = VMCHAIN_MAKE_FROM_VM_BLOCK( arrayVMBlock );
        myAppRef.VMTAR_vmFile =     mailboxVMFile_g;

        msgArgs.MSCRMA_summary =            mySubjectStr;

/* destApp not used for remote delivery
 *
 *      strcpy( msgArgs.MSCRMA_destApp.GT_chars, "APPL" );
 *      msgArgs.MSCRMA_destApp.GT_manufID = MANUFACTURER_ID_GEOWORKS;
 */
        msgArgs.MSCRMA_flags =              MVMAIL_MAILBOX_MESSAGE_FLAGS;
        msgArgs.MSCRMA_startBound =         MAILBOX_NOW;
        msgArgs.MSCRMA_endBound =           MAILBOX_ETERNITY;

        /*
        * Tell the send control that the body is ready for
        * transmission; msgArgs is filled in.
        */
        @call sendControl::MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE(
                                                    &msgArgs, transHandle );

} /* MSG_META_MAILBOX_CREATE_MESSAGE */
```

We get a VM file handle from the Mailbox Library:

```
mailboxVMFile_g = MailboxGetVMFile( 0,
                                    &mboxVMStatus );
```

The call to **MailboxGetVMFile()** increments a reference count for the particular Mailbox VM file; it must be matched by a call to **MailboxDoneWithVMFile()** so Mailbox can delete this file when no longer needed. The transport driver uses the same mechanism to preserve the file until the message is transmitted.

After the message body is copied to the Huge Array, we fill in the **MSCRegisterMessageArgs** structure.

The data storage type is a VM file:

```
msgArgs.MSCRMA_bodyStorage.MS_manuf =
                            MANUFACTURER_ID_GEOWORKS;
msgArgs.MSCRMA_bodyStorage.MS_id = GMSID_VM_TREE;
```

The data format is text array. This is the only body format supported by the e-mail transport driver.

```
msgArgs.MSCRMA_bodyFormat.MDF_manuf =
                            MANUFACTURER_ID_GEOWORKS;
msgArgs.MSCRMA_bodyFormat.MDF_id =
                                  GMDFID_TEXT_ARRAY;
```

We store a pointer to the structure describing where to find the message body:

```
msgArgs.MSCRMA_bodyRef = &myAppRef;
msgArgs.MSCRMA_bodyRefLen = sizeof( myAppRef );
myAppRef.VMTAR_vmChain =
      VMCHAIN_MAKE_FROM_VM_BLOCK( arrayVMBlock );
myAppRef.VMTAR_vmFile = mailboxVMFile_g;
```

We specify the message delivery flags:

```
#define MVMAIL_MAILBOX_MESSAGE_FLAGS \
      ( MMF_SEND_WITHOUT_QUERY | \
        MMF_DELETE_BODY_AFTER_TRANSMISSION | \
      ( MMP_FIRST_CLASS << MMF_PRIORITY_OFFSET ) | \
      ( MDV_READ << MMF_VERB_OFFSET ) )

msgArgs.MSCRMA_flags = MVMAIL_MAILBOX_MESSAGE_FLAGS;
```

Mailbox always sends VM data from its own VM files, so you must either specify MMF_BODY_DATA_VOLATILE (so Mailbox will copy the message body to a Mailbox VM file at registration time) or provide the message body in a file returned by **MailboxGetVMFile()**. We also specify MMF_DELETE_BODY_AFTER_TRANSMISSION so Mailbox will delete the VM chain after the message is sent successfully. (Mailbox always deletes its own copy when MMF_BODY_DATA_VOLATILE is used; you must dispose of the original.)



We specify the delivery window. These arguments request immediate transmission (if possible), with no time limit on subsequent attempts.

```
msgArgs.MSCRMA_startBound = MAILBOX_NOW;
msgArgs.MSCRMA_endBound = MAILBOX_ETERNITY;
```

Finally, we pass the parameters back to the send controller that invoked our method. The send controller will get the destination address from the transport driver's address controller and register the message.

```
@call sendControl::
      MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE(
            &msgArgs, transHandle );
```

11.1.2 Variations

A few optional or illustrative features have been included in the **email.goc** sample application.

11.1.2.1 Getting the Body Format

Because our send controller specifies e-mail transport only (via ATTR_MAILBOX_SEND_CONTROL_SINGLE_TRANSPORT), the format must be GMDFID_TEXT_ARRAY. However, if we also supported SMS messages or printing, we would use MSG_MAILBOX_SEND_CONTROL_CHOOSE_FORMAT to determine what format to supply. As shown in the example, the transaction is automatically canceled if none of our formats is acceptable to the transport driver.

```
formats[0].MDF_id = GMDFID_TEXT_ARRAY;
formats[0].MDF_manuf = MANUFACTURER_ID_GEOWORKS;
formats[1].MDF_id = GMDFID_INVALID;
formats[1].MDF_manuf = MANUFACTURER_ID_GEOWORKS;

reqFormat = @call
sendControl::MSG_MAILBOX_SEND_CONTROL_CHOOSE_FORMAT(
                          formats, transHandle );
```

11.1.2.2 Setting a Default Address

The user usually enters the destination address herself. The example shows how to supply a default address that will appear when the transport driver's address controller's dialog first comes up. (The address controller will encode this address in its internal format; this is not the format of the final **MailboxTransAddr** structure supplied to **MailboxRegisterMessage()**.)

```
transAddr.MTA_transAddr = “user@internet.com”;
transAddr.MTA_transAddrLen = strlen(
                            transAddr.MTA_transAddr );
transAddr.MTA_userTransAddr = “Your Subject Here”;
setAddr.MSCSAA_transAddrs = &transAddr;
setAddr.MSCSAA_numTransAddrs = 1;
setAddr.MSCSAA_transportAndOption.MTAO_transport.
                                    MT_id = GMTID_SMTP;
setAddr.MSCSAA_transportAndOption.MTAO_transport.
            MT_manuf = MANUFACTURER_ID_GEOWORKS;
setAddr.MSCSAA_transportAndOption.MTAO_transOption
                                                    = 0;
@call
sendControl::MSG_MAILBOX_SEND_CONTROL_SET_ADDRESSES(
                                        &setAddr );
```

11.1.2.3 Getting the Return Value and Cleaning Up

After the message-registration attempt, we handle MSG_META_MAILBOX_MESSAGE_REGISTERED to check the success code and perform any necessary cleanup.

Code Display 11-4 Handler for MSG_META_MAILBOX_MESSAGE_REGISTERED

```
@method EMailProcessClass, MSG_META_MAILBOX_MESSAGE_REGISTERED
/*
 * RETURNS:     void;
 * PARAMETERS:  MemHandle args;
 */
{
        MSCMessageRegisteredArgs * msgRegisteredArgs;
```

```
        msgRegisteredArgs = MemLock( args );
        if ( ME_SUCCESS != msgRegisteredArgs->MSCMRA_error ) {
                FoamDisplayError( @EMailMsgNotSentErrStr );
        }
        MemUnlock( args );
        MailboxDoneWithVMFile( mailboxVMFile_g );

        @send EMailBodyText::MSG_VIS_TEXT_DELETE_ALL();
} /* MSG_META_MAILBOX_MESSAGE_REGISTERED */
```

We call **MailboxDoneWithVMFile()** to release our reference to the body data. If you wished to access, copy, or re-use the body data after an unsuccessful registration attempt, you would defer the call to **MailboxDoneWithVMFile()** until you were finished. Finally, if the message registration is not successful, Mailbox will not delete the body data regardless of the MMF_DELETE_BODY_AFTER_TRANSMISSION flag; you should call **VMFreeVMChain()** at some point, unless you are eventually successful in registering the data.

11.1.2.4 Other Registration Methods

This example uses a **MailboxSendControl** object to call **MailboxRegisterMessage()** after collecting registration information from the user and your process object. You may also register outgoing e-mail with the Mailbox Library by calling **MailboxRegisterMessage()** directly.

To register e-mail with **MailboxRegisterMessage()**, you must prepare the message body and an address structure, then pass them (along with other information) via pointers stored in a **MailboxRegisterMessageArgs** structure. The routine returns a success code and message identifier. This method works well for local delivery (transport == GMTID_LOCAL), as no address structure is needed and no transport driver is loaded. For remote delivery, you must fill a **MailboxTransAddr** structure in the destination address format expected by the transport driver, in this case the e-mail driver **mltd**.

11.1.2.5 Local Delivery

For local delivery, you will need to specify *MRA_transport* == GMTID_LOCAL and a geode token in *MRA_destApp* in the **MailboxRegisterMessageArgs** structure. (This token is ignored in messages queued for remote delivery on the Communicator.) This example shows one way to fill a **GeodeToken** structure:

```
strcpy( msgArgs.MSCRMA_destApp.GT_chars, “APPL” );
msgArgs.MSCRMA_destApp.GT_manufID =
                            MANUFACTURER_ID_GEOWORKS;
```

11.2 The Preparation Sequence

Figure 11-1 gives an overview of the sending process used in this example. (Some messages have been omitted for clarity.) When the user presses the *send* trigger, the send controller receives MSG_GEN_INTERACTION_INITIATE. **MailboxSendDialog** and **MailboxAddressControl** objects are instantiated and used to manage user dialogs and message/address preparation. The basic steps are:

- A transaction handle and associated storage are created to save information as the message is prepared for registration. Use this handle when cancelling, saving, or requesting information about the transaction.
- The address controller requests the message body before the address dialog.
- The send controller instructs your process object to store the message body. When your handler is invoked, you may gather information about the requested format or make other requests as appropriate.
- When the body is ready, your process notifies the send controller.
- The send controller tells the address controller that the body is ready. The latter gets the destination addresses from the user (Figure 11-2 and Figure 11-3), then uses MSG_MAILBOX_SEND_CONTROL_GET_ADDRESSES to inform the send controller that it's ready to supply them. (This MSG goes out when the user presses *send* in the address dialog, Figure 11-3.)

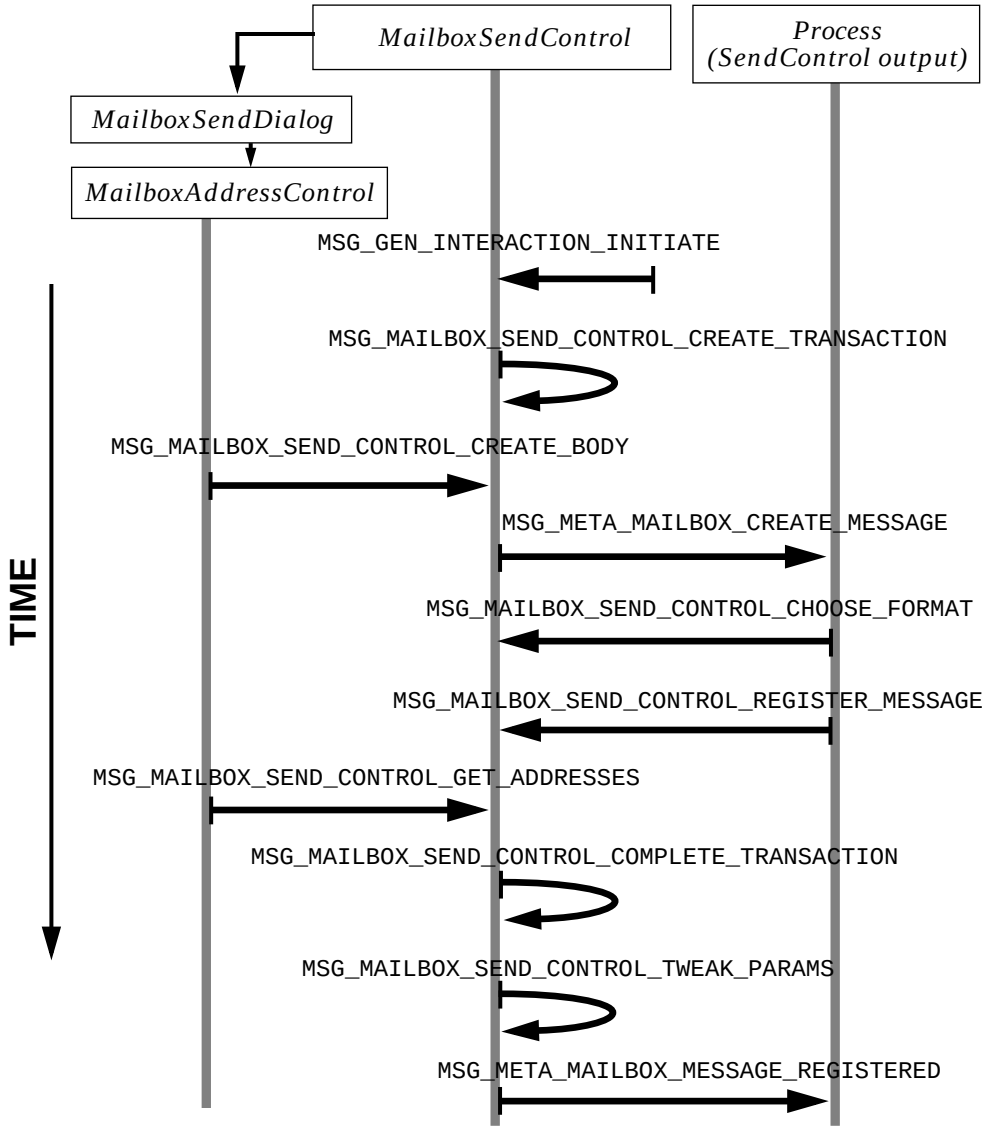


Figure 11-1 *MSG Exchange During E-mail Sending*
Many other messages are omitted or possible; see text for discussion.

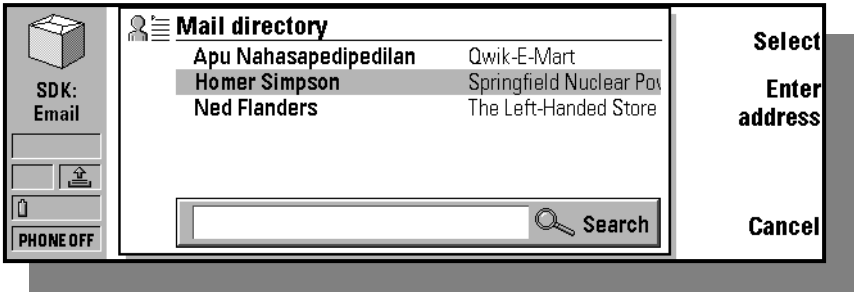


Figure 11-2 *Contacts Database Dialog*
The user may include addresses from her database or proceed to the address entry dialog.

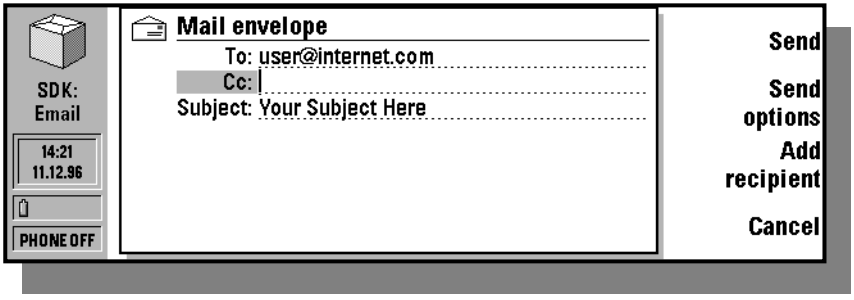


Figure 11-3 *Address Entry Dialog*
The user may go back to the contacts database, send the message, or cancel.

- The send controller completes the transaction. It sends itself the MSG_MAILBOX_SEND_CONTROL_TWEAK_PARAMS message, allowing you to subclass **MailboxSendControlClass**, intercept the message, and modify any of the registration parameters (or even cancel or defer the transaction based on those parameters).

- The send controller notifies your process that the registration is complete. If successful, the message is transferred to the outbox and assigned a **MailboxMessage** token. The outbox display (which may be brought up by the user independently) is shown in Figure 11-4.

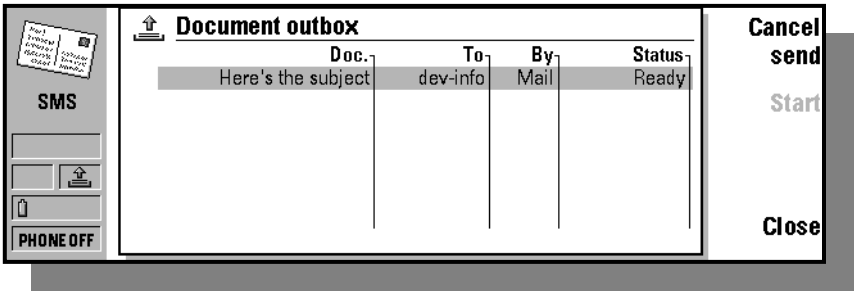


Figure 11-4 *Outbox Display*
The user may launch this application to view and modify the outgoing message queue.

11.3 Mailbox Library

The Mailbox Library insulates you from most details associated with specific transport methods and device platforms, as shown in Figure 11-5. In most cases, you simply have a message to be sent to a destination; the Mailbox Library takes care of handling the different transport and data drivers, and incoming and outgoing message queues.

Transport drivers know about different logical message formats and how to give messages to the transmission device drivers. Thus, the e-mail driver knows what an e-mail message looks like, what the pieces mean, and how to work with a data driver to send it.

Data drivers know how to move blocks of data stored in a particular way, such as in VM trees. They don't know (or care) about the meaning of those data.

Figure 11-6. shows the Mailbox Library delivery model. Incoming messages are placed in the *inbox*, along with the geode token of the destination

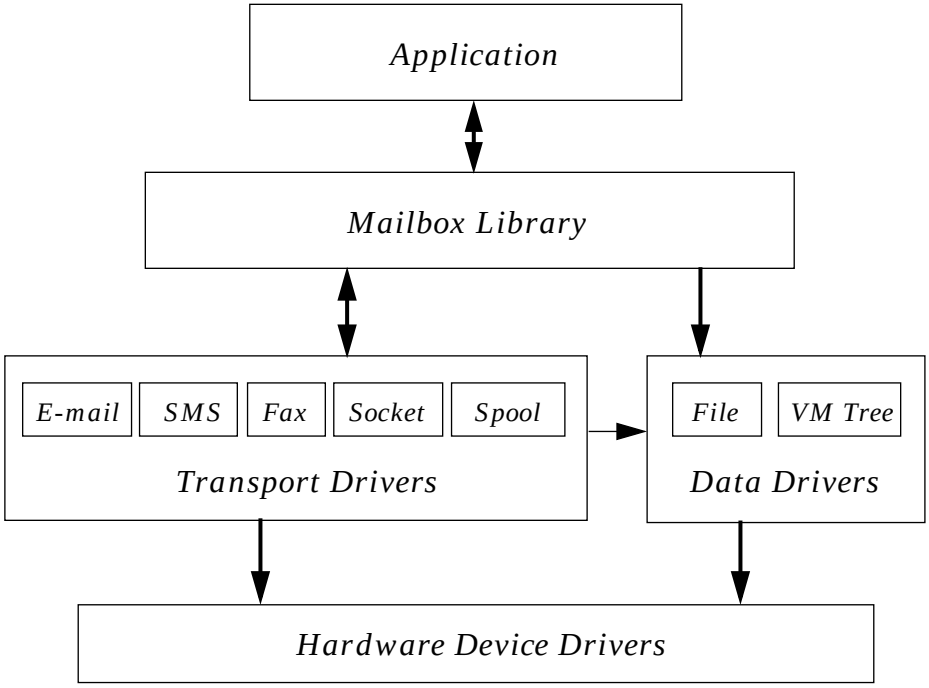


Figure 11-5 *Mailbox Library Interaction Model*
The Mailbox Library provides a uniform interface to applications, shielding them from the details of the particular transport media.

application; outgoing messages are placed in the *outbox*, along with a transport request. As a result, Mailbox knows the destination of every message in its registry. Typically, incoming messages are received by a transport driver and passed (via mailbox) to a viewing application; likewise, outgoing messages typically originate in an application and are passed via Mailbox to a transport driver. Applications may also put messages in the inbox for other applications if they know the destination's geode token.

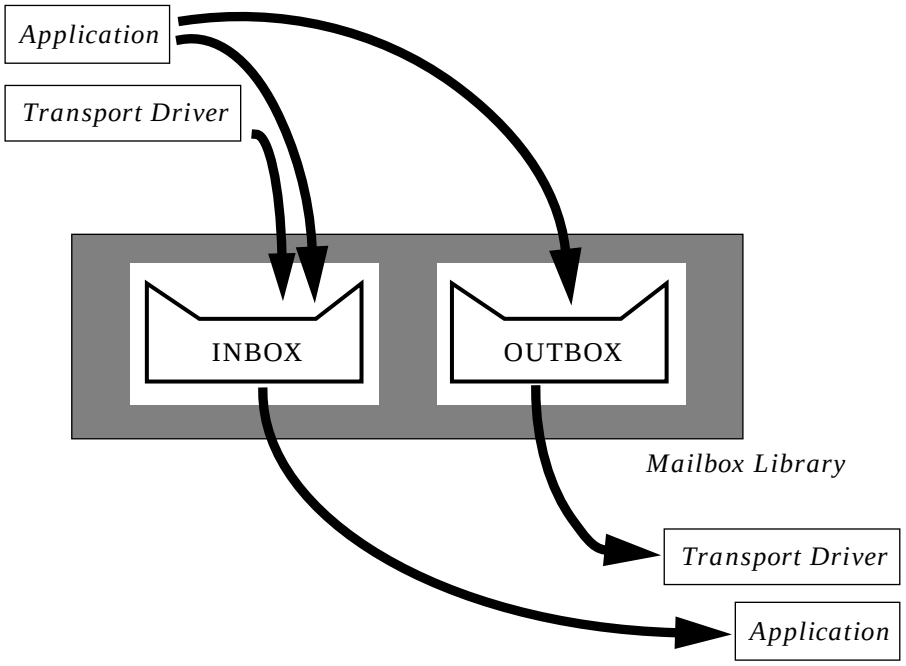


Figure 11-6 *Mailbox Library Delivery Model*

The Mailbox Library maintains a registry of incoming and outgoing messages awaiting final disposition. Transport drivers pass incoming messages to the inbox, along with the geode token of the destination application. Similarly, applications place messages in the outbox, along with a transport method request. Applications may also place messages for other (local) applications directly in the inbox.

The Mailbox Library mediates all interactions with the inbox and outbox.

11.4 Communicator-Specific Features

The Communicator does not have an inbox control panel; thus, messages may not be left in the inbox unless the Mailbox Library can immediately find the application designated by the message's destination geode token.

Furthermore, the built-in e-mail application handles all e-mail reception itself without notifying the Mailbox Library of incoming messages. The user directs the e-mail application to visit her maildrop and she fetches messages by hand; selected messages are deposited directly in the "Received E-mail" directory without any intervention from Mailbox. Thus, there is no opportunity for other applications to receive notification of received e-mail.

Finally, delivery flags (such as MMP_FIRST_CLASS or MDV_DELIVER) and the destination geode token are not encoded in the outgoing message; they are important only for direct local delivery (although specifying MMP_THIRD_CLASS might delay an outgoing message). Also, the message flag MMF_SEND_WITHOUT_QUERY is automatically asserted.

You may consult the Mailbox definition file to get the most up-to-date information or learn more about specific parameters and system messages.

mailbox.def (mailbox.goh) Main include file for the Mailbox Library. Defines **MailboxSendControlClass** and associated structures, enums, and methods, as well as Mailbox global procedures.

You may find references in **mailbox.def** to Mailbox features that are not available on the Communicator. These features are listed below.

Control Panels — Not Implemented
: Mailbox-supplied control panels for manipulating the inbox and outbox. (A separate application on the Communicator manipulates the outbox queue.)

Select Contents — Not Implemented
: A dialog allowing the user to specify what to send: document, selection, page range, etc.



Poof MessagesNot Implemented

Quick messages composed from a control panel or **MailboxSendControl** object.

No Unknown AppsEnforced

Messages may not be left for applications that cannot be found. (Without a control panel, they could not be deleted from the inbox.)

11.5 Useful Mailbox Messages

MailboxGetVMFile(), MSG_META_MAILBOX_CREATE_MESSAGE, MSG_MAILBOX_SEND_CONTROL_CHOOSE_FORMAT, MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE, MSG_META_MAILBOX_MESSAGE_REGISTERED

MSG_META_MAILBOX_CREATE_MESSAGE

```
void    MSG_META_MAILBOX_CREATE_MESSAGE(
        optr   sendControl,
        word   transHandle);
```

The send controller uses this message to prompt its output to store the message body and register it for transmission.

Source: **MailboxSendControlClass**

Destination: controller output (optr in its GCI_output instance data)

Parameters: *sendControl* The MailboxSendControl managing this request.

transHandle Transaction identifier.

MSG_MAILBOX_SEND_CONTROL_CHOOSE_FORMAT

```
MailboxDataFormat MSG_MAILBOX_SEND_CONTROL_CHOOSE_FORMAT(
        MailboxDataFormat
              *format,
        word   transHandle);
```

Send this message to your sendControl object (from within your MSG_META_MAILBOX_CREATE_MESSAGE handler) if you are serving multiple transport drivers with the same method. The send controller will query the transport driver and return the preferred format from the list you provide, or GMDFID_INVALID if none are acceptable.

Source: controller output

Destination: **MailboxSendControlClass**

Parameters: *format* Pointer to an array of **MailboxDataFormat** structures, the message formats you support. The array is terminated with a format of GMDFID_INVALID.

transHandle Transaction identifier supplied by MSG_META_MAILBOX_CREATE_MESSAGE argument.

Return: The preferred format of the transport driver, or GMDFID_INVALID.

MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE

```
void    MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE(
MSCRegisterMessageArgs
        *mscRMArgs,
word    transHandle);
```

Send this message back to the send controller that sent MSG_META_MAILBOX_CREATE_MESSAGE, informing it that the e-mail message body and registration arguments are ready.

Source: controller output

Destination: **MailboxSendControlClass**

Parameters: *mscRMArgs* Message registration parameters.

transHandle Transaction identifier supplied by MSG_META_MAILBOX_CREATE_MESSAGE argument.

Structures:

```
typedef struct {
        MailboxStorage MSCRMA_bodyStorage;
        MailboxDataFormat MSCRMA_bodyFormat;
        const void  *MSCRMA_bodyRef;
        word        MSCRMA_bodyRefLen;
        MailboxMessageFlags MSCRMA_flags;
        const char  *MSCRMA_summary;
/* Application to receive message at destination
 * (not supported by Communicator e-mail driver */
```




```
        GeodeToken  MSCRMA_destApp;
        FileDateAndTime MSCRMA_startBound;
        FileDateAndTime MSCRMA_endBound;
} MSCRegisterMessageArgs;

typedef struct {
        word            MS_id;
        ManufacturerID  MS_manuf;
} MailboxStorage;

/* Possible values of MS_id, with MS_manuf ==
 * MANUFACTURER_ID_GEOWORKS (only GMSID_VM_TREE
 * supported for e-mail on the Communicator):
*/

typedef enum {
        GMSID_FILE = 0x0,
        GMSID_VM_TREE,
} GeoworksMailboxStorageID;
#define GMSID_HUGE_ARRAY      GMSID_VM_TREE

typedef struct {
        word            MDF_id;
        ManufacturerID  MDF_manuf;
} MailboxDataFormat;

/* Possible values of MDF_id, with MS_manuf ==
 * MANUFACTURER_ID_GEOWORKS (only GMDFID_TEXT_ARRAY
 * supported for e-mail on the Communicator):
 */

typedef enum {
        GMDFID_INVALID = 0x0,
        GMDFID_STREAM_GSTRING,
        GMDFID_VM_GSTRING,
        GMDFID_TRANSFER_ITEM,
        GMDFID_TEXT_CHAIN,
        GMDFID_TEXT_FILE,
        GMDFID_DOCUMENT,
        GMDFID_ADDRESS_CARD,
        GMDFID_APPOINTMENT,
        GMDFID_INK,
        GMDFID_VM_COMPOSITE,
```

```
        GMDFID_FAX_FILE,
        GMDFID_SHORT_MESSAGE,
        GMDFID_FOAM_FILE,
        GMDFID_TEXT_ARRAY
} GeoworksMailboxDataFormatID;
#define GMDFID_END_OF_LIST    GMDFID_INVALID

/* The VMTreeAppRef structure is used to pass
 * the location of a VMTree message body */

typedef struct {
        VMChain       VMTAR_vmChain;
        VMFileHandle  VMTAR_vmFile;
} VMTreeAppRef;

typedef WordFlags MailboxMessageFlags;

#define MMF_BODY_DATA_VOLATILE (0x0080)
#define MMF_SEND_WITHOUT_QUERY (0x0040)
#define MMF_PRIORITY  (0x0020 | 0x0010)
#define MMF_PRIORITY_OFFSET 4
#define MMF_VERB (0x0008 | 0x0004 | 0x0002)
#define MMF_VERB_OFFSET 1
#define MMF_DELETE_BODY_AFTER_TRANSMISSION (0x0001)

typedef ByteEnum MailboxMessagePriority;
#define MMP_URGENT      0x1
#define MMP_EMERGENCY   0x0
#define MMP_FIRST_CLASS 0x2
#define MMP_THIRD_CLASS 0x3

typedef ByteEnum MailboxDeliveryVerb;
#define MDV_DELIVER 0x0
#define MDV_VIEW   0x1
#define MDV_PLAY   0x2
#define MDV_ACCEPT 0x3
#define MDV_READ   0x4
#define MDV_FILE   0x5
```




MSG_META_MAILBOX_MESSAGE_REGISTERED

void MSG_META_MAILBOX_MESSAGE_REGISTERED(
MemHandle args);

The send controller's output should handle this message if you wish to check the return status after a call to MSG_MAILBOX_SEND_CONTROL_REGISTER_MESSAGE or perform any post-registration cleanup.

Source: **MailboxSendControlClass**

Destination: controller output

Parameters: *args* Handle to the **MSCMessageRegisteredArgs** structure for a message you have attempted to register.

Structures:

```
typedef struct {
      MailboxError      MSCMRA_error;
      MailboxMessage    MSCMRA_message;
      MailboxStorage    MSCMRA_bodyStorage;
      MailboxDataFormat MSCMRA_bodyFormat;
      word              MSCMRA_bodyRefLen;
} MSCMessageRegisteredArgs;

typedef enum {    /* MailboxError codes */
      ME_SUCCESS = 0x0,
      ME_NOT_ENOUGH_MEMORY,
      ME_CANNOT_LOAD_DATA_DRIVER,
      ME_CANNOT_LOAD_TRANSPORT_DRIVER,
      ME_UNABLE_TO_CREATE_TRANSMIT_THREAD,
      ME_CANNOT_ENQUEUE_MESSAGE,
      ME_USER_CANCELED,
      ME_LOST_CONNECTION,
      ME_CANNOT_CREATE_MESSAGE_FILE,
      ME_CANNOT_RESIZE_MBOX_REF,
      ME_CANNOT_SAVE_MESSAGE_FILE,
      ME_CANCELLED_BY_RECEIVER,
      ME_MESSAGE_BLOCKS_ARE_MISMATCHED,
      ME_DATA_DRIVER_CANNOT_STORE_MESSAGE_BODY,
      ME_INSUFFICIENT_DISK_SPACE,
```

```
      ME_CANNOT_CONNECT,
/*----------------- Start of Unrecoverable errors */
      ME_ADDRESS_INVALID=ME_UNRECOVERABLE,
      ME_DATA_DRIVER_CANNOT_ACCESS_MESSAGE_BODY,
      ME_UNSUPPORTED_BODY_FORMAT,
      ME_MESSAGE_BODY_INVALID,
      ME_INVALID_MESSAGE,
      ME_REPLY_ADDRESS_NOT_AVAILABLE,
      ME_DESTINATION_APPLICATION_UNKNOWN,
      ME_UNKNOWN_DISK_ERROR,
      ME_APP_REF_BUF_TOO_SMALL,
      ME_CANNOT_OPEN_MESSAGE_FILE,
      ME_CANNOT_READ_MESSAGE_FILE,
/* ---Generic permanent failure to connect */
      ME_CAN_NEVER_CONNECT,
} MailboxError;
```






N9000C

Fax Sending



12

12.1 Destination Phone Number 237
12.2 Drawing the Contents 240
12.2.1 Declaring the Objects 240
12.2.2 Supplying the Document's Name 241
12.2.3 Supplying the Fax's Contents 241





This chapter describes how an application can use the fax libraries and Mailbox library to compose and send a fax. Most GEOS devices, including the Nokia 9000 Communicator, come with an application that composes and sends simple fax documents; this chapter will show you how you can write a program that composes a custom fax using graphics commands.



This chapter assumes that you know the basics of GEOS programming. It also assumes that you know how to draw the image that you want to fax, given a GState to draw to. (Most visible objects draw themselves when sent a MSG_VIS_DRAW; documentation of kernel drawing commands may be found in "Graphics Environment," Chapter 23, "Drawing Graphics," Chapter 24.)

A PrintControl object provides the API for drawing the contents of the fax; while this chapter will tell you what you need to know for simple faxes, more information about the PrintControl is available in "The Spool Library," Chapter 15 of the Object Reference Book.

Sending a fax may be broken down into the following tasks:

- Finding out where to send the fax; and
- Drawing the contents of the fax.

Ideally, these tasks are carried out using standard object classes that conform to the device's look and feel.

12.1 Destination Phone Number

To prompt the user for the fax's destination phone number while employing the device's standard look and feel, we will use the Mailbox library and its fax adjuncts. Specifically, we will use a **MailboxSendControl** object, configured for use with fax.

Make sure the following lines appear in the application's .gp file:

```
library mailbox   # Defines MailboxSendControlClass
library spool     # Defines PrintControlClass
```




We will use a **MailboxSendControl** object to provide the UI that prompts the user for a phone number. When it comes time to draw the contents of our fax, we'll also need a **PrintControl** object.

We make sure that these controller objects are added to the correct GCN lists in the declaration of our application's Application object:

Code Display 12-1 Adding the Fax Controls to the Proper GCN Lists

In the declaration of the application's Application object, we make sure that the MailboxSendControl and PrintControl objects are placed on the proper GCN lists.

```
@object … MyApp = {
    …
    gcnList(MANUFACTURER_ID_GEOWORKS, GAGCNLT_SELF_LOAD_OPTIONS) =
                                … @MyPrintControl, @MySendControl;
    gcnList(MANUFACTURER_ID_GEOWORKS, MGCNLT_ACTIVE_LIST) = … @MySendControl;
    gcnList(MANUFACTURER_ID_GEOWORKS, GAGCNLT_MAILBOX_SEND_CONTROL) =
                                                        … @MySendControl;
    ATTR_GEN_APPLICATION_PRINT_CONTROL = @MyPrintControl;
}
```

We make sure that the **MailboxSendControl** will be visible to the user; the straightforward way to do this is to add it to the application's generic UI tree. For example:

```
@object GenPrimaryClass MyPrimary = {
    GI_comp  = …, @MySendControl;
}
```

Finally, we're ready to declare the **MailboxSendControl** object itself:

Code Display 12-2 MailboxSendControl Object Ready for Fax

```
@object MailboxSendControlClass MySendControl = {
    GI_visMoniker = “Send”;
    GII_visibility = GIV_POPUP;
    GCI_output = process;
    MSCI_defBodyType = MOT_SELECTION;

    ATTR_MAILBOX_SEND_CONTROL_SEND_SELECTION;
    ATTR_GEN_INIT_FILE_CATEGORY = “fax” ;
```

```
    ATTR_MAILBOX_SEND_CONTROL_SINGLE_TRANSPORT = {
    {GMID_CELL_MODEM, MANUFACTURER_ID_GEOWORKS},
    {GMTID_FAX_SEND, MANUFACTURER_ID_GEOWORKS},0};

    ATTR_MAILBOX_SEND_CONTROL_TRANSPORT_HINT_OPTR =  {
      {GMTID_FAX_SEND, MANUFACTURER_ID_GEOWORKS}, 0,
      @MyPrintControl};
}
```

(If your application is to run on the Nokia 9000 Communicator, use ComplexMonikerClass to give your **MailboxSendControl** the standard N9000 look and feel. In the example above, you would replace

```
@object MailboxSendControlClass MySendControl = {
    GI_visMoniker = “Send”;
```

…with…

```
@object ComplexMonikerClass MySendControl = {
    ComplexMoniker = MailboxSendControlClass;
    CMI_topText = CMT_SELECT_RECEIVER;
    HINT_SEEK_REPLY_BAR; /* make this button 0 */
    HINT_SEEK_SLOT = 0;
```

)

ATTR_MAILBOX_SEND_CONTROL_TRANSPORT_HINT_OPTR tells the **MailboxSendControl** that when it comes time to send a fax, it should look to the **MyPrintControl** object to provide the data. The FaxSendTD Mailbox transport driver knows how to interact with PrintControl objects; we will add code to our program to work with the PrintControl to draw our fax contents.

ATTR_GEN_INIT_FILE_CATEGORY tells the send control where to find the user's faxing preferences within the INI file—the “fax” category.

ATTR_MAILBOX_SEND_CONTROL_SINGLE_TRANSPORT specifies that this **MailboxSendControl** only sends data by fax. To allow your application to send data in additional ways, get rid of this ATTR.

The **MailboxSendControl** would ask the user what data they wish to send: the current selection, current page, all pages. In most fax programs, we don't want to give the user that choice. In our example, we use the ATTR_MAILBOX_SEND_CONTROL_SEND_SELECTION vardata field to



prevent the controller from presenting the user with this choice—the **MailboxSendControl** will always try to send the "current selection," though our program ignores any idea of selection.

12.2 Drawing the Contents

We have configured the **MailboxSendControl** so that when the user has specified a recipient for the fax, the control will query a **PrintControl** object for the fax's contents.

12.2.1 Declaring the Objects

Again, make sure the following line is in the application's .gp file:

```
library spool    # Defines PrintControlClass
```

In the previous section, we saw that the **PrintControl** object needed to appear on certain GCN lists and in the **MailboxSendControl**'s ATTR_MAILBOX_SEND_CONTROL_TRANSPORT_HINT_OPTR field. (See Code Display 12-1 and Code Display 12-2.)

Assuming that the **PrintControl** won't be used for anything else, the following declaration will suffice:

```
@object PrintControlClass MyPrintControl = {
    GI_states = @default & ~GS_ENABLED;
    PCI_docNameOutput = process;
    PCI_output = process;
}
```

Here we have turned off the GS_ENABLED flag because the **PrintControl**'s UI will never appear in the application; it isn't part of the application's generic UI tree. (To include the PrintControl's UI in your application, leave it enabled, add it to the generic UI tree, and see "The Spool Library," Chapter 15 of the Object Reference Book for more information.)

The *PCI_output* line specifies that the application's process object will be responsible for drawing the fax's contents. The *PCI_docNameOutput* line

specifies that the application's process will be responsible for providing the fax's name.

12.2.2 Supplying the Document's Name

The process object (or whatever object is specified in the **PrintControl**'s *PCI_docNameOutput* field) must supply the fax's name when queried. (On the Nokia 9000 Communicatory, this name is not especially important; this is just a required part of the **PrintControl**'s API.) The name need not be unique.

To fulfill its responsibility, the object must handle MSG_PRINT_GET_DOC_NAME, responding by sending MSG_PRINT_CONTROL_SET_DOC_NAME. If the message is not handled in this manner, the fax will never be sent.

The following is an acceptable handler:

Code Display 12-3 Handling MSG_PRINT_CONTROL_SET_DOC_NAME for faxing

The printCtrlOD value is an argument to the message.

```
@method MyProcessClass, MSG_PRINT_GET_DOC_NAME {
    @send printCtrlOD::MSG_PRINT_CONTROL_SET_DOC_NAME("Good Morning");
}
```

12.2.3 Supplying the Fax's Contents

The process object (or whatever object is specified in the **PrintControl**'s *PCI_output* field) describes the fax's contents in the method for the imported message MSG_PRINT_START_PRINTING. It will receive this message after the user has specified a recipient for the fax. This message is defined:

```
void MSG_PRINT_START_PRINTING( \
                    optr            printCtrlOD , \
                    GStateHandle    gstate );
```

There are some things that the handler for this message must do:

- It should draw the fax's contents.
- After drawing the fax's contents, it should call **GrNewPage()** to generate a form feed. (It should also call **GrNewPage()** between the pages of a multi-page fax, of course.)
- It should send the **PrintControl** MSG_PRINT_CONTROL_PRINTING_COMPLETED.

There are many things that the handler for this message may do. For information about some of them, see "The Spool Library," Chapter 15 of the Object Reference Book. (If you read that chapter, you might think it's important that the print job's page range is also determined; this is not important for faxing.)

The following example shows a handler that draws a fax's contents to contain some text and a rectangle:

Code Display 12-4 Handling MSG_PRINT_START_PRINTING when faxing

```
@method MyProcessClass, MSG_PRINT_START_PRINTING {
/* We draw the contents of the fax. */
    GrDrawText(gstate, 20,40, "Good Morning", 0);
    GrDrawRect(gstate, 10,30, 110,55);

/* Make sure that our last drawing command is a form-feed */
    GrNewPage(gstate, PEC_FORM_FEED);

/* Let the PrintControl know we're done */
    @send printCtrlOD::MSG_PRINT_CONTROL_PRINTING_COMPLETED();
}
```

In the example, we draw the fax's contents with a couple of kernel graphics commands. We might just have easily passed gstate as the graphic state argument to some Visible object's MSG_VIS_DRAW message (also passing the **DrawFlag** DF_PRINT).

We also need to tell the **PrintControl** how many pages we're printing. The first page that we draw on is page one. If there is a separate cover page, that will be page zero. To specify how many pages there are to print, we call MSG_PRINT_CONTROL_SET_SELECTED_PAGE_RANGE, and then

MSG_PRINT_CONTROL_SET_SELECTED_PAGE_RANGE, passing the number of pages to print as that message's second argument.

It is possible that the fax cover page will appear on the same page on which drawing starts. To find out how much vertical space this cover page will take up on the first page, call the PrintControl with MSG_PRINT_GET_FIRST_PAGE_OFFSET; the return value will be the height of the cover page. (To properly follow naming conventions, this message should be called MSG_PRINT_CONTROL_GET_FIRST_PAGE_OFFSET; as of this writing it has not yet had its name fixed.)





N9000C

Update



13

13.1 Software Version Number 247
13.2 User's Name 248
13.3 IMEI Code Number 249
13.5 New Standard Monikers, Bitmaps 252
13.6 Non-exclusive Lists 252
13.7 Incremental Searches 253
13.8 Contact's URL and Password 253
13.9 ContactList Searches 253
13.10 Unknown Phone Numbers 254





This chapter discusses various new API features that have been added in recent versions of the Nokia 9000 Communicator.

13.1 Software Version Number

The INI file contains the version number. The [ui] category's *productName* field will be one of the following strings:

- Responder Build 3.8.8 (original release, does not support features described in this document)
- Responder Build 4.3.7 (update)
- Responder Build 4.8.? (later update)

There is also a software serial number, in the *swSerialNumber* field. This field has the format: 000000/00/AAABBCCDDLLLFKK/0

AAA: hardware version
1st digit: hardware major protocol (2)
2nd digit: hardware minor protocol (a)
3rd digit: hardware variant number (0)

This is not the actual hardware version, but rather which version the software was compiled for. All released images are for 2.10.0 ("2a0" hex).

BB: build major number (03,04)

CC: build minor number (08,03)

DD: build variant number (08,07)
Same as for productName, but padded with extra zeros.

LLL: localization country code (044 = English, 033 = French, 049 = German)

F: 0=all current releases 1=certain future releases

KK: keyboard mapping version (01 = English, 02 = French, 03 = German, 04 = Scandinavian)



Use one of the the **IniFileReadString...()** routines to find out the value of these INI file fields.

Code Display 13-1 Getting the Software Version Number

```
@start StringRes;
  @chunk TCHAR UICategory[] = “ui”;
  @chunk TCHAR SWSerialNumber[] = “swSerialNumber”;
@end StringRes;

…

@method TestProcessClass, MSG_TEST_PROCESS_GET_VERSION
{
    TCHAR version[28] = “”;
    TCHAR *category;
    TCHAR *key;
    word  value;

    MemLock( OptrToHandle( @UICategory ) );
    category = (TCHAR *) LMemDeref( @UICategory );
    key = (TCHAR *) LMemDeref( @SWSerialNumber );

    /*
     * Extract the information you want from version string.
     */

    MemUnlock( OptrToHandle( @UICategory ) );
}
```

13.2 User’s Name

The ContDB library has a new function, **ContactGetUserName()**, which returns the user’s name.

13.3 IMEI Code Number

To read the phone's IMEI number, you will need to use the **EciImeiGet()** function, provided by the eci library. This function takes one parameter, a pointer to a buffer to fill with the code number string. The buffer must be at least 17 bytes long, to hold the null-terminated 16-character code string. The function returns zero if the IMEI code was read succesfully, non-zero otherwise.

Code Display 13-2 Reading the IMEI Code

```
@include <eci.goh>

...

@method TestProcessClass, MSG_TEST_PROCESS_GET_IMEI
{
    TCHAR imei[17] = “”;

    if ( EciGetImei( imei) == 0)
    {
      /*
       * We got IMEI code, do any check you want.
       */
    }

    else
    {
      /*
       * For some reason query failed.
       */
    }
}
```

13.4 New SMS Header

In addition to the old way to send messages to application using their **GeodeToken** in front of the actual data, a new user data header is now

introduced. Otherwise sending and receiving messages has not changed. The new header adopts the concept from Narrow Band Socket specification.

The new header format is //SCKddoo or //SCKLddddoooo where dd (and dddd) represent “destination port” and oo (and oooo) represent “originator port”. Both these headers are terminated with space character.These ports are ascii coded hexadecimal numbers.

Short header covers ports from 0 to 255 (00 to FF in hex) and the longer form from 0 to 65535 (0000 to FFFF in hex).

The range of port numbers is divided to reserved port address range and to dynamic and/or private port address range. Usage of port address range 0 to 49151 is restricted and assignment of a port in this range requires contacting port address assignment authority. The reserved port address range is further divided to ports for “well-known protocols” (range 0 to 239 and 256 to 1024) and to registered ports (range 1025 to 49151). Nokia has reserved ports 5500 to 5599 for company-specific protocols. Port addresses in the dynamic address range 240 to 255 or in the dynamic and/or private address range 49152 to 65535 may be used freely by any vendor.

In order to receive these messages application must register themselves to SMS receive application. This can be done using smreg library. This library offers functions for registering and unregistering.

SMRegRegisterApplication() takes there parameters: **GeodeToken** of your application, a pointer to buffer containing “port numbers” you register for and number of messages you are registering. Port numbers must be ASCII coded hexadecimal numbers.

Code Display 13-3 Registering to receive “destination port” SMS Messages

```
@method TestProcessClass,  MSG_TEST_PROCESS_REGISTER
{
    GeodeToken appl;
    TCHAR messageProtocol[5];
    word protocol;
```

```
/* We register ourselves to listen port 43962 that is ABBA in hex.  */
    protocol = 43962;
    appl.GT_chars[0] = 'T';
    appl.GT_chars[1] = 'E';
    appl.GT_chars[2] = 'S';
    appl.GT_chars[3] = 'T';
    appl.GT_manufID = 11;

/* Let's ask smreg library to convert 43962 to ascii coded hex for us. */
    SMRegIntegerToHex( protocol, messageProtocol,
                        SMRITH_NULL_TERMINATE |SMRITH_WORD_LENGTH);

/* Now we have messageProtocol = "ABBA" */
    SMRegRegisterApplication( appl, messageProtocol, 1);
}
```

You can register more than one port by calling the function:

```
SMRegRegisterApplication( appl, "ABBADADA0055", 3);
```

Then you will receive messages ABBA, DADA and 55.

The function returns a **SMRegResult** value: SMRR_OK if your registration was succesful; otherwise it will be SMRR_FAIL.

SMRegIntegerToHex() function converts passed decimal integers to ascii coded hexadecimal strings. You pass a word containing integer to be converted, a character pointer where the result is stored and flags how the conversion is done. You can convert either byte length integers i.e. 0 to 255 and the result is two characters to word length i.e. 0 to 65535 and the result is four characters. By default the destination string is not null terminated, but if you pass flag SMRITH_NULL_TERMINATE null is added at the end of the string.

If you want to stop receiving messages you can unregister yourself. This can be done using **SMRegUnregisterApplication()** function. The parameters are identical to SMRegRegisterApplication. In addition SMMR_OK and SMRR_FAIL, this unregister function may return SMRR_APPLICATION_NOT_REGISTERED to indicate that you were not registered to listen this message.

The message routing table is stored to .INI file. You may freely modify it yourself, but it is not recommended. More than one application may want to



listen same messages, so writing .INI file direct may cause problems to other applications. The category is *SMSRouteTable* and port numbers represent keys underneath.

Code Display 13-4 INI file: SMSRouteTable example

Here TEST application wants to receive messages 00F0 and ABBA and TES2 wants to receive message ABBA.

```
[SMSRouteTable]
00F0 = “TEST”,11
ABBA = { “TEST”,11
          “TES2”,11 }
```

13.5 New Standard Monikers, Bitmaps

There are new ComplexMonikerType values and a new ComplexMonikerBitmap:

- CMT_MENU Useful for “Menu” buttons.
- CMT_SEARCH
- CMB_SEARCH

13.6 Non-exclusive Lists

GenItemGroups may now be of type GIGBT_NON_EXCLUSIVE, allowing the user to select more than one item at a time.

GenItemGroupClass now has two new messages which allow you to get and set which item has the “focus”--which single item the user currently has highlighted, as opposed to the selected items:

- MSG_GEN_ITEM_GROUP_GET_FOCUS_ITEM

- MSG_GEN_ITEM_GROUP_SET_FOCUS_ITEM

ContactLists:

If you are using a ContactList (from the Contact DB library), youmay set it up for multiple selections by giving it a ATTR_CONTACT_LIST_NON_EXCLUSIVE_LIST variable data field. To find out which records are selected in such a list, use MSG_CONTACT_LIST_GET_IDS_OF_SELECTED_RECORDS.

13.7 Incremental Searches

To allow the user to narrow the choices shown in a list when they type into a text entry, use two new classes in the Foam library: FilterListClass and FilterTextClass. Their API may be found in the include file foam.goh.

These classes are only available after Build 4. Check the "**F**" field of the version number, as described above.

13.8 Contact's URL and Password

The Contact DB Library has two new ContactFieldType values: CFT_URL and CFT_PASSWORD.

These values are only available after Build 4. Check the "**F**" field of the version number, as described above.

13.9 ContactList Searches

Normally, a ContactList will lose track of the user's current search if the focus moves to another object. To force the list to remember the current search, send it a MSG_CONTACT_LIST_PRESERVE_SEARCH_ON_NEXT_LOST_FOCUS before it loses the focus.



This message is only available after Build 4. Check the "**F**" field of the version number, as described above.

13.10 Unknown Phone Numbers

If the user enters a valid phone number in the search box of aContactList, but that number is not found, an application mayrequest that the ContactList send out a notification using a new variable data field.

Give the ContactList a ATTR_CONTACT_LIST_NOTIFY_NUMBER_ENTERED_MSG variable data field. This field's data is a message.

The message should have parameters:

```
void MSG_MY_CL_NOTIFY( MemHandle phNumber = cx);
```

The passed block contains the phone number string. The message's recipient is responsible for freeing the passed block.

When using the Contact Log library to log a contact, if you don't know the phone number, you can pass LEF_WILDCARD_NUMBER to **LogAddEntry()** to specify that this call's number should match any phone number.